University of Southampton

Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science

Advances in Hazard Management for Machine Learning based Safety-Critical Systems Operating in Unpredictable Environments

by

Haider M. Al-Shareefy
ORCID ID https://orcid.org/0000-0002-8291-5620

Thesis for the degree of Doctor of Philosophy

March 2026

University of Southampton

Abstract

Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science
Doctor of Philosophy

Appendix E AIC Systems Approach Key Concepts Theoretical Justification and Extended Specification

by

Haider M. Al-Shareefy

This Appendix is part of a PhD titled: Advances in Hazard Management for Machine Learning based Safety-Critical Systems Operating in Unpredictable Environments.

Appendix E provides the extended theoretical and methodological foundations underpinning the AIC systems approach developed in the main thesis. It is organised into nine sections that progress from foundational uncertainty concepts through to practical ML assurance guidance.

§E.1 characterises the three-layer uncertainty landscape facing the Architect and the machine, environmental, Architect, and ML model uncertainty, and establishes why conventional safety tools calibrated for bounded, Mediocristan systems may be insufficient when applied to open, Extremistan-type Compounded Uncertainty Problems in open Environments (CUPEs). It introduces the distinction between failed appreciation and failed control as two structurally distinct uncertainty failure modes, and develops a probability-theoretic account of complexity, complicatedness, and predictability that motivates the need for lateral thinking in safety analysis.

§E.2 examines typical and atypical AIC interaction timing, establishing the temporal dimension of AIC dynamics as a factor in scenario emergence.

§E.3 develops the safety and security implications of AIC interaction types, introducing the intrinsic hard hazard of appreciation, the structural vulnerability arising from any interaction in which a source has no control over its sink, alongside soft hazards in complexity fields, and formalises the distinction between hard and soft hazards in the AIC context.

§E.4 presents a controlled thought experiment comparing vertical and lateral predictive thinking applied to an autonomous security drone scenario. The experiment demonstrates that vertical thinking produces a metastable, axiom-sensitive prediction model that systematically neglects tail interactions, while lateral thinking produces broad coverage at the cost of convergence. The section concludes by inferring why the AIC approach constitutes a Transformative Thinking (T-thinking) synthesis of both modes, and maps all thought experiment activities and results to the thesis concepts defined in §3.5.3.

§E.5 operationalises lateral predictive thinking as a three-step general process, broadening base perspective, generating cross-domain analogies, and formulating auditable predictions, with iteration guidance.

§E.6 develops the Architect's Predictive Thinking Process (ArcPT) and its Structured Thinking Chain-of-Thought (STCoT) implementation, including the STCoT structure and the epistemic risks specific to engineered ML datasets.

§E.7 provides the full CuneiForm characteristic specification across eight machine training topics, pictorial horizon attitude, TOI aesthetic complexity, TOI motion and optical effects, background environment complexity, background object dynamics, TOI positioning, TOI 3D spatial orientation, and TOI pictorial distance, with traceability guidance linking images to their originating textual requirements.

§E.8 defines the Comprehensive Multi-Environment Operational Domain Definition (Multi-ODD) framework, combining land-based and air-based autonomous systems applications with an aleatoric uncertainty grading system for ODD conditions. §E.9 addresses ML assurance under novel scenarios, motivating ALARP-based epistemic risk reduction for out-of-distribution data, characterising the risk of unmitigated OoD data shifts, adapting the SHARCS assurance process for the Black Swan-driven training pipeline, and providing a systematic incremental pipeline for Black Swan training, testing, and safety regulation compliance justification.

Table of Contents

Table of Tables

Table of Figures

Research Thesis: Declaration of Authorship

Print name: Haider Al-Shareefy

Title of thesis: Advances in Hazard Management for Machine Learning based Safety-Critical Systems Operating in unpredictable environments

I declare that this thesis and the work presented in it are my own and has been generated by me as the result of my own original research.

I confirm that:

This work was done wholly or mainly while in candidature for a research degree at this University. Furthermore,

Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated. Furthermore,

Where I have consulted the published work of others, this is always clearly attributed. Furthermore,

Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work. Furthermore,

I have acknowledged all main sources of help. Furthermore,

Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself. Furthermore,

Signature: ..... Date: 16/03/2026

University of Southampton

Acknowledgements

Al-Shareefy is supported by a Thales EPSRC iCASE Award. Butler and Hoang are supported by the HD-Sec project, which was funded by the Digital Security by Design (DSbD) Programme delivered by UKRI to support the DSbD ecosystem.

Glossary of Terms

Term	Definition
Actions Matrix	A quick, first-pass simplified ArcMatrix tool used once relevant complexes are identified, constructed as an N^2 grid with complexes as both rows (sources) and columns (sinks).
AIC Perspective Shift	A deliberate cognitive intervention in which the Architect challenges the initially assumed typical operations AIC model (the intuitive, expected interaction schema) by altering interaction properties, AIC type (Appreciation ↔ Influence ↔ Control), direction (source ↔ sink reversal), effect type (supportive ↔ obstructive ↔ neutral), or combinations thereof. Perspective shifts enable Lateral Thinking by moving attention from what makes sense (expected, salient, control-framed) to what could happen if assumptions break (atypical, low-salience, influence/appreciation-framed).
AIC Systems Theory	A high-level conceptual approach originally proposed by William Smith for pragmatism about how purposeful systems relate to their wider environment.
Aleatoric Uncertainty	Uncertainty arising from inherent randomness or variability in the complexity being studied, which cannot be reduced by collecting more data or improving models. Represents the irreducible stochasticity of the real world.
Analytical Attention Distribution (ATD)	A histogram-based metric quantifying how the architect's cognitive or analytical attention is distributed across predicted situation types during a thinking session. ATD is constructed by tallying mention frequencies of predicted situations per ontology category (e.g., guidewords, control actions, interaction types), then computing (frequency/total situations) × 100 for each category.
Analytical Attention Management	The empirical allocation of cognitive or analytical attention across the space of possible situation types (hazard types, failure modes, interaction classes, situation categories) during modelling and analysis. It describes the architect's approach to thinking as a means of managing the architect's attention across noted complexity. Attention management is qualitatively examined via the Analytical Attention Distribution (ATD) histogram.
Anticipatory Thinking	A mental skill requiring imagining beyond the present perspective about a noted or expected complexity to an alternative perspective at a different time point, directly impacted by attention, memory, situational awareness (how much noted complexity is analysed vs. discarded), and domain expertise.
Appreciation, Influence and Control (AIC)	The three fundamental relationship types in AIC systems theory that characterise how a complex interacts with its environment. Appreciation (a) denotes no outward governance. Furthermore, the system is more influenced than it influences, with no causal path from the system to the target. Influence (I) denotes indirect, probabilistic governance via mediators, with lower certainty of causing immediate change. Control (c) denotes direct, high-certainty governance with bidirectional feedback and one-to-one change within coupling boundaries.
Architect Attention	The finite cognitive or analytical focus that the Architect allocates across the space of possible situation types, interaction categories, hazard modes, or failure scenarios during a thinking session, operationalised as the empirical distribution of mentions per ontology category (Analytical Attention Distribution, ATD).

Architect's Predictive Systems Thinking Approach (ArcPT)	A structured pragmatism approach that leverages the Architect's Analytical Attention Distribution (ATD) approach and chosen systems ontology (e.g., General Systems Model, GSM) to systematically reduce the complicatedness of an inquired scenario and assert the likelihood and impact of typical and novel scenarios. ArcPT follows a Systems Theoretic Chain-of-Thought (STCoT).
ArcMatrix	An umbrella term for Complicatedness Reduction Matrices, simple, repeatable matrix-based tools that force the inquirer (Architect) to make explicit predictions about what is interacting with what, typical or atypical, surfacing implicit assumptions that otherwise remain tacit.
asystem	A complex that undergoes a valid, purposeful OrgXP but fails to satisfy one or more of the four system-specific constraints
Black Swan (aleatoric view)	According to Taleb's definition, they are statistically rare, high-impact events that can be explained after they occur. The thesis accepts the notion that aleatory Black Swans are hard to predict. However, Black Swans are explainable after they occur. Furthermore, they are not impossible to imagine with creative thinking.
Black Swan (epistemic view)	Black Swans are most likely novel (they may or may not receive the least attention, are relatively original, and are judged atypical), have an extreme impact on system performance (e.g., ML reliability degradation), and are systematically imaginable using thinking approaches. It is usually hidden in plain sight of the observer. Epistemic Black swans can only be distinguished relative to a prior definition of White Swans. Epistemic Black Swans may or may not manifest as aleatoric Black Swan events.
Black Swan Pictorial Dataset	A pictorial dataset that satisfies four Black Swan properties: (1) Pictorial Novelty (relatively original via OoD dissimilarity + atypical operational scenario), (2) Least Attended (intentionally omitted from training dataset, zero or near-zero attention during initial development), (3) In-Context (within target operational domain defined by CuneiForm, e.g., railway track zone, not zoo or shelf), (4) Impactful (reduces ML reliability, model performance degrades on Black Swan dataset. Furthermore, if model passes, dataset is not Black Swan, indicating training coverage was sufficient).
Complex	A complex is any non-empty set of entities, rules or both (specifiable or not), e.g. Train track zone. By negation, the only thing that is not a complex is the empty set, no rules and no entities. A train tracks zone is a complex.
Complexity of a scenario	What is visible are the things. Given that a scenario is a complex of situations, Complexity is the objective, aleatoric uncertainty inherent in the scenario, measured by how the structure of interactions and entities changes over time. Complexity is an intrinsic objective state of the world (Normal, Long-tailed, or Uniform occurrence distribution of situations) that cannot be eliminated but can be better understood.
Complicatedness of a scenario	What is invisible are the rules. Complicatedness is the set of rules that govern and explain complexity. Rules can be present or absence. They can be present but work in different ways, leading to unpredictability. Rules can complement each other, leading to harmonious predictability.
Compounded Epistemic Uncertainty Problem (CUPE)	A scenario in which living agents, computational agents, deterministic systems, natural systems, and a highly volatile open operational environment coexist, such that perturbations to any constituent element produce emergent interactions and behaviours unpredictable to the inquirer due to hidden, atypically salient complexities that interact and compound non-trivially as domain complexity escalates.
CuneiForm	A CuneiForm is a pictorial requirement that serves as an abstract image template that guides dataset collection and validation. In context images are those that satisfy all or most CuneiForm dimensions, ensuring explainable mapping from requirement to CuneiForm to dataset to ML training example.

AIC-Matrix	A detailed ArcMatrix extension that decomposes each source-to-sink relationship in the Actions Matrix into a structured emergence record, supporting justifiably mapped pragmatism about intelligent and non-intelligent behaviour. For each N^2 cell, the inquirer records: Supra Source (larger complex source belongs to), Supra Source PrimeP (its primary purpose), Source Goal (Goal/no-goal/unknown), Goal type (A/I/C/unknown), Source Action (Action/no-action/unknown), Action type (A/I/C/unknown), Effect (supportive/obstructive/neutral).
Disorganisational Experience (DisOrgs)	a dialectical opposite to OrgXP where rules are randomly changing or no rules apply in a complex. Rules may or may not be present, and if present, they are incomprehensible, leading to unpredictable complexity.
Epistemic Uncertainty	Uncertainty that can be reduced by collecting more data, improving models, increasing the number of views or refining knowledge about the system. Arises from a lack of knowledge, incomplete data, incorrect assumptions, or ignorance about a scenario's complexity. In the context of a thesis, epistemic uncertainty complicates the Architect's task: predicting novel situations is difficult due to knowledge gaps about which situation types exist, how they interact, or which rare events matter.
Excess Kurtosis	A statistical measure that captures the possible presence of Black Swan events. In the context of Analytical Attention Distribution ATD, excess kurtosis is used to characterise the ATD histogram's shape (tail weight, interpreted as the architect's attitude towards Black Swan scenarios based on the nature of attention distributed).
General Complex	A set of elements in particular situations (regardless of nature) that may or may not constitute a system, depending on the inquirer's perspective.
General Systems Model (GSM)	A set of general systems rules (conjectures) about a complex, if noted and validated, may systemise the complexity for the inquirer. GSM can be updated over time and in terms of complexity. Novel scenarios have little or no prior literature. Furthermore, GSM serves as an auditable guarantor of the plausibility or likelihood of predictions generated by STCoT, especially novel scenarios.
Goals and Actions Impact Types	Classification of how goals and actions affect other goals/actions within AIC interactions, characterised by impact type (supportive, obstructive, neutral) and effectiveness (effective, ineffective, absent).
Guiding Prompt (GP)	A prediction-guiding instruction that directly instantiates a General Systems Model (GSM) rule in a given scenario, providing procedural steps or heuristics for the Architect to execute a thinking step. Guiding Prompts translate abstract Predictive Queries (PQs) into actionable cognitive tasks, specifying what to identify, characterise, or predict.
InC (In-Context) of a CuneiForm	Pictorial datasets that conform to a CuneiForm, which captures a target operational domain and problem space characteristics.
InD (In-Distribution) Pictorial Datasets	Pictorial datasets whose visual feature distributions closely match another dataset (for example test dataset is ID with the training and validation datasets), indicating that test examples are drawn from the same or similar underlying distribution (abstracted by CuneiForms) as the model's learning data. High in-distribution (InD) performance accuracy often gives a false sense of security about ML's actual performance in safety-critical applications because it does not test the model's ability to handle rare, atypical, or adversarial scenarios (Black Swans) that may arise in real-world deployment.

Lateral Thinking	A Uniform ATD-based approach that assumes all possible events are equally frequent (regardless of noted or not), treating notably frequent observations as incorrectly and incompletely characterising the inquired complexity, thereby discarding salience-driven prioritisation. Shifts the architect's attention non-sequentially, without scope constraints, exploring indirect, non-obvious, out-of-scope, or seemingly unrelated contributing factors. Assumes all situations are equally probable and impactful (no Black/White Swan distinction by definition).
Long-tailed ATD Mode	An attention-allocation regime in which the architect's attention behaves analogously to a heavy-tailed (long-tailed) distribution: a small number of contributing situations still receive the most attention (the head), but the tail retains non-negligible allocation because rare categories are treated as plausibly consequential and must be explicitly explored and designed for. The architect assumes the problem's event space is dominated by low-frequency contributors that can still govern outcomes (Black Swans, emergent couplings, OoD data shifts).
Machine Training Class	A single CuneiForm is considered as a class or lesson within a Machine Training Syllabus, comprising multiple pictured situations (depicted pictorial behaviours) for the machine learning model to be trained on and to pass tests. Each class represents a distinct abstract pictorial scenario defined by specific CuneiForm dimensions (e.g., TOI type, aesthetic complexity, motion trajectory, background objects, positioning, orientation, optical distance).
Machine Training Syllabus	A comprehensive set of training, validation, and testing classes (CuneiForms) that collectively map to a high-level ML training concept or safety requirement. The syllabus defines the curriculum of pictorial scenarios the machine learning model must learn to recognise, organised into discrete classes (lessons) covering diverse operational scenarios (typical and Black Swan).
Machine Training Topics	Foundational dimensions that characterise the real-world complexity a perception model must learn, subdividing Machine Training Classes into specific perceivable situations. Topics define the ontology of pictorial variability, ensuring systematic coverage of sources of epistemic uncertainty. The eight topics are: TOI type, aesthetic complexity, motion trajectory, background objects, positioning, orientation, optical distance, and horizon orientation.
Negative Affordances	Properties, features, or capabilities of a complex that are misused to hinder, obstruct, or oppose the manifestation of desired outcomes, goals, or purposes within an AIC interaction. Negative affordances correspond to obstructive effect types: the source's action impedes the sink's achievement of its PrimeP or introduces obstacles to goal attainment.
non-system	a complex that results from a Disorganisational Experience
Normal ATD Mode	An attention-allocation regime in which the architect's analytic effort behaves analogously to a normal (Gaussian) distribution: attention concentrates around a small set of high-salience, most likely/most typical contributing situations (the mean region), while attention allocated to increasingly low-salience situations fall off rapidly (thin tails). Operationally, the architect assumes that a finite set of common scenarios largely captures the relevant complexity (e.g. hazard/interaction) space, and that rare categories can be treated as negligible or deferred.
Novelty reduction rate (Novelty reduction rate rate)	is a quantitative metric that measures the extent to which a thinking approach (e.g., HAZOP, STAMP, AIC-Core STCoT) successfully reduces the space of unexplored, atypically salient scenarios (Black Swan candidates) by systematically surfacing novel situations during hazard analysis or design sessions.

Novelty Test	A heuristic that evaluates whether a predicted situation, scenario or hazard is novel by assessing three criteria: (1) Originality: absence or low frequency in prior literature or historical databases (assessed via corpus comparison), and (2) Typicality: expert judgment classifying the situation as atypical (counterintuitive, not standard component-level cause-and-effect, or outside expected relevance).
OoC (Out-of-Context) Outside the Context of a Defined CuneiForm	Pictorial datasets that fall outside the target operational domain or problem space defined by a CuneiForm's characteristics (horizon, TOI aesthetic, motion, background, positioning, orientation, distance). OoC images may depict the same target object of interest (TOI) but in irrelevant or non-operational contexts (e.g., a drone photographed in a museum, on a retail shelf, or in a laboratory rather than in the railway track zone operational environment).
OoD (Out-of-Distribution) Pictorial Datasets	Pictorial datasets whose visual feature distributions diverge significantly from one another (e.g. Test dataset OoD with the training and validation datasets), measured via dissimilarity metrics (e.g., perceptual hashing, Jaccard IoU on unique pHashes, distribution divergence tests). OoD datasets represent data shifts, changes in input distributions that may degrade ML model performance if the model has not been trained to generalise beyond in-distribution (InD) examples.
Organisational Experience (OrgXP)	A process scenario occurring within a complex in which entities repeatedly apply a set of governing rules in response to internal or external aleatoric uncertainty perturbations. The process is characterised by structural or dynamic relationships among entities and recurs over time (or steps) within the same complex, resulting into either typical or atypical whole.
Originality Assessment	A quantitative evaluation of whether a predicted situation is novel (statistically infrequent) relative to a defined reference population, corpus, or prior literature, operationalised by comparing the frequency of the predicted situation in the current analysis to its frequency in historical analysis (like failure databases, risk assessments, or previous case studies). Inspired by Wilson's uncommonness method, a situation is classified as original if it appears zero in prior sources, and not original if it appears at least once.
Pictorial Hazard	The absence of visual information in training, validation, or testing datasets could prevent harm caused by the machine learning perception system, arising from insufficient epistemic variety in pictorial scenarios that the model must learn to recognise. Pictorial hazards are sources of epistemic uncertainty in ML development.
Pictorial Novelty	A composite property of pictorial datasets indicating that they are relatively novel according to the Novelty Test criteria applied to visual/image data: (1) Pictorial Originality (out-of-distribution, OoD, with respect to typical operations datasets, measured via dissimilarity metrics like perceptual hashing), and (2) Pictorial Typicality (derived from atypical operational scenarios or situations judged atypical by experts, e.g., camouflaged Adversarial Drones vs. regular quad-copters).
Pictorial Originality	A quantitative assessment of whether a pictorial dataset is out-of-distribution (OoD) with respect to a reference dataset representing typical operational scenarios (White Swans), measured via dissimilarity metrics such as perceptual hashing (pHash) followed by Jaccard Index of Intersection over Union (IoU) on unique pHash values.

Pictorial Typicality	A qualitative classification of whether a pictorial dataset is derived from typical (expected, routine, high-frequency) or atypical (counterintuitive, rare, adversarial) operational scenarios, assessed via explainable mapping to atypical situations or interactions discovered during design and expert judgment on operational context. A dataset is pictorially typical if it depicts standard, expected system behaviours. A dataset is pictorially atypical if it depicts rare, counterintuitive aspects that are not typically considered in standard operational planning.
Positive Affordances	Properties, features, or capabilities of using a complex that enable, facilitate, or support the manifestation of desired outcomes, goals, or purposes within an AIC interaction.
Predictive Query (PQ)	A prediction-provoking question derived from a General Systems Model (GSM) rule, designed to guide the Architect's attention toward specific types of interactions, goals, or situations relevant to the inquired scenario.
Predictive Systems Thinking	Structured pragmatism that employs systemic regularities (general systems principles) to forecast novel complexity (e.g., risks, hazards, failure modes) before they manifest, using model-first (top-down) deductive logic. Distinguished from ad hoc brainstorming by: (a) imposing General Systems Model (GSM) conjectures on analysis to enable predictions, (b) following Systems Theoretic Chain-of-Thought (STCoT) to instantiate GSM via guided prompts.
Predictive Thinking	The strategic manipulation of attentional allocation to reduce epistemic uncertainty (gaps in the architect's knowledge) about yet-to-be-salient aleatoric uncertainty (real-world variability) in a scenario. Predictive thinking involves generating statements or claims about complexity that may or may not be currently noted, using structured thought steps that guide attention across typical and atypical situations.
Predictive Thinking Modes	Categories of cognitive strategies that architects employ when pragmatism about complexity, characterised by their Analytical Attention Distribution (ATD) profiles and prioritisation logic. The three primary modes are: Vertical Thinking (Normal ATD: systematic, depth-first, prioritizes salient cause-effect chains, efficient but risks tail-neglect), Lateral Thinking (Uniform ATD: exploratory, breadth-first, treats all situations equally, maximizes novelty discovery but may be inefficient), and T-Thinking (Long-tailed ATD: hybrid, balances depth on head situations with deliberate tail exploration, iteratively updating priors). Mode selection influences which hazards are surfaced: Vertical misses atypical Black Swans. Furthermore, Lateral may overthink trivial interactions. Furthermore, T-Thinking adapts dynamically.
Predictive Thinking Pipeline	A conceptually interrelated series of Systems Theoretic Chain-of-Thought (STCoT) steps and other thinking steps, whose title represents the primary prediction goal expected to be asserted by the interrelated constituent steps.
Scenario	A series of situations involving sequences of interactions among complexes, expressed as chains of source-action-sink relationships that describe how events unfold over time within a defined context.
Situation	A perceptible state or condition involving a single complex of things behaving in some context (environment), characterised by the complex's dynamic or static properties.

STCoT Structure	The formalized template for a Systems Theoretic Chain-of-Thought, comprising: (1) STCoT Title (descriptive title representing inference goal), (2) STCoT Input (description of inquired situation/scenario), (3) General Systems Model (GSM) (set of general systems rules/conjectures that output must preserve, defining acceptable predictions), (4)A series of Predictive Queries + Guiding Prompts + Step Completion Criteria per thinking step, (5) STCoT Output Prediction (series of predictions from Architect assertion: predicted situations are pragmatically consistent, structurally complete, acceptably likely and impactful, with respect to GSM validity).
Structured Lateral Thinking Approach	A structured operationalisation of Lateral Thinking that systematically facilitates analogising (asserting comparisons T is like B to project relational structure from base domain B to target unrelated domain T, generating inferences). Three-step procedure: Step 1: Broaden base perspective. Step 2: Generate analogies with unrelated domains. Step 3: Formulate predictions.
Supra-System. Furthermore, Supra-Complex(es)	The larger system or complex within which the complex under inquiry exists as a subsystem or component.
System	A system is a predictable complex whose behavioural dynamics follow a consistent set of rules. Fr example, any complexity generated by Conway's Game of Life rules would be considered a system. Any stochastic random complexity would not be considered a system.
system	a complex undergoing a predictable, purposeful, OrgXP of definable entities where rules are definable and verifiable. When the complex is perturbed, the rules guarantee convergence to stable order.
The Architect	A subclass of inquirer, an agent (individual, collaborative team) that employs systemic and systematic methods to construct predictive mental models intended to make sense of and forecast the behaviour of an inquired complexity, in order to design a specific solution space satisfying design intents within a complexity.
The Inquirer and the Inquired Situation	The inquirer is a guided predictive-thinking agent (human architect, design team) that employs a thinking approach to conjecture about and make predictions regarding an inquired situation, a perceptible situation involving a single complex behaving in some context (environment) that the inquirer seeks to make sense of.
Thinking Step	A thought-provocative query and a prompt that influences the Architect's predictive thinking approach (specifically the Architect's Analytical Attention Distribution, ATD), to make an informed prediction based on some general systems perspective.
T-Thinking	A balanced, Long-tailed ATD-based approach that strategically integrates Vertical and Lateral Thinking during analysis, treating the inquired complexity's head (frequent interactions) and tail (rare interactions) as a dynamic continuum.
Typical Operations Datasets	Pictorial datasets representing expected, routine, high-frequency operational scenarios (White Swans) that are expected and characterised by typical environmental conditions, object appearances, and interactions.
Typicality Assessment	An expert-driven qualitative evaluation of whether a predicted situation is typical (readily identifiable by a competent, engineer, routine, expected within the problem space) or atypical (counterintuitive, outside standard component-level cause-and-effect framing, not typically notable, requiring a perspective shift or lateral thinking to surface).
Uniform ATD Mode	An attention-allocation regime in which the architect treats all possible situations or interaction classes as equally likely to persist over time and equally worthy of analytical effort, allocating approximately equal time, questioning depth, and evidence-seeking to each bin in the ontology.

Valuable Minimum Variety Quality	A demonstrable pictorial variety qualification approach is defined as the minimum diversity of pictorial scenarios required to mitigate pictorial hazards (absence of visual information that could prevent harm) while enabling justifiably mapped validation that safety requirements are faithfully captured. A valuable image: (a) traces unambiguously to a descriptive parent requirement, (b) contains at least one feature addressing each of 8 training topics, constituting a minimum variety coverage.
Vertical Thinking	Objectivity-only framing of reality. Characterised by a Normal ATD-based approach that assumes that notably frequent noted interactions fully characterise the inquired complexity, treating any other possible events (if not typically notable by frequency) as negligible or rare.
White Swan (aleatoric view)	According to Taleb's definition, they are common, easy-to-identify and explain events.
White Swan (epistemic view)	Typical operations, high-salience scenarios that are expected, well-documented in prior literature, and readily identified by competent objectivity-oriented engineers.

Appendix E AIC Systems Approach Key Concepts: Theoretical Justification and Extended Specification

The primary purpose of the AIC systems approach is to forge an Ideal System out of a chaotic, harder-to-predict, and more complex phenomenon involving autonomous, non-autonomous, living, and non-living systems into an easier-to-predict and more comprehensible system. The whole makes up the architect's epistemic complexity field where all that needs to be known exists. However, to achieve complete coverage of all that needs to be known, the architect requires a systematic and iterative approach, along with a comprehensive ontology, to reduce epistemic uncertainty. Architectural epistemic uncertainty can stem from gaps in knowing what needs to be observed and understanding the complexity of observed field dynamics (past, present, and future). Thus, a solution for this (as well as the other sources of epistemic uncertainty) is a comprehensive categorical system that intends to enable the architect to think laterally and vertically.

To achieve this, we require an ontology that maps perceived information into reasoned, categorical knowledge. Hence, the AIC systems approach is a systematic knowledge acquisition activity based on general universal systems theory known as AIC Systems Theory. In simple terms, AIC stands for Appreciation, Influence and Control. The general premise here is that all systems in the universe, in any scenario, form three categories of relationships: AIC. As described by its founder, Dr Smith (see literal review), the current system theory describes AIC as a form of purpose and actions. We adapt the theory to make it more practical for systems engineering and formulate the following concepts, some of which are inspired by AIC philosophy. In contrast, others are derived from our understanding of general systems and interpretations of prior art.

E.1 Environment, Architect and Machine Uncertainty

This section consolidates the conceptual work undertaken to clarify what “uncertainty” means within the AIC systems approach and why it matters for safety-critical autonomous systems operating in unpredictable environments. The discussion is motivated by recurring challenges encountered when attempting to (i) express the discovery and mitigation of low-frequency, high-impact operational conditions (often described as Black Swan events) in a form suitable for assurance; (ii) justify the linkage between such mitigations and confidence arguments addressed to regulators and stakeholders; and (iii) distinguish between different sources of difficulty in open problem domains, including the concepts of complexity and complicatedness. The central premise is that the safety-assurance difficulty in such domains is frequently less

about enumerating failures of a single component and more about managing uncertainty across interacting systems, their purposes, and the assumptions embedded in models, datasets, and design decisions.

These considerations motivate treating uncertainty not merely as a contextual complication but as a primary design concern. In particular, the approach may be interpreted as addressing an uncertainty-reduction problem: reducing the architect's uncertainty through structured analysis and predictive reasoning, and reducing the machine's uncertainty through traceable, systematically engineered datasets. Within this framing, a recurring challenge concerns predictability: the extent to which an architect (and, by extension, an assurance authority) can form defensible expectations about which operational conditions may arise, and how likely (or how consequential) those conditions may be over the lifetime of deployment. This does not require the claim that complete certainty is attainable; rather, it motivates methods that can make assumptions explicit, expose gaps, and support iterative refinement under evolving evidence.

In more traditional engineered settings, it can sometimes be practical to treat some low-frequency conditions as negligible when their impacts are demonstrably bounded and when operating assumptions remain stable. For autonomous systems deployed in open environments, this simplification may be less defensible, because atypical perceptual conditions and rare system–environment interactions can plausibly lead to disproportionate behavioural deviations. Illustrative incidents in the autonomous-vehicle domain (e.g., widely discussed cases involving misclassification under unusual lighting and occlusion) indicate that failures may arise from the interaction between perception, data coverage, and operational context rather than from a single, easily localised control defect. Accordingly, the present discussion uses such examples as motivation to examine how uncertainty accumulates across the environment, the architect's modelling assumptions, and the machine's learned behaviour.

Public investigations of safety incidents have also highlighted organisational and process factors (e.g., governance, safety culture, validation practices) that can shape the quality of assurance claims and the robustness of deployed behaviour. Such observations can be interpreted as reinforcing a systems-oriented view: safety outcomes emerge from the coupling of technical design choices, operational constraints, human oversight arrangements, and environmental conditions. Within an AIC-oriented interpretation, these couplings may be analysed in terms of how systems appreciate, influence, and control one another, and how breakdowns in these relations can contribute to undesirable outcomes. This motivates the need to characterise uncertainty more precisely and to clarify how the AIC framing can structure predictive reasoning about rare or novel conditions.

Further motivation is provided by existing assurance-oriented guidance that characterises autonomy as operating under partial information. For example, SACE (Safe Assurance of Autonomous Systems in Complex Environments) emphasises the distinction between the operating environment's real state and the autonomous system's belief state, noting that decisions are made under uncertainty about which state currently obtains. This distinction invites the foundational question of how the space of relevant environment states can be identified and represented for the purposes of design, testing, and assurance.

> "The different possible scenarios and environmental states that relate to the decision can then be identified by considering the real-world state and the belief state of the AS at the point at which the decision is made, along with each of the possible options. The real-world state represents the actual state of the operating environment, whilst the belief state represents the understanding that the AS has of the state of the operating environment".

This motivates the question of scope:

> **How can the set of relevant operational environment states be identified with sufficient completeness to support assurance arguments?**

In open operational contexts, the set of relevant states may include not only the autonomous system's internal modes and failure conditions but also the states and behaviour of other systems in proximity (e.g., infrastructure, other vehicles, bystanders, environmental processes) and the couplings among them. The challenge is not simply combinatorial; it concerns the credibility of the abstractions used to decide which distinctions matter for safety, and how those distinctions may change under deployment. In AIC terms, this motivates the analysis of which external systems must be appreciated (because they cannot be controlled), which can be influenced (through indirect mechanisms), and which can be controlled (through direct actuation or enforceable constraints).

> **This illustrates how uncertainty may be distributed across different agents and artefacts, including the architect's modelling assumptions, the autonomous system's belief state, and the operational environment itself.**

A salient implication is that the autonomous-systems architect is often required to reason about a broader "complexity field" than the system-of-interest boundary alone, because safety-relevant behaviours can be mediated by external systems and their interactions. The following general expression captures this dependency:

> *In open operational settings, changes in the real-world state shape the autonomous system's belief state (via sensing, interpretation, and learned representations), and the autonomous system's actions can, in turn, alter the real-world state. Safety-relevant*

> *failures may therefore arise from mismatches between real-world state and belief state, as well as from interactions among multiple systems pursuing their own operational purposes*.

Illustrative incident analyses can be reinterpreted through this lens: the safety outcome depends on the coupling between the autonomous system's perceptual–decisional pipeline, the supervision and operational procedures adopted during testing or deployment, and environmental/infrastructural conditions (e.g., lighting, visibility, road layout). Rather than attributing causality to individuals, such incidents may be used to motivate the broader point that the operational environment includes multiple interacting systems whose purposes and constraints can shape the set of plausible hazardous scenarios. The AIC framing is introduced to support more systematic consideration of such dependencies.

The intention of this discussion is not to provide a detailed forensic account of any particular event, but to highlight a systems-level implication aligned with the SACE distinction between real state and belief state: the autonomous-systems architect cannot restrict attention to the autonomous system alone. The behaviours, constraints, and potential failure modes of other systems in the operational environment can become relevant through indirect couplings. Consequently, the design approach benefits from concepts that encourage explicit reasoning about what is being appreciated (i.e., depended upon without direct governance), what can be influenced, and what can be controlled.

Implication for the AIC framing

The phrase "failures of real-world systems to achieve their intended purposes" is used here as a modelling device rather than an assertion about intent in every case. In general systems theory, "purpose" is often employed as an explanatory abstraction that supports prediction of system behaviour under constraints and interaction. Within the AIC framework, purpose can be treated as a driver that shapes which relations are predominantly appreciative, influential, or controlling in a given context. For safety-critical autonomous systems, this abstraction is used to prompt the architect to consider how other systems may act to preserve their own operational objectives, and how such actions may alter the autonomous system's belief state or create new constraints on control. In this sense, AIC offers a structured way to represent dependence (appreciation), indirect coupling (influence), and direct governance (control) when reasoning about the wider operational field.

E.1.1 Failed appreciation vs failed control uncertainty.

A further distinction concerns whether a given unsafe outcome is more appropriately understood as a breakdown of control, or as a breakdown of appreciation that subsequently propagates into inappropriate control decisions.

Under what conditions should an observed failure be conceptualised as primarily a control failure, versus primarily an appreciation failure?

In many autonomy-related incidents, the immediate control mechanisms (e.g., actuation, planning, braking/steering execution) may appear to operate in accordance with their internal logic, while the upstream interpretation of the environment is inadequate for the encountered context. This motivates an analytic separation between (i) control failures in the narrow sense (where the intended control policy is not executed) and (ii) appreciation failures, where the system-of-interest—or the broader socio-technical system that produces the system's training data and operational constraints—fails to represent or anticipate relevant aspects of the operational field. Because learned controllers and perception-driven decision pipelines depend on what has been represented in data and models, control adequacy is often contingent on appreciation adequacy.

A useful way to characterise this phenomenon is through *epistemic ignorance*: a situation in which new information does not simply reduce uncertainty but instead reveals previously unrecognised gaps in the underlying classification or modelling scheme. For example, an architect may initially adopt a plausible partition of scenarios (e.g., separating threats by apparent approach direction), and later observe evidence that the partition itself is insufficient (e.g., decoys and coordinated behaviours that exploit the partition). In such cases, uncertainty may temporarily increase because the architecture of the belief system requires revision. Within the AIC framing, this can be interpreted as an appreciation limitation: the set of appreciated constraints and dependencies has not yet been represented at a granularity adequate for the operational field.

In that case, we cannot say with utmost confidence that it was a control action failure. It was instead an appreciation failure on the part of the architect, whose understanding of the problem domain failed to consider such a scenario. From the victim's perspective, they were in complete control of themselves and reliably crossing the road; no control action had failed, but rather a lack of appreciation for the environment. Both the architect, the machine and the victim's knowledge base can be described as having experience "epistemic ignorance" which is a general flaw or intellectual fault of knowledge systems. However, the architect, and by extension their training datasets, and by extension the

reliability of the trained model, can be understood to suffer from an inherent deficiency that leads to “epistemic ignorance”. We understand this deficiency as a general assumption associated with any finite knowledge system, in particular the architect, by proposing the following general assumption and explanation:

Finite knowledge systems (including human mental models and engineered representations) are necessarily incomplete with respect to open operational domains. This incompleteness can often be reduced, in practice, by systematically expanding coverage across relevant classes of information and by refining the partitioning scheme used to represent the domain. Nevertheless, in open-ended settings, it is prudent to assume that residual gaps may remain because new classes of relevant conditions can emerge (or be recognised) over time, and because it is rarely feasible to obtain exhaustive evidence across all conceivable contexts. Within this limitation, lateral and vertical predictive thinking are treated as complementary: lateral thinking supports the discovery of overlooked classes and analogies, while vertical thinking supports rigorous refinement and traceable specification once classes are proposed.

Under this interpretation, the architect’s predictive thinking and knowledge base shape decisions about what is represented in datasets and models, and these representations shape the reliability envelope of learning-based components under deployment. Similarly, human stakeholders in the environment act on the basis of their own belief states and expectations. Because these belief states are finite and context-dependent, residual uncertainty should be treated as a persistent design consideration rather than as an anomaly. This motivates the use of the term *influence* (rather than *control*) when describing many socio-technical couplings: changes in one system’s assumptions can shift the conditions under which another system’s control actions remain appropriate, without implying a direct, deterministic governing relation.

With that in mind, we can understand the requirement for successful and safe operations among the architect, the training datasets and the victim as follows:

Illustrative mapping to AIC roles (human road user): A road user’s safe behaviour can be analysed as an interplay of appreciation (perceiving and interpreting traffic conditions, including cues about vehicle behaviour), influence (signalling intent to other road users through movement, positioning, and compliance with social/traffic norms), and control (executing physical actions such as walking, stopping, or accelerating). Importantly, some factors (e.g., the autonomy status of approaching vehicles, visibility conditions, infrastructure design) are typically outside the individual’s control; they can only be appreciated and accommodated. In this sense, appreciation uncertainty concerns incomplete or unreliable understanding of the environment, whereas control failures concern the inability to execute intended actions (e.g.,

due to physical impairment or mechanical malfunction). The distinction is relevant because mitigations differ: improving appreciation often involves improved signalling, infrastructure, information provision, and training/awareness; improving control often involves mechanical or physiological reliability measures.

Illustrative question (appreciation-oriented hazard elicitation): What additional, low-probability conditions affecting a road user's ability to act (e.g., sudden physical impairment, distraction, or misperception) could plausibly interact with limited machine perception to produce unsafe outcomes? Framing such questions in terms of appreciation encourages consideration of contributory conditions that may be neglected if analysis is restricted to "failed control actions" alone.

From this perspective, the following generic observations (rather than incident-specific attributions) can be made:

- A road user may have limited ability to appreciate the operational characteristics of nearby vehicles (including autonomy-related behaviours) when such characteristics are not readily observable or signalled.
- If appreciation is degraded (e.g., due to poor visibility, ambiguous cues, or incomplete mental models), influence attempts (e.g., crossing decisions) may become poorly calibrated relative to the actual traffic dynamics. Conversely, the operational context may impose appreciation demands on both human road users and autonomous systems that are not adequately supported by infrastructure or operational procedures.

Environmental conditions such as illumination, contrast, occlusion, and weather may further moderate appreciation for both humans and machine perception systems. In machine-learning perception, these conditions can introduce aleatoric variability in the sensor signal and can expose epistemic gaps if the training data does not include sufficiently representative instances. Accordingly, infrastructure and operational measures (e.g., lighting standards, vehicle signalling, headlight policies) may be considered as part of the wider system-of-systems design space, although the appropriate mitigations are context-dependent and subject to trade-offs.

- Operational lighting and visibility constraints can affect the reliability of perception, potentially increasing the likelihood of false negatives for both human observers and computer vision systems.

- For learning-based systems, these visibility regimes motivate explicit consideration in dataset design (e.g., representative variations in illumination and sensor exposure), as part of an evidence-based argument that the training data reflects plausible deployment conditions.

Illustrative mapping to AIC roles (learning-based perception system): For an autonomous system to achieve safe behaviour, a perception component must support appropriate downstream control decisions (e.g., braking, steering, speed regulation) under the conditions in which it is deployed. In many operational settings, the perception component cannot control which scenarios are encountered; rather, it must appreciate them sufficiently to support robust decision-making. In turn, the quality of this appreciation is shaped by the training and validation regime designed by the architect. The AIC framing therefore encourages explicit representation of (i) what the model must appreciate in the operational field, (ii) which variables influence the model's inputs but remain outside its control, and (iii) which control actions are taken as a consequence of the appreciated state.

Within this interpretation, plausible machine-side contributors to unsafe outcomes include:

- Insufficient appreciation of a context-relevant visual configuration (e.g., partial occlusion, unusual illumination, atypical object presentation), particularly if such configurations are under-represented in training evidence.

This distinction also motivates a methodological question: if a systems engineering method foregrounds only a narrow subset of failure concepts (e.g., primarily control deviations), how reliably does it prompt consideration of appreciation- and influence-related vulnerabilities in open environments?

Many established hazard analysis approaches operationalise safety through structured prompts and pre-defined classes of deviation, failure mode, or functional interaction. In practice, these prompts can be highly effective for bounded systems whose primary uncertainty concerns known deviations in well-understood control structures. In open environments with learning-based components, however, some safety-relevant deficiencies may arise from limitations in what has been represented and anticipated (appreciation) and from indirect couplings that do not resemble direct control loops (influence). The AIC adaptation used in this thesis is therefore positioned as a complementary lens: it aims to broaden the space of questions asked during analysis by explicitly distinguishing appreciation, influence, and control relations, while remaining compatible with established systems engineering methods when they are applicable.

E.1.2 AS open-ended belief state uncertainty

Rationale for the term "open-ended belief state"

The term "open-ended belief state" is used to emphasise that many learning-based systems do not operate with a closed-form, fully enumerated specification of each concept they must recognise. For example, when a perception model is trained to detect a class such as "person", the operational meaning of that class is induced from examples and is therefore sensitive to context, data coverage, and the representational capacity of the model. This differs from a deterministic controller in which acceptable ranges and rule-bounded responses may be explicitly specified. In an open environment, the space of possible appearances and contexts is effectively unbounded, and the belief state formed by the autonomous system can therefore be treated as open-ended with respect to future, unobserved variations.

Under this view, "openness" refers to the breadth of plausible operational variations, including both variations intentionally targeted during design (e.g., known environmental conditions) and variations not explicitly anticipated but nonetheless encountered (e.g., novel combinations of occlusion, lighting, viewpoint, and background structure). The aim of dataset and assurance design is not to claim exhaustive closure, but to provide defensible evidence that key classes of safety-relevant variation have been considered and that residual uncertainty is managed through monitoring, constraints, and iterative refinement.

So, how do we understand the parts of the problem articulated by SACE?

To interpret the SACE distinction between real-world state and belief state in engineering terms, several linked concepts require clarification:

- Openness of the operational environment, and the corresponding open-endedness of the autonomous system's representational categories.
- Real-world states: the external conditions and dynamics that obtain, irrespective of what is perceived.
- Belief states: internal representations used by the autonomous system to select actions under uncertainty.
- Uncertainty propagation: how mismatches between real state and belief state, and how changes in the environment, affect decision reliability over time.

In addition, assurance depends on the architect's uncertainty about both the environment and the autonomous system's behaviour under that environment, including uncertainty induced by modelling choices, abstraction boundaries, and evidence limitations.

- Architect epistemic uncertainty: uncertainty associated with the adequacy of the architect's operational model and the completeness of the evidence used to justify design decisions.

We look at the components of the problem from the standpoint of “Architect vs Problem”, so we understand the situation as:

- Architect predictive capability problem (ArcPC).

In the above articulation, we recognise three main problems to manage:

- The operating environment is open and complicated with a multitude of real-world state types (dominated by hidden Black Swan real-world states).
- We expect the engineered AS to behave or execute its designed belief states as expected in those black swans.
- The architect's belief system that predicts and defines those states.

In this context, the **architect's belief system** refers to the internal mental model, predictive assumptions, and reasoning framework the architect uses to understand, anticipate, and design for both the real-world operational environment and the behaviour of the autonomous system (AS). This internal mental model can be in two distinct situations when facing complexity: **Clear** or **Confused**. These two situations determine the quality of the engineering judgement and design decisions related to the final AS design.

It encompasses the architect’s current knowledge, expectations, and uncertainties about how the AS will perceive and act in open, unpredictable scenarios, particularly in relation to unknown or Black Swan events. So, from an architect's belief-system development perspective, the more categories of parts and the more parts in each category or type are involved in the open environment and the behaviour of their engineered system, the faster the architect's epistemic uncertainty about the robustness of their assumptions compounds. Hence, we use the term “Compounded Uncertainty Problem”.

The complexity of the physical environment includes inherent randomness, such as irregular lighting conditions due to wind and rain (read section 2.1.1.5 for background literature on uncertainty and ignorance). Such randomness is a form of aleatoric uncertainty that cannot be physically reduced with more added elements and factors. The physical environment uncertainty is then translated into informational uncertainty in ML training and validation images at the pixel level. However, the more examples we capture, the more sources of environmental aleatoric uncertainty are captured, and the less epistemic uncertainty (about the environment) becomes apparent in the training and validation datasets. However, epistemic uncertainty in training and validation datasets depends entirely on the architect's epistemic uncertainty about the environment and abstraction/design skills. While we cannot influence or control the open environment, we can influence and enable the architect to make the most out of their predictive capability using an ontology and process that helps to minimise sources of epistemic ignorance (a risk associated with epistemic uncertainty).

In other words, physical environment randomness (physical aleatoric uncertainty) is translated into pixel-level epistemic uncertainty in images, which impacts the architect's uncertainty about the image's fidelity to capture the real world accurately and sufficiently. At the same time, the physical aleatoric uncertainty of the environment (how random it becomes) directly impacts the architect's predictive capability to know what to include in the training and validation of the ML component. This reflected the coverage and nature of information captured in training and validation datasets (source of the machine's epistemic uncertainty). Hence, to assure coverage in the dataset, we must ensure that the architect's systemic perception of the open operational environment is comprehensive and accurate.

Here, where architect's epistemic uncertainty becomes the ultimate receiving end of physical environment randomness. In section 4.8, we define a categorical system that captures Operational Design Domain (ODD) sources of aleatoric uncertainty (randomness) based on environmental conditions that we believe involve more challenging situations that can negatively impact the architect's confidence in assumptions made about the operational domain, which ultimately challenges the reliability of the training process of ML components. Hence, we proposed that the safety of autonomous systems in open, complicated environments is the architect's predictive capability problem. Based on that, we develop Predictive Thinking techniques (STCoT).

From an ML training syllabus development perspective, specifying or designing pixel-level uncertainty (writing traceable requirements) is impractical. Hence, we must pool them and abstract them enough in the form of epistemic pictorial training classes. With training classes being the architect's design problem, aleatoric uncertainty becomes a question of balancing the right granularity of epistemic uncertainty by the architect. Here, architects' knowledge (chosen assumptions) of the real-world complexity field and their abstraction skills are crucial in managing the risk of ignorance. Hence, we proposed the CuneiForm as a traceable pictorial abstraction method to design those pictorial training classes.

E.1.3 Epistemic Uncertainty's Impact on ML Safety: Why Should We Care?

Epistemic uncertainty, the uncertainty arising from incomplete knowledge about the adequacy of the data-generating process, poses a critical yet often underappreciated threat to the safety of ML components. In the decision-theoretic framing of safety, Möller et al. emphasise not only the reduction of expected risk but also the minimisation of unexpected harms, i.e., those arising from "unknown unknowns" in the system's behaviour[1]. While classical risk minimisation tackles only the probability of known harms, epistemic uncertainty encompasses the potential

for entirely Novel operational scenarios by the architect and the training set, which, when severe enough, constitute genuine safety hazards.

One primary source of epistemic uncertainty is the mismatch between training data and operational conditions. When samples {(xi, yi)} are drawn from a distribution that differs, either subtly or drastically, from the true deployment environment, the learned model may exploit spurious correlations, leading to harmful mispredictions under domain shift [2] [3]. In safety-critical settings such as autonomous vehicles or medical devices, even rare shifts (e.g., unusual lighting conditions or atypical patient demographics) can precipitate catastrophic outcomes. Moreover, because epistemic uncertainty is reducible (in principle) through additional data or model constraints, failure to acknowledge it means foregoing crucial opportunities to bolster safety via targeted data collection or robust learning formulations.

A second manifestation of epistemic uncertainty arises from sparse coverage of the feature space. In regions of low sample density, the inductive bias encoded in the hypothesis of some class H dominates the learned decision rule, rendering model behaviour effectively unpredictable to human overseers[4]. Unlike aleatoric uncertainty, which remains irreducible noise, even an arbitrarily powerful learner cannot “learn” in areas where no data exists. This is especially problematic in cyber-physical systems, where the input–output space is high-dimensional and safety-relevant failure modes may reside in remote corners of that space. Without explicit demonstration that epistemic uncertainty is rigorously addressed during design time, ML components may make overconfident assertions that extend beyond their domain of competence, violating the “fail safe” principle and exposing users to undue risk.

Furthermore, the distinction between actual risk and operational risk underscores the amplifying effect of epistemic uncertainty on individual cases. Statistical learning theory assures convergence of empirical risk, Emp(h), to the actual risk R(h) only in the limit of infinite data[5]. In real systems, however, a finite and sometimes small number of critical decisions may be made under conditions where the realised (operational) risk vastly exceeds the ML model’s expected performance. Each outlier misprediction, driven by epistemic gaps in training, can directly translate into a safety incident, ranging from a misrouted surgical robot tool to a failure in a collision avoidance system.

Considering these considerations, any ML application aspiring to safety must explicitly model and mitigate epistemic uncertainty. Strategies such as inherently safe design (e.g., interpretable models that expose regions of high uncertainty), safety reserves via robust optimisation over uncertain parameters, and safe-fail mechanisms (e.g., uncertainty-aware reject options) all hinge on reliable estimates of epistemic risk. Procedural safeguards, such as virtual-environment hazard simulation, likewise depend on identifying where a model’s knowledge is

weakest. ***Our approach*** attempts to qualitatively reduce epistemic uncertainty by enhancing the architect's predictive thinking. The virtual environment discussed in the literature will require an architect with deep, objective predictive capabilities to make it as comprehensive as possible (high fidelity to the problem domain).

Thus, to fulfil the dual objectives of reducing both expected and unexpected harms, this work posits that accounting for epistemic uncertainty is not optional but fundamental to ML safety.

E.1.4 When Mediocristan systems safety tools lack in the face of Extremistan risks

We cannot evaluate existing safety and systems engineering approaches without first highlighting fundamental concepts for understanding the risks associated with event probability distributions in complex systems. A fundamental concept underpinning our evaluation of existing methodologies (such as STAMP, FRAM, and HAZOP) is how these approaches prompt the user or architect to consider problem or system complexity. To build such intuition, we needed to understand the range of views an architect may hold regarding any safety problem, where traditional approaches fit within this mindset, and how our solution fits within this framework. Since we view the problem of understanding risks in a complexity based on being an uncertainty problem, we needed a well-founded theory that we can use as our basis. This is Taleb's theory for understanding risk for complex problems (see section 2.1.1.4).

Based on Taleb's distinction between Mediocristan and Extremistan[6], we can recontextualise these domains as metaphors for how safety engineering approaches conceptualise complexity, risk, and the emergence of hazards in engineered systems. In Mediocristan, events are governed by thin-tailed distributions, and risks emerge incrementally through the accumulation of small, predictable failures. This domain aligns with traditional risk assessment paradigms, such as HAZOP (Hazard and Operability Study), which assumes that hazards can be systematically identified by systematically examining deviations from expected operational conditions. Such methods imply a Mediocristan worldview: risks are tractable, distributions are regular, and failures result from known causes that can be identified through thorough decomposition and analysis.

By contrast, Extremistan describes systems governed by long-tailed distributions, where rare, high-impact failures dominate system behaviour and elude prediction. These methods acknowledge the Extremistan nature of modern socio-technical systems, where wild randomness, high interdependence, and scalable impacts (e.g., a software bug leading to an airline disaster) undermine the assumptions of predictability and control.

One important implication of engineering Mediocristan systems in Mediocristan environments is that it is reasonable to rely on statistical tools, such as the mean, standard deviation, linear regression, and principal component analysis, as robust methods to estimate risks. However, in engineering Extremistan systems in Extremistan environments, those tools are not robust to handle variations in the complexity field to trust risk estimates. This is where we need to introduce another dimension to reduce risks. That dimension is the enhancement of practitioners' or problem solvers' predictive capability to predict harder-to-predict risk factors.

In general, safety approaches prompt practitioners to adopt either a Mediocristan or Extremistan view of engineered complexity. Traditional, control-based hazard analysis methods assume a Mediocristan landscape, where risk is a sum of small parts, failures are mild, and system behaviour is decomposable. Conversely, holistic systemic approaches should reflect an Extremistan mindset, recognising that emergent behaviour, complex couplings, and disproportionate event impact are not only possible but expected in modern systems.

E.1.5 The impact of a control-biased Mediocristan approach to safety

How do systems engineering approaches tacitly encourage the architect to view a problem as a normal-distribution probability problem? And why is such an approach alone not enough in solving CUPEs?

Systems engineering approaches do so by defining a specific (limited) set of types of failures, events, or behaviours to consider and prompting the architect to think of what can make logical sense. For example, HAZOP prompts the architect to consider a set of deviations, such as guidewords, and expects the architect to make an engineering judgement (constrained by expertise and legacy experience) to choose what makes logical sense and ignore what makes no sense. By ignoring a subset of deviations, the architect predicts that the ignored subset of deviations poses 0 risk to the system. This activity prompts the architect to consider a subset of events as the primary concern and assume others as negligible. Which is like tacitly thinking.

Let's assume that all safety problems can be reduced to two types of events: those we consider logically important and those we consider negligible, such that if their impact or probability of occurrence is assumed to be 0 or minimal, no significant effect is expected on the overall safety of the system.

That is the characteristic of viewing the complexity of the problem as a normally distributed set of events. However, CUPEs, where we are dealing with different types of non-deterministic systems operating in an open non-deterministic environment, is not the right place to make

such simplification. In fact, we believe such simplification is the initial sin in engineering judgment that an architect or a project makes.

HAZOP, and STAMP utilise this technique by defining some key considerations and assuming that the system is a single type of cybernetic problem: a control-only problem. We experienced this phenomenon when we applied HAZOP in Section 2.3, we realised that our predictive thinking prioritised only the guidelines that we found easier to make sense of (See Section 2.3, and Appendix G).

Why do we think so?

The fundamental aspect of any normal distribution is that there is a finite set of the most repeated or frequent pins or classes of information, among some number of pins (pins, in this case, would mean types, categories, or things). Hence, if the engineering approach encourages the focus on specific types of failures and allows for labelling certain types as “irrelevant” or “not-applicable,” it indirectly encourages the architect to assume a finite number of types of events. However, in a non-linear distribution of events and categories type of complexity field, such bias entails neglecting hidden, unorthodox kinds of events or “real-world states” misjudged as “irrelevant”.

Thus, pre-set-based approaches, such as HAZOP and FMEA, may not be able to capture essential types of events solely using guidewords. Additionally, the system of interest’s behaviour is also unpredictable. It is influenced by unknown variables, emergent interactions, and unseen factors, often involving multiple autonomous agents trained on confusing datasets. As a result, traditional methods that rely on predefined categories, likelihoods, or deterministic assumptions are insufficient or inadequate in certain areas, necessitating new models of predictive cognition, adaptive reasoning, and exploratory systems thinking even to begin engaging with the problem space. Hence, AIC Systems Approach is designed to propose a process to solve such problems.

E.1.6 Impact of lateral thinking process on safety in CUPE

Part of the safety assurance process is the identification of hazards and the definition of mitigation requirements (Safety Concept). In addition to predicting hazards, the architect or safety engineer must conduct a qualitative or quantitative risk assessment. This includes assessing the likelihood and the impact of a given hazard. Usually, with established safety analyses and well-known hazards, risk assessment of failures benefits from considerable legacy and a body of knowledge. However, when dealing with CUPE where the problem domain is dominated by novel hazards, including soft hazards (see

section E.3 for more details on the types of hazards in CUPE), due to the novelty of such hazards, often there would be minimal or no legacy body of knowledge to back up the safety engineer's risk assessment.

For example, a moon that appears like a traffic light to an autonomous car or a drone. Suppose the architect predicted that such a hazard (associating the moon with traffic lights is a bizarre association to most people). How could they justify the association, let alone justify any form of risk assessment in the absence of logical rigour or a legacy body of knowledge? It is an analogical association that is hard to logically connect (hence systematic thinking engineers may miss such associations). Here, the lateral thinking process can provide a systematic justification that explains why such a rare example is impactful.

Bear in mind that the risk assessment of CUPE differs in expectation from that of a regular system. The likelihood of an event, in a complexity where all events are assumed to be equally likely and their impacts also the same, does not make sense to quantify likelihood and impact. Risk assessment of hazards and failures for CUPE can be approached holistically based on the concept of demonstrated reduction of epistemic uncertainty. This means that risks are reduced to ALARP based on the deliberate pursuit of predictable Black Swans, and the architect utilised both lateral and vertical predictive thinking techniques to arrive at as comprehensive a set of hard and soft hazards as possible. This notion will become important when assessing risks associated with training coverage.

E.1.7 Compounded Uncertainty of Open Complicated Problems

"The complexity of an object is in the eyes of the observer" Klir[7]

- The operating environment may be open and complicated, with a large space of possible real-world state types, some of which may be low-frequency yet high-consequence.
- An engineered autonomous system (AS) may be expected to behave acceptably across a subset of these state types, including atypical conditions that were not fully observed during development.
- The architect's belief system (i.e., the modelling assumptions, abstractions, and evidence base used to define and justify coverage) shapes which state types are represented and which may remain underspecified.

These three considerations motivate the notion of compounded uncertainty in open complicated problems. In such settings, uncertainty is not solely a property of the operational environment, nor solely a property of the engineered system. Rather, uncertainty may accumulate across (i) real-world variability, (ii) the adequacy of the architect's representations

and assumptions, and (iii) the resulting behavioural envelope of learning-enabled components. When the operational domain is open, it is rarely feasible to enumerate all relevant states, and the discovery of new state types may require revision of the categorisation scheme itself.

In this appendix, the phrase **architect's belief system** is used to denote the working mental model and associated artefacts (assumptions, ontologies, scenario partitions, dataset abstractions, and assurance arguments) by which the architect anticipates how the AS will perceive and act. This belief system may be more or less *clear* depending on the extent to which its assumptions are supported by evidence, and more or less *confused* when novel observations suggest that important distinctions have been omitted or mischaracterised. Such clarity or confusion should not be interpreted as a binary state; rather, it may be better understood as a continuum that varies with experience, available evidence, and the evolving operational context.

The term **compounded uncertainty** is therefore used to emphasise that, as the number of interacting factors and interacting systems increases, the architect's epistemic uncertainty about the adequacy of assumptions and coverage may increase nonlinearly. This does not imply that uncertainty must always increase with scale; rather, it suggests that in open domains the *rate* at which uncertainty can be reduced may be sensitive to how the domain is partitioned and which sources of variation are treated as relevant. Consequently, a central design concern becomes the management of epistemic uncertainty: not merely collecting more evidence, but also improving the representational scheme (ontology and abstractions) used to decide what evidence is needed.

To motivate the subsequent argument, the discussion distinguishes between two related but different questions: (i) how frequently different event types occur (a probability-oriented view), and (ii) how well the generating mechanisms for these events can be anticipated using an explicit model (a predictability-oriented view). Both perspectives can be relevant for assurance, but they may support different styles of reasoning and different kinds of evidence.

The remainder of this subsection therefore introduces the intuition behind why open complicated problems may be difficult to treat as if they were governed by stable, thin-tailed distributions over a fixed set of event categories. It is not claimed that all open domains are necessarily heavy-tailed, nor that a single distributional form always applies. Rather, the position taken is that, where rare and high-impact conditions are plausible, approaches that implicitly disregard low-frequency categories as negligible may be insufficient on their own. This motivates the use of a complementary predictive-thinking framing (including both lateral and vertical reasoning) to iteratively refine scenario partitions and to make residual gaps explicit.

- The operating environment is open and complicated with a multitude of real-world state types (dominated by hidden Black Swan real-world states).
- We expect the engineered AS to behave or execute its designed belief states as expected in those black swans.
- The architect's belief system that predicts and defines those states.

The following subsections introduce working distinctions used throughout Appendix E—particularly the distinctions between probability-oriented and predictability-oriented views of complexity, and between complexity and complicatedness. These distinctions are introduced as interpretive tools to support the AIC-based argument about how uncertainty may accumulate and how it may be reduced through structured analysis. They are not intended to replace existing definitions in the systems literature, but to provide a consistent vocabulary for the subsequent thought experiments and engineering interpretations.

Section 2.1.1(e) describes the current state of the literature on the topic of complexity. We have noticed a general theme across the board regarding definitions:

- Complexity is a feature of an observed phenomenon.
- It is directly related to randomness in phenomena.
- Complexity increases as randomness increases.

Such characterisation is closely associated with aleatoric uncertainty (see section 2.1.2). where aleatoric uncertainty is characterised with the following properties:

- Related to the observed system.
- Related to randomness in the system.
- As randomness increases, so does aleatoric uncertainty.

Thus, we can see a link between the standard definition of complexity and aleatoric uncertainty. They may be the same or at least dependent on each other. However, if this is the case, what happens when randomness decreases? Does complexity decrease? The literature assumes it does. In other words, if we add more elements to the observed complexity, we also increase its randomness; hence, it is considered that aleatoric uncertainty is irreducible, meaning that the more we add, the worse the problem becomes (adding more fuel to the fire of aleatoric uncertainty).

Where there is randomness, there is probability, and where there is probability, there is "predictability". Hence, some authors have characterised complexity in terms of predictability, such that the more unpredictable the system is, the more complex it is. A concept with which we can intuitively agree. From that, we conclude that the literature suggests:

- Adding more parts to a system increases the risk of high randomness.

- An increase in randomness means an increase in aleatoric uncertainty.
- An increase in aleatoric uncertainty indicates degraded predictability, which in turn implies an increase in complexity.
- The opposite is also true: removing parts may decrease randomness, which may decrease complexity.

However, if we have an entirely predictable system, in a closed predictable environment, and systematically add more elements to it (for example, adding more propellers to a drone, where the drone is tethered to some power source, or more tyres to a car driving in circles around an indoor ring), do these additions relatively increase the unpredictability of the car or drone system? We know precisely how a propeller is connected, and we can comfortably predict what can go wrong or right. In this case, increasing the number of predictable elements in a predictable system within a predictable environment may not lead to an increase in unpredictability. Thus, a predictability-based complexity definition may not hold in such a scenario. This means the following:

- Predictability does not seem to align consistently with changes in general complexity.
- A larger, predictable system is not more complex than a smaller, predictable one. In fact, it is the dream of any architect to develop a system that remains explainable and predictable, regardless of its size.

There may be a missing distinctive factor in this observer-independent approach to complexity, where the probability of events and predictability have a clearer impact!

Technically, adding more predictable elements to a predictable system does not necessarily (assuming a perfect world) change its predictability. That is the whole point of systematic increments, in systems engineering, which controls the predictability of systems as more elements are added.

This raises the following questions:

- What is the holistic quality of an organisational state-of-disorder that directly depends on its scale and frequency of events, regardless of whether these events are predictable or not? This is a probability problem.
- What is the holistic quality of an organisational state-of-disorder that directly depends on its predictability, regardless of its scale or frequency of event changes? This is a predictability problem.

The literature on the probability and predictability of systems or organisations is vast. The reader is welcome to explore the field. Below is our re-interpretation in the context of predictive architecting and problem solving of the literature about those two concepts:

E.1.8 Probability View of Complexity

A probability-oriented view treats a problem domain as a set of outcome types (or event categories) together with an associated distribution over those categories. In this view, the analytical task is primarily to (i) decide which categories should be distinguished, and (ii) estimate how frequently each category is expected to occur (either empirically, or as a degree of belief).

Two common interpretations of probability are often contrasted:

- **Frequentist:** probability is interpreted as long-run relative frequency under repeatable conditions.
- **Bayesian:** probability is interpreted as an agent's degree of belief, conditioned on prior knowledge and updated by evidence.

For engineering practice, both interpretations can be useful. In either case, however, the modelling decision about which categories to include is central. Many established hazard analysis methods can be interpreted as providing structured prompts for category selection (e.g., guidewords, failure modes, deviation classes), and then encouraging practitioners to prioritise categories deemed more likely or more consequential. When the operational context is relatively stable and the system boundary is well defined, this approach can support effective risk triage.

For open operational environments and learning-enabled components, the category-selection step may be more fragile, because the space of relevant operational variations may be only partially known at design time. In such settings, it may be difficult to justify treating low-frequency categories as negligible solely because they are rare in available evidence. This does not imply that probability-based reasoning is inapplicable; rather, it suggests that probability-based registers may require complementary methods for discovering overlooked categories and for challenging the completeness of the partition.

Accordingly, within the AIC framing, probability is treated as one lens for summarising observed variability. It is complemented by a predictability-oriented lens (Section E.1.10), which emphasises how an architect's explanatory model (and its assumptions) supports or constrains the ability to anticipate novel conditions.

- **Frequentist:** Probability is the long-run relative frequency of an event in repeated trials. An objective measure, independent of the observer's knowledge.
- **Bayesian:** Probability is a degree of belief about the likelihood of an event, given an observer's prior knowledge (subjective).

A probabilistic view of problems or systems involves considering all outcomes and their corresponding probabilities of occurrence. to construct a probability distribution, you need to do two activities:

- Select the relevant pins (categories) for the things or types of scenarios that are currently occurring, may occur in the future, or have happened in the past. For example, the HAZOP process uses a set of “guide words” to categorise types of failures.
- Estimate the frequency of their occurrence. For example, in the HAZOP process, the practitioner is given a choice of what they consider the most suitable category if failure were to occur for a given interaction or event in a system.
- The most frequent categories are the ones that are more important and worth attention. Least frequent categories are negligible since they are the least likely types of events to occur in a system.

For example, in the HAZOP process, the practitioner's bias toward the most obvious category for a given interaction or event in a system often results in a frequency distribution that disproportionately favours a small subset of guide words. The HAZOP process does not require consideration of all guide words for every event or interaction in a system. See Section 2.3 for more details on how we applied HAZOP.

If you follow the frequentist approach, you should consider all relevant factors when estimating the most suitable option and record the final frequencies of the options you choose. A frequentist approach, such as HAZOP and FMEA, falls into this category, as they are “guide words” category-based thought processes. If you follow a Bayesian approach, you need to use an iterative, hierarchical method based on continuous refinement of a set of beliefs. SHARCS and STAMP/STPA processes follow such an approach.

In either case (frequentism or Bayesian), probability is not necessarily about our capacity to accurately predict outcomes; it is a measure of how likely an event, failure, or state is to occur in a system, whether objectively or subjectively. Hence, most probabilistic complexity-inspired approaches (HAZOP, FMEA, etc) come with a probability-based risk register.

However, non-deterministic engineered systems, such as ML-based systems, operating in open and complicated environments exhibit emergent behaviour that is heavily dependent on interactions with the environment, as they adapt dynamically rather than deterministically. Furthermore, they accept infinitely many inputs (as an ML-based computer vision system is expected to handle any variation in a target object of interest, such as a drone). In such a type of complexity, Black Swans and rare events cannot be ignored as negligible.

These characteristic differences require consideration of an infinite number of event categories and failure modes to ensure completeness and exclude negligible events. Such a task suggests

that probabilistic approaches to the problem may not effectively address its vastness. Because predetermined guide words may bias the practitioner toward a limited set of predictive thoughts, this can lead to the omission of valuable failure modes or requirements.

In other words, it is more effective to regard ML-based systems as a “predictability” problem that requires a predictive-thinking model to discover categories of interactions, as they are applied iteratively and dynamically.

E.1.9 Probability distribution of events-based classification of systems

From an assurance perspective, it can be useful to characterise engineered complexes by the apparent distribution of behaviourally relevant events (or, more cautiously, by the distribution an architect *models* or *expects* given available evidence). The intent of this subsection is not to propose a definitive taxonomy of systems, but to provide a working classification that supports later discussion of why certain analysis approaches may be more or less effective in open environments.

The following categories are therefore framed as *modelling stances* rather than intrinsic properties of the artefact. In practice, a system may move between these stances as evidence accumulates, operational constraints change, or the system is deployed in new contexts.

- **Normal-system (thin-tailed stance):** a complex whose behaviour is modelled as being dominated by a relatively small set of recurring event types, with other event types treated as increasingly negligible. In such cases, it may be feasible to argue that a finite catalogue of scenarios provides substantial coverage, particularly when the operational environment is controlled and the system mechanisms are well understood.
    - This stance is often associated with well-bounded engineered artefacts where repeatable operating conditions can be specified and verified.
    - It does not imply that rare events cannot occur; rather, it implies that they are treated as sufficiently bounded (or sufficiently low-consequence) for the purposes of the assurance argument.
- **Abnormal-system (heavy-tailed stance):** a complex whose relevant event space is modelled as heavy-tailed or otherwise non-normal, such that low-frequency event types may remain non-negligible because they can plausibly dominate outcomes in some circumstances. Under this stance, the completeness of any finite catalogue is harder to justify, and the architect may

rely more heavily on iterative refinement, monitoring, and deliberate exploration of the “tail”.

- This stance is often invoked for open environments and for learning-enabled components that are sensitive to distribution shift and atypical contexts.
- A key implication is that categorisation schemes may need to evolve as new kinds of situations are observed or hypothesised.

- **Ideal-system (limiting case):** a conceptual limiting case in which the relevant behaviour is sufficiently governed by stable rules that the architect can make highly confident predictions across the intended operational envelope. This notion is used as an analytical contrast rather than as an empirical claim that real-world autonomous systems achieve perfect predictability.
- **Stochastic confusion (limiting case):** a conceptual limiting case in which the architect cannot form a stable, defensible set of rules or categories that reliably predict behaviour or outcomes. In such a case, the difficulty is not only that many event types exist, but that the partition and the governing regularities remain unclear.

This working classification is aligned with the later use of Table E.1 and Table E.2, which organise problem types in terms of (i) the distributional character of observed events and (ii) the degree to which the situation is clear, fuzzy, or confusing to the architect. Importantly, these categories are used in this appendix to motivate why a single style of analysis (e.g., one that implicitly assumes thin tails or a fixed set of deviations) may be insufficient for certain open, safety-critical domains.

- **Normal-system:** A normal-system is a complex whose uncertainty distribution of its emergent behaviours is assumed to follow a normally distributed profile or tendency. In such systems, there exists exactly one finite subset of events that is most likely to occur. Assurance in such systems would be based on proving that indeed those subsets of events are the and we can precisely verify this using mathematical proves. In normal-systems it is reasonable to assume complexity to be Mediocristan.
  - In normal distribution, there is a clear boundary between a range of bins [-b to b] where the probability of any event outside the range is precisely 0.

        - Normal systems are easier to trust the assurances made by the architect of the number of emergent behaviours in deterministic means. Hence, they are easier to have confidence in.

This means that the Predictive Problem Solver can be confident that only a specific or desired set of events will occur, whose impact is worth their attention and mitigation. Any other rare but negligible events. Examples of normal-systems such as a door, a regular car, or a computer. The architect assumes the normal system to be ideal. Domino-like deterministic systems can be viewed a special case where the bell curve is very narrow, since there can only be a specific subset of events that can occur in those systems. So, the architect's uncertainty can be reduced to near 0 given enough time to learn how they work. However, this cannot be said for ML-based CPS; there is always residual uncertainty and surprise.

- **Abnormal-system:** A clear complicated complex whose uncertainty profile or probability distribution of its emergent behaviours does not follow a normal distribution (bell-shaped curve). In other words, it may follow a heavy-tailed distribution or be uniform. For long-tailed systems, the architect can only possess a priori knowledge of the most likely events (the head events); however, this prior knowledge is not fixed and will be updated and refined over time to determine whether that subset is exhaustive or whether other events should be included.

    - In contrast to a uniform and normal distribution over a range of bins $[-b,b]$, the $P(x)$ never converges to 0. This indicates that there are no clear-cut boundaries for systems whose emergent behaviours follow a long-tailed distribution.

    - On their own, with an inherent boundless category of emergent behaviours (bins in histogram), they are challenging to trust assurance claims of their designed behaviours and that they will only behave in a way that was intended for and never more.

The approach to make them more transparent and trustworthy, the abnormal complicated complexes are:

- Defining loosely bounded abstract concepts (like purpose) which are open-ended to accept more variations of unaccounted designed behaviours. For example, instead of a concrete specification of what a human looks like, use an abstract image of a typical human for a dataset for a vision system. This way, more variations of what a human is can be validated to be acceptable.

- Clearly demonstrate that abnormal predictive thinking is employed as much as possible to predict long-tailed emergent behaviours. Abnormal thinking is another word to "lateral thinking" which allows to accept loosely concrete strong traceability of specification. For example, a picture of a human in a painting may also be an acceptable interpretation of a requirement that asks for including a human in the training dataset. Another example may be considering reflections of light on raindrops over tree leaves as a potential source of failure for an ML component.

For example, an ML-based perception system trained to detect drones in specific open green fields. We can say that the events in the training dataset are those we are sure will be detected if the perception system encounters them in the real world. However, we also expect that other unseen images are not part of the training dataset, and therefore not included in our prior subset of events. For instance, images from different open green fields or even cities. As you can see, an abnormal system also has a set of unseen events that contribute to its capabilities. In abnormal systems, it is not reasonable to assume the complexity to be Mediocristan, and it is safer to assume it to be Extremistan.

Such systems are characterised by sensitivity to butterfly effects and Black Swan scenarios. In other words, perceived rare and insignificant events cannot be overlooked (these are the Long-tailed events). Open, complicated environments; neural-network, ML-based autonomous systems; and their combination are examples of abnormal complexes. Abnormal complexes can be described as "Abnormal-systems" when an architect models their behaviour and describes how they work and what to expect to observe about them.

  - **An ideal system:** A clear complicated complex, a special case of normal systems, its emergent behaviours entirely predictable, complex in all world views for any Predictive Problem Solver. You can imagine the bell curve of an ideal system to be completely flat at the far ends of the bell, and a very narrow and long bell.

Ideal systems operate in "complete knowledge" of an operational domain, such that epistemic uncertainty (residual epistemic Ignorance) equals zero. The randomness of the observation is entirely predictable. For example, domino-like deterministic systems (like a comprehensively modelled and explained analogue controller or a regular computer) are a good approximation of ideal systems in theory because the system is predictable and explainable. A fully automated factory in a highly controlled environment (factory condition) is an ideal system since it is predictable.

- **Stochastic Confusion:** A confusing, complicated, complex, an extreme form of abnormal complex**,** A completely unpredictable VUCA complex or one that is completely confusing in all world views for any Predictive Problem Solver. This corresponds to “complete ignorance” of an operational domain, such that architect’s epistemic uncertainty = 100%. The randomness of its emergent behaviours is entirely unpredictable, and range of types of events are boundless. It is a nightmare for any general systems architect.

With the above classification of types of complexes, we can categorise four approaches that an architect can take regarding CUPE, which also determines the types of safety paradigms (see sections 2.1.3 and the introduction to Chapter 5). Figure 4.1 differentiates the types of available safety approaches for an architect to deal with CUPEs in engineering based on the general attitude of the approach towards Black Swans (dominate feature of CUPEs). Formal methods (like Event-B) and HAZOP can be classified as the initial phase of safety, where the system's complexity and its environment is thought of as a set of controllable interactions in a domino-like fashion. This would be an ideal view of the problem and is much suited for highly controllable environments and deterministic systems. This paradigm considers Black Swans (rare and impactful events) negligible.

The same attitude towards Black Swan was inherited by the follow-up generation of safety approaches (Safety II), STAMP, FRAM, and SHARCS. Our experience in applying STAMP and FRAM in Section 2.3 confirmed this belief (see Section 2.3 and section 9.2). Safety II looks at the safety problem as a hierarchical functional control structure. With such a view of complexity, there is an expectation of a rigid boundary where some information is deemed irrelevant or not applicable. For example, whether the operator of a control switch is wearing a white or black shirt is irrelevant information about why a switch fails. Tacitly, calling such a situation irrelevant is an indirect dismissal of causal impact due to the rarity of a shirt colour having anything to do with a failed switch. Such a mindset cannot be effective when dealing with CUPE, because architect's assumptions at any given time and scale of complexity are never solid and rigid—only temporary correctness.

Hence, we need an approach that believes in the following principle: “There is no such thing as an irrelevant, and there is no such thing as a rigid boundary. Here, where control only view of stochastic conclusion becomes a liability for a problem solver because they will miss those rare and impactful events as irrelevant or not applicable.

Here, where AIC (Appreciation, Influence and Control) comes in to add three graded of rigidity in boundary definition:

- **Control:** direct coupling between two components. Hard-bounded with rigid (unchanging) exchange of effect. For example, the operator physically flips the switch on or off.
- **Influence:** indirect effect between two components. Softer-bounded functional exchange, rigid (unchanging) exchange of effect. For example, the operator's manager demanded that the person wear formal attire that day, but the operator did not obey this directive, which forced the manager to fire the operator that day.
- **Appreciation:** the absence of control or influence from one component on the other. The exchange can change dependents on the appreciated system behaviour, whereby the appreciating has no power to control the appreciated behaviour. For example, the country's current economic situation impacts the operator's ability to find another job.

If we applied a control-driven mindset, there is no reciprocal control or influence between the operator and the economic situation of the country, where a simplistic feedback control loop model (as per STAMP principle) would not necessarily be helpful if we are dealing with complicated problem in which there is no rigid assumptions can be trusted to remain true for a long time. Such an attitude may lead to unwanted surprises for open, complicated environments and non-deterministic engineered agents.

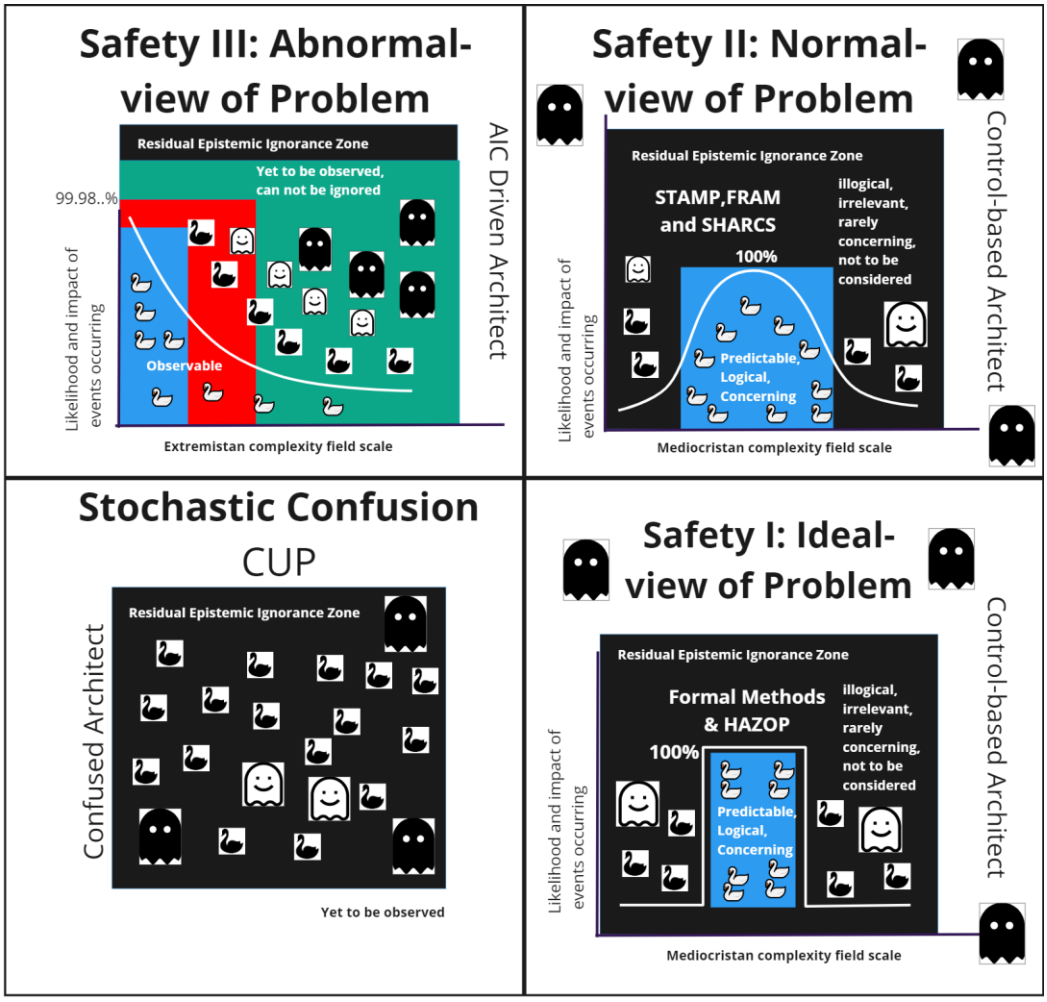

Figure E.1 Types of architect approaches to complexity of in CUPE. Compounded Epistemic Uncertainty Problem (CUPE)

The swans and ghosts' icons represent pictorial scenarios in different visibility levels to the architect's perception of the engineered complexity problem[1].

For full theoretical detail behind Predictability, Probability distributions, Complexity, Complicatedness, Epistemic Uncertainty, Aleatoric uncertainty, System, Stochastic Confusion (non-system), see section E.1 (Appendix E). Given our evaluation of the topic in section E.1, we define the following key concept:

> A **Compounded Epistemic Uncertainty Problem** CUPE is a type of problem that involves agents, deterministic systems, and an open, complicated environment, where any changes in any part of the problem lead to an unpredictable whole for the

[1] See sections 4.9 and 9.13 for a more detailed description of what the icons mean

predictive observer. The epistemic uncertainty of the predictive observer is understood to compound nonlinearly as the complexity of the problem domain changes unpredictably.

There is a set of properties in such problems which can be sources of risks:

1. Any prior beliefs about such problems by the observer can be volatile. There is a very high likelihood that prior beliefs will not prevail over time.
2. Confusing, purposeless, uniformly, or non-normally distributed complexity of events and behaviours.

It is a scenario when unpredictable changes in an observed complexity (aleatoric uncertainty/randomness) cause a non-linear (compoundedness) increase in architect epistemic uncertainty (lack of knowledge) about the complexity field, making the problem fundamentally harder to predict (confusing). In general systems wisdom, concepts like VUCA (volatility, uncertainty, complexity & ambiguity)[8] and Complex Adaptive Systems[9] are other terms that describe problems that can become a Compounded Epistemic Uncertainty Problem (CUPE) for a predictive architect. To resolve such problems:

The best an architect can do to address such problems is to deliberately combine lateral and vertical thinking. Approaching such problems from a control-theory biased perspective may lead to low resilience and inefficient solutions.

These problems arise in open, unstructured, and dynamic environments in which the architect cannot confidently define or rank the likelihood of events or interactions, because the uncertainty profile is unpredictable due to increasing complexity. Reducing such complexity based on the premise that “we shall assume the probability distribution of events to be normally distributed” may be an incorrect characterisation of the problem. With a normally distributed probability distribution of events, narrowing the issue into a pre-set of or exactly a finite number of general scenarios or situations is possible. However, the by-product of such a belief system is that any rare or improbable events can be assumed to be negligible or not concerning.

In other words, this uncertainty profile means that relying on assuming a normal probability distribution, which would constrain the analysis to a finite set of scenarios and dismiss rare (Black Swan) events as negligible, is an inadequate characterisation for open, unstructured, and dynamic environments.

E.1.10 Predictability: the architect's ability to forecast scenarios in a complexity

In this appendix, **predictability** refers to the extent to which a predictive observer (e.g., an architect, or an explicit model used by an architect) can form defensible expectations about what outcomes may occur, under what conditions, and with what plausible consequences. Predictability is therefore treated as observer-relative: it depends not only on the world's variability, but also on the adequacy of the observer's representation, the quality of available evidence, and the inference method used to generalise from that evidence.

Under this interpretation, a domain can be highly variable yet still partially predictable if stable regularities can be identified and validated. Conversely, a domain can appear unpredictable if relevant regularities exist but are not represented in the observer's model (a form of epistemic limitation). This view is compatible with the glossary distinction between *aleatoric uncertainty* (irreducible variability in the world) and *epistemic uncertainty* (reducible uncertainty due to incomplete knowledge).

Many engineering approaches can be viewed as attempts to improve predictability by making the governing rules explicit—whether these rules are deterministic constraints, probabilistic assumptions, or structured causal narratives. In Bayesian terms, the architect's model can be interpreted as encoding prior beliefs about the mechanisms that generate observed outcomes; these priors may then be updated as additional observations are obtained. Where the operational context is stable and the mechanistic structure is well understood, iterative refinement can increase confidence that the model will remain adequate over time.

In open environments with adaptive or learning-enabled systems, however, two limitations may become salient. First, the event categories that matter for safety may not be known a priori, meaning the partition itself may need revision. Second, some "rules" may only hold conditionally or temporarily, for example when operating assumptions remain satisfied. When either limitation applies, predictability may be better described as *partial* or *fuzzy* rather than absolute. This motivates methods that explicitly surface assumptions and that treat unexpected observations as potential indicators of missing categories, rather than as mere outliers to be ignored.

The practical implication is that, for safety-critical autonomy in unpredictable environments, it may be insufficient to rely solely on the expectation that a fixed catalogue of hazards and deviations will remain adequate. Instead, predictability-oriented reasoning may place greater emphasis on (i) identifying candidate regularities that appear to govern outcomes, (ii) articulating the conditions under which these regularities are expected to hold, and (iii) systematically searching for evidence of failure of these conditions. In AIC terms, this search may include explicit consideration of what must be appreciated (dependencies without

governance), what may be influenced (indirect couplings), and what may be controlled (direct governance), because changes in these relations can alter which regularities remain valid.

Regardless of a system's complexity or the likelihood of events, a few fundamental rules or principles typically govern its behaviour, thereby determining the observed frequency of events. The process that enables the architect to infer or design these rules underpins the architect's mental model for predicting how the system will behave across various scenarios. In predictability problems, the architect's goal is to develop a set of fundamental rules and continually refine or update them over time. The architect assumes that, regardless of the observed or to-be-observed probability distribution profile, there are underlying rules that generate it.

How do we find and be sure of them? This is the role of systems approaches.

In Bayesian terms, the architect's prior knowledge of those rules reflects their understanding of how observed complexities behave and thus predictions are made accordingly. As new observations about the system accumulate, these priors are updated, effectively refining the architect's predictive model. Hence, systematic processes (such as event-B and SHARCS) increase the architect's confidence in their assessment of the system's complexity with each refinement introduced. Such a belief is observer-independent, meaning anyone who knows SHARCS ontology well can verify SHARCS models and arrive at the same conclusion. However, systematic processes (akin to vertical thinking-based processes) are highly sensitive to initial assumptions to remain preserved. For example, an electric current can flow only in one direction through a wire. If, for argument's sake, we have a situation in which the electrical current becomes sentient and begins to choose which direction to flow, systematic models lose their validity.

Therefore, although the frequencies of expected outcomes inform the likelihood of behaviours occurring given the satisfaction of guard conditions, a Bayesian perspective on predictability would interpret those observed frequencies in terms of obeying specific structural rules or mechanisms that shape the system's probabilistic behaviour. Systematic approaches provide the architect with a way to infer such rules. In conclusion, systematic solution approaches such as HAZOP, SHARCS, and STAMP are Bayesian-based, architect-predictive thinking approaches that depend heavily on the architect's confidence in the validity of assumptions over time.

The architect's perception and confidence in those rules cause many design mistakes (especially for autonomous systems). Open problems involving free-will, constrained-will agents, deterministic systems, and an open environment are not the same as an electrical current flowing across a copper wire. To minimise design errors caused by engineering

misjudgement (in such problems), we cannot avoid managing the architect's perspective about the general rules that govern complicated open environments and how the autonomous system will interact with Novel complexities. Traditional systems engineering approaches needs to be adapted to such problems.

- So what? What have we learned from the above articulation?
    - When the predictive architect analyses a complexity, the architect's primary purpose is to determine the simple, general rules that govern the complexity's behaviour or the system, part of a larger whole, and the interactions among its constituent parts. The architect goes beyond merely observing event types or frequencies; the focus is on identifying whether general rules govern occurrences and repetitions.
    - When the predictive architect designs complexity, they use simple rules of interactions to create a predictable complexity. For example, the architect may use AIC rules to design the holistic behaviour (system-level behaviour) of a system of interest or explain why a particular behaviour occurred in some complexity.

Suppose a system or an organisation of things and events is highly deterministic, and the architect can specify the governing rules and the required initial conditions precisely. In that case, we might say it is highly predictable (even if it has nontrivial probabilities for various outcomes when partial information is considered). Such probabilities of events would occur predictably to the architect.

- For example, A deterministic system operating within a closed environment, such as an automatic building door.
- If the architect faces a problem involving one door or a thousand independent doors, from a probabilistic perspective, the frequency of failures will drastically differ between the two. If we use a probabilistic sense of complexity, more frequencies indicate greater complexity, while fewer frequencies and parts imply less complexity.
- From a predictability perspective, having a thousand predictable doors or just one door does not change the predictability state of the problem. Both problems, regardless of scale, are predictable organisations. Meaning there can be only a finite set of categories of failures that will repeat repeatedly. The architect would have predicted and accounted for all these failures in his design.
- Because it is a predictable problem, it makes sense to ignore any modes of failures that are very unlikely or rare.
- In such a context, the architect may assume that the predictability of the problem can eventually be reduced to a normally distributed set of events,

whereby there are a handful of categories of events and failures that remain constant as the number of doors increases or decreases. For such problems, HAZOP, FMEA, STAMP, and SCHARCS can be effective.

- So what? well, those approaches come with the tacit assumption that problems can be reduced to a normally distributed set of events, whose risks can be objectively quantified. This cannot be the case with Compounded Uncertainty Problems.

Suppose a system is subject to chaotic dynamics. In that case, small changes in uncertainties in assumptions or even missing ones can lead to significant differences in predicted outcomes, making the process less predictable. In such problems, any pre-set category of events or failure modes (such as those expressed as guide words) may not be sufficient to predict enough events or hazards.

- For example, an AI-based system operating in an open, complicated environment, such as an ML-based perception system operating in an open train tracks zone.
- The probability distribution of events in these types of problems is characterised as being heavy-tailed distributed[10].
- Rare events are non-trivial and cannot be ignored automatically, as they have a significant impact. Hence, if an approach that tacitly accepts the focus on a set of the events and ignores less likely events is used to solve such problems, the results will be an insufficient characterisation of the problem.
- Heavy-tailed distributed complexity is dominated by low-probability, high-impact events, commonly referred to as Black Swans. Risk registers that classify risks based on likelihood and impact can be misleading methods (on their own) for such problems, as these scoring techniques accommodate the concept of a “low-probability, low-impact event"[10, 11]. Such risks often lack a statistical basis to support their acceptability level argument for Black Swan scenarios.

From above, we recognise the dependency between randomness (probabilistic uncertainty) and inability to trust predictions (predictability of the observed randomness). The literature delineates two types of uncertainty, which we view as fundamental concepts to our main topic “understanding architect predictive capability in facing complexity fields”:

- **Involvement of uncertainty types:**
  - **Aleatoric (random) uncertainty** can render an event unpredictable in principle (e.g., the randomness of events during a storm concerning an ML-perception system).

- **Epistemic (lack of knowledge) uncertainty** can also hamper predictability (e.g., a predictive architect not knowing specific parameters or types of factors involved in a storm).

The logical link between the two uncertainties can therefore be understood as:

> Lack of knowledge (ignorance) about the rules that govern a probability distribution lead to an observation's unpredictable probability distribution (and vice versa), which in turn leads to "stochastic confusion" for the predictive observer.

Now we have laid out the concepts of Probability and Predictability in the context of architect predictive thinking and the complexity of observation. We can then distinctly define the difference between the two fundamental concepts of our approach**:**

E.1.11 Complexity vs Complicatedness

Several systems scholars have argued that the description of "complexity" is, in part, observer-dependent: what is regarded as complex may depend on what distinctions the observer can perceive and what explanatory structures the observer can apply. This appendix adopts that general intuition as a motivation for focusing on the architect's role as a predictive observer. The aim is not to redefine complexity universally, but to introduce a consistent vocabulary for discussing uncertainty in open operational domains.

Within this appendix, the following *working distinction* is used:

> *Complexity refers primarily to the aleatoric variability of an observed scenario (i.e., how the structure and state of the world may vary), whereas complicatedness refers primarily to the architect's epistemic difficulty in representing, explaining, and predicting that variability using an explicit set of rules, abstractions, or categories.*

The phrase "refers primarily" is used intentionally: in practice, complexity and complicatedness can interact. For example, as environmental variability increases (greater aleatoric uncertainty), an architect may require richer models and more extensive evidence to maintain the same level of predictive confidence (reducing epistemic uncertainty). Conversely, even moderate environmental variability can become highly complicated if the architect's categorisation scheme is inadequate or if key interactions remain hidden.

To make this distinction more operational, complicatedness can be discussed in terms of the observer's *clarity* about the domain's governing regularities. This clarity may be high when rule-like structures are stable and well evidenced, low when the observer lacks such structures, and intermediate when the observer can explain some aspects but not others. In this sense, complicatedness may be described as having at least three qualitative modes: **clear**,

confused, and **fuzzy** (partially clear and partially confused). These modes align with the broader interpretation that epistemic uncertainty is reducible in principle, but may be reduced at different rates depending on the adequacy of the modelling approach.

By contrast, complexity is discussed here in terms of the apparent distributional character of events and states (e.g., whether behaviour appears dominated by a relatively small set of frequent categories, or whether rare categories remain non-negligible). The intent is not to assert that any one distributional assumption always holds, but to recognise that different assumptions about event distributions can drive different engineering attitudes toward what is "negligible" and what must be explicitly explored.

The combination of these two dimensions (complexity mode and complicatedness mode) motivates the illustrative classification in **Table E.1**, while **Table E.2** provides example definitions that map these modes to representative problem contexts. These tables are used later to motivate why safety analysis for open environments may require a combination of exploratory (lateral) and refining (vertical) reasoning.

From the above, we further define the following premise:

> *Complicatedness refers to the architect's epistemic uncertainty resulting from a potential increase in aleatoric uncertainty of the complexity of a complex*.

We use the term "potential increase," which encompasses actual increase, perceived increase, or possible increase in randomness. Complicatedness can be in three modes of predictability:

- **Clarity:** where the complexity is somewhat predictable by the architect. Such complexity can be described as a clear "system". By which the architect can determine the nature, time, and place of every event and state in the system.
- **Confusion:** the complexity is somewhat random and unpredictable. Such complexity (for systems engineering) can be described as a Stochastic Confusion or "non-system". By which the architect cannot determine the purpose, nature, time, and place of any event or state in the observation.
- **Fuzziness:** Partially clear yet partially confusing. This indicates that the architect can confidently predict certain aspects of the problem and comprehend the general principles that govern its probability distribution. Conversely, there are areas where the architect experiences confusion, rendering them unable to confidently make accurate predictions.

It is essential to recognise that clarity and confusion depend on the observing architect's knowledge and mental model, which are often fuzzy. As for complexity, there are three holistic modes of complexity based on the probability distribution pattern of events and states:

- **Normally distributed complexity[2]:** the probability distribution of events and states is said to be normally distributed, where there exists a subset to is more likely to occur, and others least likely to occur. Rare events can be negligible, and most likely events can be biased by the problem solver's knowledge and mental model. An architect may design a normal complexity and include constraints to ensure that the complexity remains normal around expected behaviours and that inputs also remain specifiable.
- **Abnormally distributed complexity:** the probability distribution of events and states is said not to follow a usual distribution trend. For example, a heavy-tailed distribution, but power-law distributions or light-tailed distributions can also be included. Such complexity is characterised by Black Swan events that cannot be neglected. They can be biased depending on the predictive architect's knowledge and the accuracy of their mental model that defines the types of things or events to be observed.
- **Uniformly distributed complexity:** The probability distribution of events and states is said to be uniformly distributed, meaning that all identified types of events and states are equally probable to occur at any time or place. They can be problematic (if confusing) and non-problematic (if an ultimate purpose can be identified and verified for such randomness to serve).

Based on the above classification of problem types, we can define 9 distinct modes of complexity and complication in problems.

Table E.1 Complexity and complicatedness-based classification of systems engineering problems.

	Complexity Modes		
Complicatedness	**Normal Complexity**	**Abnormal**	**Uniformed**

[2] Note that we are also including a deterministic system in this normally distributed complexity category. In a highly deterministic system, such as a light switch, the likelihood of a single event occurring at a specific time and place is a special case of a normal distribution curve. Whereby the standard distribution curve would be 0 probability at all other categories of possible events to occur in any given time, and 100% on the triggered event. Like the histogram of a single colour image that has only one pixel type (0,0,0).

Modes	Field	Complexity Field	Complexity Field
Clear	**Clear, purposeful, Normal Complexity (solution System, non-problematic)**	**Clear, purposeful, Abnormal Complexity (solution System, non-problematic)**	**Clear, purposeful, Uniformed Complexity (solution System, non-problematic)**
Fuzzy	**Fuzzy, Purposeful, Normal Complexity (evolving problem, partially problematic)**	**Fuzzy, Purposeful, Abnormal Complexity (evolving problem. partially problematic)**	**Fuzzy, Purposeful, Uniformed Complexity (evolving problem, partially problematic)**
Confusing	**Confusing, Purposeless, Normal Complexity (problem / non-system)**	**Confusing, Purposeless, Abnormal Complexity (problem / non-system)**	**Confusing, Purposeless, Uniformed Complexity (non-system, problematic)**

Table E.1 classifies the complexity of problems according to how confusing their probability distribution is concerning the predictive architect. The following are the definitions of each type, with an example complexity:

Table E.2 Problem types of definitions

Problem Type	Predictability	Probability Distribution of Events	Architect's attitude towards the problem	Example
Clear, Normal Complexity (Solution System)	High (Predictable)	Normal	A deterministic system with fixed behaviour; all events and states are known and repeatable.	Light switch, electronic watch, non-autonomous car.
Clear, Abnormal Complexity (Solution System)	High (Predictable)	Abnormal (Heavy-tailed)	Rare or Black Swan events are explicitly modelled; the system is explainable and controlled despite irregular distributions.	AI training simulator for autonomous vehicles with rare Black Swan bias.
Clear, Uniformed Complexity (Solution	High (Predictable)	Uniform	Events have an equal likelihood; all categories of scenarios are known	Cuneiform-based uniformly distributed AI image dataset

System)			and predictable, with high epistemic confidence.	with known variations.
Fuzzy, Normal Complexity (Evolving Problem)	Partial	Normal	Some frequent events are well-known, while rare events exist but are negligible; prediction accuracy improves with time and experience.	Automated delivery robots on campus, affected by minor anomalies (e.g., a dog chase).
Fuzzy, Abnormal Complexity (Evolving Problem)	Partial	Abnormal (Heavy-tailed)	Frequent categories are identified, but unpredictable high-impact events (Black Swans) exist and cannot be ignored; they can be measured with some prior knowledge.	Autonomous cars or drones with machine learning (ML)-based components are tested under known distributions, but unpredictable events can occur in rare cases.
Fuzzy, Uniformed Complexity (Evolving Problem)	Partial	Uniform	Cannot identify any most likely scenarios; every type of event is equally probable, as the architect lacks reliable priors; the unpredictable part is unknowable in advance. but some parts of the complexity are predictable.	Drone swarm in post-disaster rescue, an unknown urban environment, with an unstructured and volatile operating domain.
Confusing, Normal Complexity (Problem/Non-System)	Low (Unpredictable)	Normal	The architect does not yet understand why a deterministic system behaves abnormally; root causes exist but are unclear, and rare causes are assumed to be negligible.	Warehouse robot fails to respond due to obscure or hidden issues in a closed environment.
Confusing, Abnormal Complexity (Problem/Non-System)	Low (Unpredictable)	Abnormal (Heavy-tailed)	Non-deterministic system; high unpredictability; causes unclear; rare events cannot be ignored; some	An autonomous robot fails its mission in a semi-controlled environment, such as road

			interactions are hidden or unknown.	traffic, despite being functional.
Confusing, Uniformed Complexity (Problem/Non-System)	None (Unpredictable & Unclear)	Uniform (Unknown)	Compounded Uncertainty Problem — complete unpredictability in open, unconstrained environments; the architect cannot prioritise risks; complete epistemic and aleatoric uncertainty.	Multiple autonomous systems trained on confusing datasets (where the architect has no grasp of the scenarios used in those datasets) operating in a busy marketplace or open airspace (e.g., above jungle or urban areas).

Table E.2 provides a structured classification of systems engineering problem types based on their predictability, the probability distribution of events, the architect's cognitive relationship to the problem, and real-world examples. It distinguishes between solution systems, evolving issues, and problem/non-system scenarios, highlighting how increasing uncertainty and environmental openness escalate the system's unpredictability and complexity. As problems transition from clear to fuzzy to confusing, the architect's ability to model, predict, and manage system behaviour diminishes, especially under conditions where Black Swan events or uniform uncertainty dominate. This classification informs the nature of challenges engineers face. It guides the selection of appropriate design, assurance, and reasoning approaches, ranging from deterministic control in clear systems to adaptive and exploratory strategies in complex, uncertain contexts.

E.1.12 Randomness, Predictability, Confusion and Eureka

A recurring intuition in discussions of uncertainty is that increased randomness in the world may be associated with reduced predictability for a given observer. However, the relationship is not necessarily linear, because predictability depends on whether the observer possesses (or can develop) an explanatory structure that remains adequate as conditions change. In other words, the same level of environmental variability may appear more or less predictable depending on the observer's model and evidence base.

In this appendix, the term **stochastic confusion** is used as a modelling label for situations in which the observer does not (yet) possess a stable, defensible set of categories and rules that supports reliable anticipation. Under such a label, new information may not immediately reduce

uncertainty; instead, it may reveal that the current categorisation scheme is incomplete. This is consistent with the notion that epistemic uncertainty can increase temporarily when the model itself must be revised.

Conversely, it is possible for an observer to achieve a form of partial "resolution" when previously confusing variability becomes explainable through the identification of stable regularities. The term **eureka** is used here informally to describe that transition: a point at which the observer's model becomes sufficiently structured that patterns which previously appeared random can be understood as instances of a rule-governed process. This is not intended as a claim that randomness disappears, nor that complete certainty is achieved; rather, it indicates that the observer's representation has improved such that uncertainty becomes more manageable.

From this perspective, learning and modelling can be viewed as iterative: episodes of confusion (where the current model is inadequate) may alternate with episodes of clearer understanding (where regularities are identified and validated), and this cycle may repeat as the operational environment changes or as the system interacts with new contexts. This interpretation motivates the emphasis placed in the thesis on iterative predictive thinking: structured methods are used to surface assumptions, test their adequacy against evidence, and refine the ontology and scenario partition as new classes of conditions are discovered.

More aleatoric randomness means more potential unpredictability for a predictive observer. The impact of randomness on predictive observer confusion depends on how well the chosen mental model handles the relative outlier without requiring changes to the initial categorical system.

We defined the concept Stochastic Confusion as the state of disorder where the complex is random and thus unpredictable. The question then left would be: what would be the state experienced by the predictive observer, where it receives sufficient knowledge and experience with randomness, such that randomness becomes predictable? In other words, predictable randomness. Such a state is truly "confusing," because randomness is inherently unpredictable. This is a kind of positive (relative to the observer) type of confusion, where the observer is confused but in a good way. Maybe we can name it "Eureka"! Eureka is a state where the predictive observer finally gains enough knowledge to predict the randomness of an observation. But Confusion is still present in Eureka! It is understandable, because even If the observer gains more knowledge, the fundamental construct of knowledge remains "randomness" or "uncertainty," so it becomes a doubtful or temporary or false absence of confusion. Learning is a cyclic process between stochastic confusion and Eureka. When the architect reaches Eureka, the architect has defined a system.

E.1.13 Competitive relevance assumption

In long-tailed (or otherwise non-uniform) operational domains, it may be useful to treat "relevance to outcome" as dynamic rather than fixed. The term **competitive relevance assumption** is used here to denote a working hypothesis: as a situation evolves over time, a subset of interacting factors may become persistently more influential for the realised outcome than others, while many remaining factors continue to exist as occasional or context-specific contributors.

This assumption is introduced as a modelling convenience for repeatable reasoning about open, complicated environments. It does not claim that all domains necessarily exhibit a strict competitive-selection process, nor that a single subset of factors will always dominate. Rather, it suggests that, during inquiry cycles, an architect may observe (or infer) that certain interactions repeatedly appear as higher-leverage contributors to safety-relevant outcomes, while others appear less consistently consequential given the current evidence and assumptions.

An illustrative example can be framed as follows. Suppose the outcome of interest is "unsafe train-track zone", and an initial survey identifies a broad set of coexisting complexes:

{moving train, adversarial drone, police presence, trackside vegetation, fencing, power lines, wind, ambient light, passing civilians, etc.}

At early stages, it may be unclear which factors will prove most consequential, and the architect's epistemic uncertainty may be high. As structured inquiry is applied (e.g., evidence gathering, AIC interaction analysis, and iterative scenario reasoning), some factors may be treated as more consistently relevant to the outcome within the current model, while others may be treated as less consistently relevant. One possible representation is to distinguish a provisional "head" (more consistently relevant under current assumptions) from a provisional "tail" (less consistently relevant or more context-dependent):

- **Head (illustrative):** {moving train, adversarial drone, power lines, wind}
- **Tail (illustrative):** {vegetation, fencing, pollen, ambient light, passing civilians, etc.}

Importantly, this partition is not assumed to be permanent. If new mitigations or new interacting systems are introduced (e.g., a security drone intended to deter an adversarial drone), then the relevance structure may change. Under the competitive relevance assumption, mitigation can be interpreted as an attempt to reshape the relevance distribution: demoting certain contributors (e.g., a previously dominant adversarial behaviour) while potentially promoting others (e.g., additional operational dependencies introduced by the mitigation system).

Reinterpreting the 80:20 (Pareto) intuition (illustrative)

The Pareto principle is sometimes used as an informal reminder that a minority of causes may account for a majority of observed effects. Here, it is reinterpreted cautiously and dynamically: as inquiry proceeds and more information is gathered, the set of factors treated as "dominant" may become smaller and better justified, while the set of possible occasional contributors may remain large. This is not offered as a universal quantitative law; it is used as a qualitative heuristic consistent with the observation that complex systems can exhibit a small number of high-leverage interactions alongside a long tail of context-dependent contributors.

Within the AIC framing, this heuristic supports a practical question: which complexes' goals appear to be reinforced by others (supportive interactions that may sustain dominance), and which appear to be obstructed (interactions that may reduce their ability to shape outcomes)? Such questions are intended to guide systematic attention allocation without implying that low-frequency contributors can be dismissed as irrelevant, particularly in safety-critical contexts where tail events may still be consequential.

Generalising the assumption

Although introduced in relation to long-tailed settings, similar reasoning may be applied (with appropriate caution) in other distributional stances. In thin-tailed settings, "competition" may be interpreted as competition to occupy the most common operating modes. In uniformly modelled settings, the assumption may be de-emphasised, because the modelling stance treats categories as equally probable and therefore does not privilege a persistent head. The usefulness of the assumption is therefore context-dependent and should be validated against evidence where possible.

- **Head:** {moving train, adversarial drone, powerlines, wind}
- **Tail:** {police, trackside vegetation, fence, pollen, ambient light, passing civilians, etc.}
- **Head:** {moving train, police, eagle drone, passing civilians, powerlines, wind}
- **Tail:** {adversarial drone, trackside vegetation, fence, pollen, ambient light, etc.}

E.2 Typical and Atypical AIC timing

E.2.1 What AIC Timing Means

Every interaction within a complexity field involves an Appreciation act, an Influence act, and a Control act, but these three acts do not always occur in the same order, at the same time, or with the same degree of deliberation. The concept of AIC timing describes the sequencing and

coordination of these three interaction types, and it applies at two distinct levels that must be carefully distinguished:

1. **Thinking-process timing**, the order in which an Architect's predictive reasoning moves through Appreciation, Influence, and Control when analysing a problem or deriving requirements.
2. **Physical system timing**, the order in which an autonomous agent executes Appreciation, Influence, and Control interactions within its operational environment during actual deployment.

Both levels exhibit the same three timing patterns, Harmonic, Typical Non-Harmonic, and Atypical Non-Harmonic, but the consequences and significance of each pattern differ depending on the level at which it is observed. The following subsections define each timing pattern, then apply it to both levels.

E.2.2 Harmonic Timing

Harmonic timing occurs when all three AIC interactions arise spontaneously, simultaneously, and in natural balance, without requiring deliberate sequencing or imposed order. The system (or the Architect's reasoning) appreciates, controls, and influences as needed, with no interaction type lagging behind or preceding the others in a constraining way.

At the thinking-process level: The Architect simultaneously holds an appreciation of the relevant environmental factors, an understanding of the control actions available, and a model of the influence those actions will produce. No deliberate sequencing is required because the problem space is sufficiently familiar and bounded. Harmonic thinking is characteristic of expert intuition in well-understood domains.

At the physical system level: A mature, well-calibrated autonomous agent performs appreciation (sensing), control (actuation), and influence (effecting change on the environment) in natural, continuous feedback, no timing mismatch occurs between sensing and acting. A well-tuned self-driving car that seamlessly integrates obstacle detection, steering adjustment, and trajectory change in real time exemplifies harmonic system timing.

Why harmonic timing is the exception, not the rule: Harmonic timing presupposes that all three AIC interaction types are available, accurate, and mutually consistent at all times. Under CUPEs, epistemic uncertainty means that Appreciation is inherently incomplete, the agent or Architect does not have full knowledge of the complexity field, and Control is inherently constrained, not all desired effects can be directly effected. Harmonic timing is therefore an idealised limiting case that real systems approximate but rarely achieve.

E.2.3 Typical Non-Harmonic Timing: Clockwise AIC

When harmonic timing is unavailable, because epistemic uncertainty is present, or because the sequence of available information constrains the order of reasoning or action, the most natural and logically consistent sequence is the clockwise AIC order:

Appreciate → Control → Influence (A → C → I)

This sequence is *typical* because it reflects the most intuitive causal logic: an agent must first perceive and understand the relevant aspects of its environment (Appreciate) before it can define or execute the actions required (Control) to produce the desired effects (Influence). Attempting to Control before Appreciating risks acting on incomplete or incorrect information; attempting to Influence without prior Control is structurally impossible since Influence is the emergent outcome of Control actions directed at appreciated environmental states.

At the thinking-process level: The clockwise sequence maps directly to the Architect's requirements derivation process. The Architect first appreciates the operational complexity, identifying the relevant environmental factors, agents, and constraints that cannot themselves be controlled but will affect the system (this is the input to Stage 1, §4.2). The Architect then defines the control interactions, the specific actions the system must be capable of executing to address the identified complexity (this is the output of Stage 3A, §4.3). Finally, the Architect defines the consequent influence, the emergent safety-relevant effect that the combination of appreciation and control is intended to produce (this is captured in safety requirements and HazTOPS mitigations, §4.3.3–§4.3.4). Example: Eagle Drone crossing train tracks:

Table E.3 Implementing the typical AIC iterative thinking cycle

AIC Step	Thinking-Process Activity	Requirement Type Derived
Appreciate: the status of train movement across tracks	Architect identifies that the drone must perceive train position and speed before any crossing decision	Training dataset requirement: trains at various distances and speeds
Control: safe crossing, avoid power lines, obstacles	Architect defines control decisions and constraints for the crossing manoeuvre	Behavioural requirement: control logic for crossing or avoidance
Influence: security effectiveness on the far side of the tracks	Architect mandates that the crossing capability must demonstrably contribute to intrusion deterrence	Test scenario requirement: demonstrating deterrence post-crossing

At the physical system level: The clockwise order is the standard operational loop of any perception-action autonomous system: sense the environment → decide on an action → execute the action → observe the effect. A drone that first appreciates the position of obstacles, then controls its heading to avoid them, and thereby influences its safe passage through the operational zone is executing clockwise AIC timing. Figure E.2 illustrates this standard clockwise sequence.

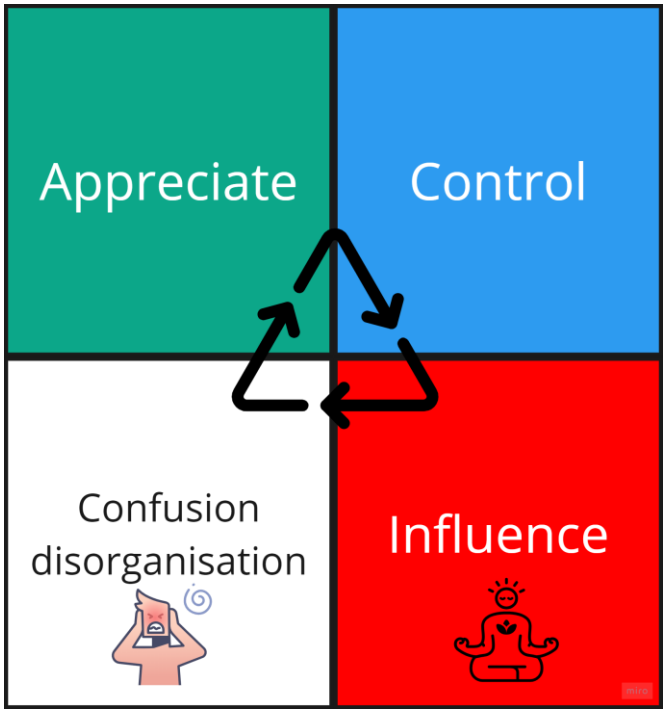

Figure E.2 Typical non-harmonic clockwise AIC timing (Appreciate → Control → Influence)

E.2.4 Atypical Non-Harmonic Timing: Counter-Clockwise or Disordered AIC

Atypical non-harmonic timing occurs when the clockwise AIC sequence is disrupted, either reversed, partially inverted, or disordered, such that Control precedes Appreciation, Influence precedes Control, or the three types occur in a sequence that violates the clockwise logic. Atypical timing is counterintuitive: it involves an agent or thinker acting before fully appreciating or producing influence before the enabling control actions have been properly grounded in appreciation.

At the physical system level: Atypical timing represents an emergent system behaviour that may constitute either a hazard or an affordance, depending on context.

Example: Camera operator scenario: A cameraman lifts a ribbon (Control action) before noticing an approaching robot (late or absent Appreciation). The Control act precedes the Appreciation act, the operator acted without fully understanding the state of the environment.

The resulting Influence (the robot is unexpectedly deflected) was unintended and potentially hazardous.

Example: Autonomous vehicle emergent behaviour: An autonomous car learns to steer into another lane (Control) at a particular time of day before it has detected any obstacle (Appreciation), because historical patterns have made this a reinforced behaviour. The control action precedes appreciation, an emergent soft hazard, because the control action may be triggered in conditions where no obstacle exists, creating an unnecessary risk.

Example: Useful affordance: An autonomous agent that begins collecting environment data (Appreciation) before any control action is triggered may appear to violate clockwise order if a nominal system would initiate control first. However, this inversion may represent a deliberate design choice, pre-appreciating the environment before committing to a control action increases the quality of the subsequent control decision. In this case, atypical timing is an intentional affordance rather than a hazard.

At the thinking-process level: Atypical timing in the Architect's reasoning is one of the primary mechanisms for generating lateral thinking predictions and Black Swan scenarios. When the Architect deliberately inverts or disrupts the clockwise AIC sequence of their own reasoning, asking *"what would happen if the system controlled before it appreciated?"* or *"what if influence preceded control?"*, they are performing the AIC perspective shift that underlies Stage 4 of the approach (§4.5). By deliberately thinking in a counter-clockwise or disordered AIC sequence, the Architect surfaces interaction scenarios that would be structurally invisible under clockwise reasoning, because clockwise reasoning assumes appreciation is always complete before control is attempted.

Example: Predictive lateral thinking application: The Architect asks: *"What could cause the Eagle Drone to execute a control action (alter flight heading) before it has completed appreciation of the surrounding environment?"* This inverted reasoning generates candidate scenarios: electromagnetic interference disrupts sensor data mid-appreciation cycle; a software fault triggers a control response before the perception pipeline has completed; an adversarial signal mimics a high-priority control trigger, bypassing the appreciation stage. Each of these constitutes a potential Black Swan hazard that clockwise-reasoning alone would not generate, because clockwise reasoning assumes the appreciation step is always completed before control is engaged.

E.2.5 Summary: AIC Timing as a Predictive Tool

The three AIC timing patterns and their significance at both levels are summarised in the following table.

Table E.4 Implementing the atypical AIC iterative thinking cycle

Timing Type	Sequence	Thinking-Process Significance
Harmonic	A, C, I simultaneously	Expert intuition in familiar domains; not generative of novel predictions
Typical Non-Harmonic (Clockwise)	A → C → I	Standard requirements derivation logic: the Architect's natural reasoning sequence for known scenarios
Atypical Non-Harmonic (Counter-clockwise Disordered)	C → A → I, or I → A → C, or other inversions	Lateral thinking trigger: deliberate inversion of reasoning order generates Black Swan candidate scenarios (Stage 4, §4.5)

The key insight motivating Stage 4's AIC perspective shift (§4.5) is that typical clockwise reasoning, because it may be a natural and efficient thinking sequence, systematically excludes scenarios that arise from atypical timing at the physical system level. The Architect who only ever reasons clockwise will only ever predict scenarios that are reachable by clockwise physical system behaviour. Scenarios produced by counter-clockwise or disordered physical system timing are therefore structurally invisible to the clockwise-reasoning Architect, precisely the condition that defines a Black Swan from the Architect's perspective. Deliberately inducing atypical timing in the thinking process is therefore a principled mechanism for surfacing these otherwise invisible scenarios.

E.3 Safety and Security Concepts in the Context of AIC

Safety: When we think of safety, we think it of the following terms:

> Do not cause unintended physical harm to yourself or others by effectively appreciating and controlling your own and environmental behaviours.

In other words, Safety refers to a system's ability to influence and prevent harm or danger to the primary purpose of the engineered system. In the definition, we also include the anticipation of causing harm or danger to other complexes' primary purposes within the complexity field. It often concerns preventing harm to humans, property, or the environment. It encompasses the system’s capability to operate safely under normal conditions and to mitigate risks arising from

Novel circumstances. Safety is evaluated by identifying hard and soft safety hazards and implementing mitigating interactions or capabilities.

Safety Hazards: A direct, immediate harm within the system's complexity field or external environment that physically harms the system or its surroundings[12, 13].

General modes of a Safety Hazard:

- **Loss of required Appreciation:** The system fails to recognise critical environmental factors or internal states, leading to unsafe behaviours (e.g., misperception of obstacles, sensor blindness in critical conditions).
- **Loss of required Control:** The system loses the ability to directly regulate its behaviour or an environmental system directly, causing unintended physical harm (e.g., an autonomous vehicle failing to brake due to control malfunction).
- **Loss of required Influence:** The system loses the ability to indirectly regulate other systems in its environment or itself (e.g. an autonomous car loses the ability to influence another autonomous car's decision to not overtake in a particular situation [like a sleeping driver]).
- **Emergent Risk:** Unintended system-environment interactions create new hazards, even when no immediate failure occurs (e.g., an AI-driven surveillance drone unintentionally distracting drivers and causing accidents).

Safety Risk: The likelihood of a safety hazard occurring[14], considering both current and anticipated complexity field structures.

Security: When we think of security in general, we think of the following:

> Securing the longevity of a desired autonomous system's appreciation, influence, and control effectiveness (over changes in the complexity field) from unaccounted-for emergent affordances or intentional unauthorised influence or control over the system.

Security may focus on protecting the system's ability to govern (control) intended situations from malicious attacks, unauthorised access, or intentional threats[14]. It ensures the system's integrity, availability, and confidentiality, safeguarding it from being controlled by undesirable influences (human or otherwise). While Safety is a broader concept related to protecting the system's primary purpose and others, security is more focused on securing the system of concern's ability to exert governance and control through its auxiliary control goals and actions.

Security Threats: Any emergent affordance (unaccounted for during design time), unauthorised influence, or adversarial control that compromises an autonomous system's ability to appreciate, influence, or control its operational environment, leading to potential degradation, exploitation, or loss of system integrity over time.

General modes of a Security Threat:

- **Appreciation Compromise**: The system's ability to recognise and interpret its environment is manipulated, blocked, or deceived (e.g., sensor spoofing, adversarial perturbations).
- **Influence Disruption**: The system's ability to modify or interact with its environment is hijacked, altered, or constrained (e.g., jamming, misinformation, false data injection).
- **Control Hijacking**: The system's ability to execute autonomous decisions is designated, overridden, or exploited (e.g., hacking, unauthorised access, AI model poisoning).
- **Machine's emergent defiance**: This is when the machine makes an artificially intended decision that does not align with the architect's intent and perspective of what is right or wrong based on the machine's self-assessment of what is best for itself or others.

A security threat does not necessarily cause immediate harm but introduces vulnerabilities that could be exploited over time, leading to unintended system behaviours, failure modes, or adversarial control. Security threats can originate from both external actors (cyber-attacks, adversarial entities) and internal system weaknesses (unsecured affordances, design oversights).

Situations that compromise the system's control are often caused by vulnerabilities in the system's complexity field. These vulnerabilities can be attributed to the architect's lack of knowledge or foresight on how the whole complexity field (the system being part of it) behaves over time. This accumulates as a lack of appreciation of the whole.

- **Example**: A hacker can intercept and control an autonomous delivery robot with a vulnerable wireless communication system. The system's appreciative interaction with communication systems (e.g., GPS) becomes a security vulnerability.
- **In the context of AIC**: Such vulnerabilities threaten the system's influence and control, potentially leading to obstructive forces taking over. Appreciative interactions are the apparent interactions that can open complexes to exploitation if other actors control the appreciated complexes.

Security Risk: The likelihood of a security threat manifesting, considering current and future complexity field structures.

Negative Affordances and Potential of Security Threats:

Constraining unknown possible affordances could be considered a straightforward theme differentiating security from safety. For example, the Eagle Robot flying over train tracks may be secured and safe. However, the nature of having a flying drone over train tracks behind local houses provides several side-affordances that may not be part of the design intent:

- It provides excellent training content for adversarial counter-security drones. For example, it can be filmed and photographed easily and used for training adversarial platforms.
- It provides local news with content that can be exploited for political campaigns or other social engineering purposes (an example of a soft hazard).

These cases are not safety concerns; they do not cause direct physical harm to the system, which makes them a distinct security concern. Therefore, we believe the "exploitation of possible affordances" is purely security-related. We capture those affordances in the HazTops method (section F.5) under the "Opportunities/Affordance" concept.

Key Distinctions Between Safety and Security

We distinguish some key conceptual differences between Safety and Security. We believe that confusing security requirements with safety requirements and vice versa could introduce inefficiencies in the design. For example,

- **Safety requirements:** The Eagle Drone must not fly too close to train catenary power lines to prevent an electrical short circuit that could cause fire.
- **Security requirement:** Jamming Attack: If adversaries use radio-frequency (RF) jamming, they could disrupt the Eagle Drone's GPS navigation, causing it to lose control and potentially crash or exit the secured perimeter.

A jamming attack is not a safety concern, as it may not directly cause physical damage. Given enough gust of wind energy, patrolling very close to powerlines will most likely cause a collision and fire. If patrolling next powerlines is misclassified as a security concern, the architect might assume the risk only arises from an intentional attack, such as adversarial hacking or deliberate sabotage, rather than an operational safety risk inherent to normal environmental conditions (e.g., wind gusts, loss of stability, miscalculated flight paths). This may reduce the likelihood of it occurring in the architect's risk analysis thought process, thus assigning a lower priority for mitigation. Therefore, we attempt to explain the difference between Safety and Security below in order to help the architect make better engineering judgments:

Table E.5 The difference between Safety and Security Engineering

Safety	Security
○ Deals with preventing direct physical harm, regardless of whether the system is reliable or not. ○ A broader concept concerns the impact on the system's primary purpose and its impact on other complex primary purposes within the complexity field as a whole. ○ **Example:** Ensuring a drone avoids collisions in urban environments. ○ A broken-down micro drone landing in an area that does not present a direct hazard to the environment can be a safe drone for itself and its environment.	○ Deals with protecting the system from malicious or unauthorised influence or control. When an authorised or vetted controller becomes an appreciating stakeholder of the engineered system, the unauthorised controller or user becomes an appreciated system. ○ Deals with preventing negative affordance exploitation by users or unauthorised entities. ○ **Example:** Preventing hackers from intercepting and taking control of a drone's navigation system. ○ A broken-down micro drone landing in an area that does not present a direct environmental hazard is not necessarily secured because it is vulnerable to theft. Anyone can pick it, take its parts, reverse engineer it and reuse them for something else.

Interrelation between safety and security: A lack of security can compromise safety[12]. For example, a hacked autonomous vehicle could become dangerous, blending security and safety concerns. Conversely, a system can be secure but unsafe or safe but insecure, depending on the context. The following is an illustrative example of how Safety and Security can interplay during the design process:

Table E.6 Illustrative Scenarios in Autonomous Systems

Safety & Security integration type scenario	Potential conflict of interest or contradiction examples
Unsafe but Secure	**Scenario 1:** A drone operates based on a secured training process but has a hidden uncertainty in the training set of its

	obstacle-avoidance algorithm, risking collisions. 1. A potential appreciative interaction between a trained model and the hidden biases in the training data. 2. The architect identified a problem concerning an insecure training dataset, which led the architect to define a limitation on access to the training set. This security requirement may weaken dataset validation, resulting in hidden biases that could lead to failure to avoid issues and obstacles. **Scenario 2:** The lock on an autonomous car breaks down while people are inside**.** The car is secure but potentially unsafe in case of fire. **Explanation**: Security measures prevent external interference, but internal faults make the system unsafe. So, if the security requirement imposes stronger lock mechanisms, this may mean jeopardising people's safety in case of fire due to a fault in the LiOn battery sub-system.
Insecure but Safe	**Scenario 3**: An autonomous car operates safely under some environmental conditions but is vulnerable to theft. A car may not be operational (safe), but the door is unlocked as per a safety mitigation requirement to make the vehicle unlocked when broken down, or the car could be towed away. **Explanation**: The system is safe but remains insecure due to potential theft or undesirable relocation. Another example is smoke inside the autonomous car. The internal car perception system identifies the presence of a child, deactivates the child, and the child opens the door while the car is moving. The car is safe but insecure.
Unsafe and Insecure	**Scenario**: Cyber-warfare compromises a fleet of drones, causing them to lose control and collide with obstacles.

	Explanation: The drones are both insecure (due to the attack) and unsafe (causing harm).
Secure and Safe	**Scenario**: An autonomous vehicle employs advanced encryption and intrusion detection while operating safely under rigorous testing. **Explanation**: The system maintains security and safety, minimising risks to users and the environment.

E.3.1 The Intrinsic Hard Hazard of Appreciation

All appreciative interactions are the Complexity field's weakest links for maintaining a primary purpose.

By "weakest link," I mean the likelihood of a complexity losing purpose may come from its appreciative interactions. This is because the nature of appreciation is basically "complete dependence with no possible capability to influence the behaviour of the appreciated". This is a very useful notion because the architect must think of appreciative interactions to enable control, and by naming those appreciative interactions, the architect can immediately anticipate hazardous scenarios originating from the appreciation.

The hazard is related to the potential safety and security vulnerability associated with appreciation. When system A is in an appreciative interaction with some unavoidably influential appreciated situation D, whoever controls or influences D unavoidably influences or controls system A. This means that for any appreciative relationship, the appreciative system must also appreciate other situations that can influence or control its appreciated situation. For example, an autonomous car (system A) appreciates visible traffic signs (situation D); however, whoever or whatever influences or controls the clarity and visibility of a traffic sign unavoidably influences or controls any passing autonomous vehicle. This is an important concept, especially in cyber-security, as whenever an appreciative interaction is defined, a cyber-security or safety threat is inherently associated with appreciative interactions. If appreciative interactions are exploited, the system is open to whatever exploiting system's goal or PrimeP.

E.3.2 Soft Hazards in Complexity Fields

A **Soft Hazard** is a situation that impacts the realisation and maintenance of an ideal operational environment system where an action, decision, or inaction may not directly threaten or harm the immediate safety or functionality of the system of interest. Instead, Soft Hazards pose intermediate or long-term risks or consequences that negatively impact the

success of the deployment mission, broader environment, or social context, where autonomous systems may eventually be affected by it. For example, we consider “Ethical Threats”, a re-wording of the concept “Ethical Hazard”[15] as an example of Soft Hazards; identifying such hazards is often philosophical and subjective depending on the designer's ethical awareness and culture, as they may involve decisions about intent, fairness, justice, privacy, or the unintended side effects of the deployment of the autonomous system.

The main point here is that we are considering an interaction within the overall problem complexity field which the autonomous systems are part, not necessarily where the autonomous systems are directly involved. An example of a soft hazard is an ethical hazard. Below is a figure that explains a soft hazard:

Let us take the Eagle Drone, for example, a road rage incident between a car driving over the bridge and a pedestrian may not be of Eagle Drone's direct concern. However, this ethical hazard impacts maintaining an ideal operational environment, which includes autonomous systems. Perhaps it may affect the success of drone deployment in the long term, such as blaming the drone's presence at some part of the train track zone, which causes pedestrians not to pay attention while crossing the road, thus leading to question the overall safety of local people or even be subject to criminal prosecution. It may also lead to bad press for the organisation operating the drone and, thus, potential early decommission and deployment failure. The thinking type that helps discover such scenarios is the main problem solver for long-tailed complexity problems.

Soft hazards can refer to the potential of becoming a physical hazard in the future due to a change in the system of interactions, even if it is not currently a hazard. In simpler terms, they are potential future dangers or causes of harm.

In the context of autonomous system design, Soft Hazards arise when the system behaves in a way that, while safe or functional in isolation, has negative implications for other stakeholders or broader systems. For instance, a security drone may patrol a train track safely. Still, its surveillance capabilities could unintentionally infringe on privacy rights or create social inequities, thus posing a Soft Hazard even though it performs its intended function without safety issues.

Example:

In the design process described, the AIC mental model originally conceptualised a police officer performing a control action **“remove”** on trees, which was initially seen as a neutral effect action concerning the trees. However, upon reflection, we realised it is better to assume the worst in this case to have some mitigation in place, so we updated the effect to be “obstructive”

representing a potential the police may have ill-intent towards local trees as they prioritise the success of the autonomous systems deployment over the wellbeing of the natural environment which affects local people in return and the problem domain as a whole. This is an example of a Soft Hazard; it emerges from an interaction between two complexes in the problem domain, where the system is not a party. Even if this ill intent does not impact the safety or operation of the system itself, it could negatively affect the broader context, such as environmental sustainability or social equity. This scenario highlights the role of ethical considerations in identifying and mitigating soft hazards.

E.3.3 Soft Hazard vs. Hard Hazard

We delineate two types of hazards to be considered in AIC systems Approach:

Table E.7 Types of Hazards in AIC Systems Approach

Hard Safety Hazard	**Soft Safety Hazard**
Directly threatens the operation or users of the system, potentially leading to physical harm or autonomous systems failure.	Soft hazards are risks to the realisation and maintainability of ideal operational environment dynamics, which autonomous systems are part of. These may include aspects such as societal, environmental, or ethical norms, such as privacy violations, fairness, bias, or negative long-term consequences, but they do not immediately impact the safety of the system itself. ○ A form of Black Swan hazards, which are scenarios that are deemed to be unlikely or rare or hard to believe to be of concern, yet if they do occur, will have a surprising impact on safety and security. ○ A potential future harm or threat that could evolve into a hard safety hazard due to changes in the complexity field. This may not pose an immediate danger, but it requires foresight to anticipate and mitigate. **Soft Hazards can emerge from** interactions

	within the complexity field that have to appreciate the system of interest behaviour, and the architect may miss their impact in the future. This can be predicted using an AIC perspective shift.
Example: An autonomous vehicle fails to detect a pedestrian due to hidden bias in its object detection algorithm, creating an immediate risk to the pedestrian, vehicle Occupants, and nearby property. The hidden bias becomes a source of malfunction within the system.	**Examples**: ▪ **Wildlife Interaction**: Animals near a train track zone monitored by a security drone may not present an immediate hazard but could become one if deployment conditions change and animal habits change with it. ▪ **Weather Conditions**: Adverse weather outside a drone's immediate operational zone might impact future deployment. For example, an erupted volcano in a different country could alter natural light conditions across an area where autonomous systems (trained on normal conditions) operate. ▪ **Human Factors**: Local population temperament could evolve into a safety threat in specific contexts. ▪ **Human or free-will agents' intent** is also a soft hazard since it can only become evident over time as their complexity changes. This also include the architect intent.

Properties of Soft Hazards for Safety Design Consideration:

- **Subjectivity:** Soft Hazards are often subjective, depending on the ethical values and bar set by the architect. What one may consider a Soft Hazard (e.g., excessive security drone surveillance) may not be regarded as a hazard by others.

- **Mitigation:** By identifying these Soft Hazards early in the design process, more constraints can be placed in the operational design domain, and the system can be modified to mitigate them, contributing to creating a **Trustworthiness Case**. This case demonstrates that the system's deployment is functional, safe, secure, and ethically sound.
- **Time Transcendence:** Soft hazards can happen in the future. During design time, the complexity may not pose any danger to the robot; however, some aspects of the operational domain may become hazardous in the future. For example, drones flying over the train track zone over time may affect the presence of bees and local birds, triggering community resistance to terminate the robot's operation, thereby affecting the system.

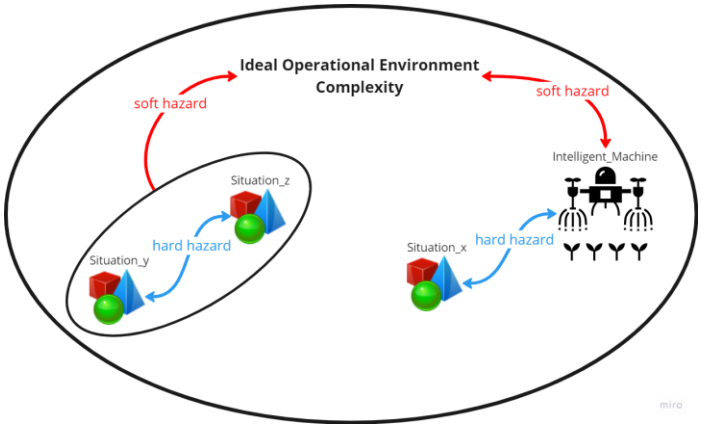

Figure E.3 Soft Hazard Model

Figure E.3 illustrates the dynamic interaction between Soft and Hard Hazards within an Ideal Operational Environment of a complex, intelligent system. The model demonstrates how soft hazards arise from broader environmental, ethical, or social interactions that may not pose immediate threats to the safety or operation of the system (e.g., an autonomous drone) but can negatively affect the long-term viability of its deployment mission or the societal context in which it operates.

In the figure, Situation X and Situation Z represent distinct operational scenarios within the system's environment. Hard hazards, which include immediate operational threats (e.g., physical collisions), are marked by blue arrows, indicating direct risks to the system's functionality. In contrast, red arrows symbolise soft hazards, reflecting broader, less immediate concerns—such as ethical or societal issues—that emerge from the system's indirect involvement in or impact on the environment.

For example, soft hazards may arise from intended or unintended consequences of the system's behaviour, such as a surveillance drone infringing on privacy. This could lead to a loss

of public trust or regulatory backlash over time. This model emphasises that while soft hazards may not directly affect the system's immediate operation, they pose a significant risk to maintaining an ideal operational environment. Therefore, it is crucial to identify and develop mitigation strategies for these hazards early in the design process.

E.4 Lateral and Vertical Predictive Thinking thought experiment.

This section investigates vertical and lateral predictive thinking as intellectual phenomena through a controlled thought experiment. The experiment uses a concrete autonomous systems scenario to expose the structural differences between the two thinking modes before deriving the analytical consequences relevant to the AIC-based approach developed in this thesis.

E.4.1 Thought Experiment Setup: The Autonomous Security Drone Problem

Consider an Architect tasked with designing the detection and classification capability of an autonomous security drone (the Eagle Drone) charged with monitoring a train track zone against unauthorised intrusions. The Architect's starting brief is a single operational rule:

"Any object on the track that is not a scheduled train is a potential threat".

This rule is deliberately simple. The thought experiment asks: given this identical starting point, how does the Architect's subsequent predictive reasoning differ depending on whether they proceed as a vertical thinker or a lateral thinker? And what are the consequences of each mode for the completeness and robustness of the resulting system design?

E.4.2 Vertical Predictive Thinking: The Sequential Refinement Process

A vertical thinker treats the starting rule as a provisional axiom and proceeds by deductive refinement, building a chain of theorems that extend, qualify, and correct the axiom in response to observed contradictions.

Step 1: Initial axiom: The Architect accepts the rule as stated: any non-scheduled object on the track is a threat. The inference chain proceeds from this single premise.

Step 2: First observation and contradiction: The Architect imagines the drone detecting a maintenance crew's equipment left on the track. The initial rule classifies this as a threat. The Architect recognises this as a misclassification, a contradiction between the rule and the observed reality.

Step 3: Rule refinement: To resolve the contradiction, the Architect refines the axiom:

"Any object on the track that is not a scheduled train or authorised maintenance equipment is a threat".

A supporting theorem is added: *"Maintenance equipment can be authorised by tagging it with specific markers detectable by the drone"*.

Step 4: Iterative extension: Each new observation that contradicts the current rule set triggers a further refinement. The Architect builds confidence incrementally, each iteration extends the axiomatic chain and reduces the set of known misclassification cases.

Step 5: Outcome: The system converges on a refined rule set: *"Authorised objects, maintenance equipment, scheduled trains, are not threats; all other objects require further classification"*. The system becomes precise within its defined categories. It handles known cases reliably.

Critical observation: At no point in this process does the Architect consider whether a structurally fixed environmental element (say, a lamp post) could cause the drone to perform an evasive manoeuvre without being physically in the drone's path. This is not because the Architect is careless; it is because the vertical reasoning process operates within a domino-logic structure: ***cause → effect → refinement → next cause***. A stationary lamp post casting a shadow that the drone's perception system misclassifies as a physical barrier may take the vertical thinker's axiomatic chain a long period of iterations and refinements. It is not a deviation from a known rule; it is an unmodelled scenario class. Then the thinking process becomes dependent on time, and analytical effort may be limited by the intrinsic optimisation rule that vertical thinking is intended to facilitate. After all, the architect is confined by time and task constraints, thus reducing the possibility of reaching remote lateral realisation. The vertical thinker's tacit assumption is that the problem domain can be fully characterised by iterating on known contradictions. Events that produce no contradiction, because they were never predicted, are structurally invisible.

Structural property identified: Vertical thinking relies on a *metastable[16]* axiomatic system, thus produces a metastable theoretical system about complexity. Its correctness is sensitive to the founding assumptions: a small change in real-world complexity that contradicts a founding axiom, for example, the discovery that track-zone vegetation moves in ways perceptually indistinguishable from low-altitude drone flight, can invalidate a significant portion of the downstream inference chain. The system is not fragile in the sense of being poorly constructed; it is fragile in the sense that its stability depends entirely on the completeness of the founding

assumption set, which is a function of the Architect's prior knowledge, an inherently finite quantity.

E.4.3 Lateral Predictive Thinking: The Analogical Exploration Process

A lateral thinker does not begin with a rule. They begin with an open observation field and proceed by generating analogies, hypotheses, and associations across interaction categories, including those that appear irrelevant, counterintuitive, or contradictory under the vertical framing.

Step 1: Initial approach: The Architect suspends the starting rule entirely. The train track zone is treated as an open complexity field in which any entity could interact with any other entity in ways not yet characterised. The operational question is not *"does this object violate rule X?"* but *"what kinds of interactions could this environment produce, including ones I have not yet imagined?"*

Step 2: First observation, multiple hypotheses generated: The Architect imagines the drone detecting an object on the track. Rather than classifying it against the rule, the Architect generates multiple candidate interpretations simultaneously:

- Could it be maintenance equipment seen in another context?
- Could it be an unauthorised object that resembles maintenance equipment?
- Could it be an entirely new category, an animal, a weather artefact, a discarded object?
- Could the drone's perception system itself be producing a false positive due to environmental conditions, shadow, reflection, vegetation movement?

No hypothesis is dismissed a priori. Each is treated as equally worthy of consideration at this stage.

Step 3: Analogical recontextualisation: The Architect draws an analogy from an unrelated domain. Consider cloud formations in meteorology: low-altitude cumulus clouds can exhibit visual morphologies indistinguishable from snow-covered tree canopies under certain lighting conditions. Mapping this into the train track zone operational domain: under certain atmospheric and lighting conditions, a low-altitude cloud formation above the track zone could be classified by the Eagle Drone's perception system as a vegetation obstacle, triggering an unnecessary avoidance manoeuvre and creating a temporary surveillance gap that an Adversarial Drone could exploit.

This scenario is not reachable by deductive refinement from the starting rule. It is reachable only by generating a cross-domain analogy that deliberately suspends the assumption that meteorological entities are irrelevant to drone perception.

Step 4: Expanding the observation class set: Rather than converging on a refined rule, the Architect adds the new observation as a candidate training class: *"atmospheric conditions producing false obstacle classifications"*. The system's understanding grows by accretion of new classes rather than by refinement of existing rules.

Step 5: Outcome: The lateral thinker produces a broader, more diverse set of candidate scenarios, including counterintuitive and low-probability ones. The system remains open to discovering unexpected interaction patterns. However, compared to the vertical outcome, the lateral process does not converge, it diverges. Without a boundary condition, the lateral thinker could continue generating hypotheses indefinitely, never reaching a stable, implementable design specification.

Critical observation: Lateral thinking's principal strength, its refusal to dismiss atypical scenarios, is simultaneously its principal weakness in engineering contexts: unbounded lateral thinking does not produce actionable design requirements. It produces a potentially unlimited set of candidate scenarios, of which only a subset are operationally relevant and practically addressable.

E.4.4 Spontaneous Realisation: What the Thought Experiment Reveals About Architect Attention

As the thought experiment in §E.4.2 and §E.4.3 was being constructed, something became apparent that had not been the starting motivation for the exercise. The thought experiment had been designed to expose the structural differences between vertical and lateral reasoning processes, how conclusions are reached, how contradictions are handled, how scenarios are generated. But midway through tracing the vertical thinker's sequential refinement steps, a different and more fundamental question surfaced:

Why does the vertical thinker never reach the lamp post shadow scenario or the cloud-formation scenario, not even after many iterations?

The immediate answer is that these scenarios do not arise as contradictions within the vertical thinker's axiomatic chain. But this answer, on closer inspection, is not a property of the scenarios themselves, it is a property of where the vertical thinker is looking. The lamp post shadow scenario and the cloud-formation scenario are not logically unreachable; they are

attentionally unreachable under vertical reasoning. The vertical thinker's cognitive resources, their analytical attention, are fully allocated to the head of the interaction distribution: trains, maintenance equipment, authorised objects, and known misclassification categories. The tail of the distribution, environmental interactions, perceptual artefacts, atmospheric analogies, receives no allocation. Not because the Architect decided to ignore it, but because the vertical reasoning process never directs attention toward it in the first place.

This was the realisation: the difference between vertical and lateral thinking is not primarily a difference in reasoning logic. It is primarily a difference in attention allocation.

The vertical thinker and the lateral thinker have access to the same complexity field. They begin with the same starting brief. The scenarios available to be predicted are identical. What differs is where each thinker's attention is directed during the analysis, which categories of interaction are considered, how much cognitive effort is invested in each, and which categories are effectively invisible because attention never reaches them.

E.4.4.1 From Attention Allocation to Prediction Completeness

Once this was recognised, a second realisation followed immediately. If attention allocation determines which scenarios are predicted, then the completeness of any hazard analysis is not solely a function of the method's logical machinery, it is also a function of the Architect's attention distribution across the interaction space.

A method that directs attention exclusively toward high-salience, frequently observed interaction categories will produce a complete analysis of those categories and a systematically empty analysis of everything else. The gaps in the analysis are not random, they are structured by the attention distribution that the method induces. This means that the gaps are, in principle, diagnosable. If the Architect's attention distribution can be made visible, measured, plotted, and inspected, then the coverage gaps it produces can be identified before they manifest as undetected hazards in the deployed system.

This led to a third realisation, which became the central motivation for examining the existing safety literature from an attention perspective: if safety analysis methods systematically bias Architect attention toward specific interaction categories, then the hazards they miss are not random omissions, they are method-induced blind spots. And if this is true, then comparing methods not by the hazards they find but by the attention distributions they produce might reveal something that conventional method evaluation misses entirely.

E.4.4.2 Connection to Peer-Reviewed Research on Attention and Cognitive Bias in Safety Analysis

This realisation was not, it turned out, entirely novel. A body of peer-reviewed literature had already documented the role of cognitive bias in safety-critical design and hazard analysis, though not, at the time of this research, with the specific framing of attention distribution as a measurable, method-induced property (see thesis §2.1.5). However, the literature didn't seem to provide a tool to diagnose or monitor attention distribution during analysis.

E.4.4.3 Connection to the Research Problem Definition

This chain of realisations maps directly to the research problem as formally defined in §1.3.1:

ASE Problem 1, the risk of misguided Architect attention leading to missing Black Swan operational scenarios, is now recognisable as the attentional problem identified in the thought experiment. The phrase "misguided Architect attention" may be the phenomenon observed: vertical thinking systematically misguides attention toward the salient head of the interaction distribution, producing structured coverage blindness in the tail, where Black Swan scenarios reside. The thought experiment is therefore not merely an illustrative device; it is a derivation of the research problem from first principles.

The thought experiment also motivated the examination of existing safety methods in §2.3 from an attention-distribution perspective rather than from a logical-completeness perspective. The question asked of HAZOP, STAMP/STPA, and FRAM was not "do these methods find the right hazards?" but "where do these methods direct the Architect's attention, and what does that distribution imply about the scenarios they structurally cannot reach?" This reframing is what led to the ATD evaluation approach in §2.3.2, which plots the attention distribution each method produces when applied to the same problem and measures its shape against the three ATD modes defined in §3.5.1.

E.4.4.4 Connection to Analytical Attention Management

The final step in the chain of realisations was the recognition that if Architect attention can be misallocated, and if that misallocation is method-induced and structurally diagnosable, then monitoring and managing attention during analysis is not merely a useful heuristic. It is a necessary engineering control for hazard analysis under CUPEs. This is the motivation for the Analytical Attention Management framework in §3.5.1. The three ATD modes, Normal, Uniform, and Long-tailed, are not arbitrary categories. They are the three qualitatively distinct patterns that emerge from the thought experiment analysis:

The diagnostic value of the ATD histogram, as described in §3.5.2, follows directly from this mapping. If the Architect's predicted situation frequency distribution can be plotted as a histogram during or after a hazard analysis session, the shape of that histogram reveals the thinking mode in operation and, consequently, the type of coverage blindness the analysis is likely to exhibit. Zero-attention bins in the tail of a Normal ATD are not random gaps, they are precisely the locations in the interaction space where Black Swan scenarios reside. Making them visible is the first step toward reducing the epistemic risk they represent.

This is why the ATD histogram, kurtosis measure, and coverage blindness metric (§3.5.2) were developed: not as theoretical constructs, but as practical diagnostic instruments motivated by the attentional insight first encountered in this thought experiment. The thought experiment revealed that the problem is attentional; the ATD framework provides the tools to measure, diagnose, and address it.

E.4.5 Structural Comparison: Axiomatic Stability versus Coverage Completeness

The difference between vertical and lateral thinking can be formalised in terms of the stability–completeness trade-off in the Architect's predictive model:

Vertical thinking produces a *stable but incomplete* model. Its axiomatic chain is internally consistent and converges on precise predictions for known categories. However, its completeness is bounded by the founding assumptions, scenarios outside the initial axiom space are structurally invisible. The model is *metastable*: small perturbations in the real-world complexity field that contradict founding assumptions produce disproportionate degradation in the inference chain (analogous to a butterfly effect in chaotic systems). The accumulative bias of iterative vertical refinement narrows the Architect's attention distribution toward the head of the interaction distribution, frequently observed, salient events, whilst systematically neglecting the tail.

Lateral thinking produces a *complete but unstable* model. Its refusal to dismiss any interaction category means that tail events, including Black Swan scenarios, are not prematurely excluded from the prediction space. However, without a convergence mechanism, the lateral thinker's prediction set grows without bound, producing an undifferentiated set of equally weighted hypotheses that cannot be directly translated into prioritised design requirements. The model is *maximally inclusive* but practically unimplementable without further constraint.

The structural implication is that neither mode alone is sufficient for designing autonomous systems operating under Compounded Unpredictability and Epistemic Uncertainty (CUPEs). A

synthesis is required, one that inherits the coverage completeness of lateral thinking and the actionable convergence of vertical thinking.

E.4.6 Mapping Thought Experiment Activities to Thesis Concepts

The following table maps the observed activities and results from the thought experiment directly to the concepts defined in §3.5.3 of the thesis.

Table E.8 Example lateral rule and predictive question

Thought Experiment Activity / Observation	Thesis Concept (§)	Mapping
Vertical thinker begins with a single axiom and refines it through contradiction resolution	Vertical Thinking (V-thinking), §3.5.3(a)	V-thinking assumes deterministic, systematic causality, with each effect caused by a single cause; the starting rule operationalises this assumption. Contradiction resolution corresponds to domino-logic sequential attention advancement.
Vertical thinker's model becomes sensitive to founding assumption violations	V-thinking metastability, §3.5.3(a)	The thesis characterises V-thinking as assuming rare events are negligible. The lamp post shadow and vegetation movement scenarios demonstrate the failure mode: events outside the founding axiom space are structurally invisible.
Vertical thinkers never generate the lamp post shadow or cloud-formation scenarios	ATD concentration / coverage blindness, §3.5.1(a), §3.5.2	The Normal ATD mode (concentrated head, zero-bin tail) describes precisely this pattern: frequently observed interactions are stabilised as constants; rare interaction categories accumulate zero analytical attention.
Lateral thinker suspends the starting rule and treats all interaction categories as equally probable	Lateral Thinking (L-thinking), §3.5.3(b)	L-thinking assigns equal analytical attention a priori to all interaction categories, operationalising the maximum-entropy principle as a heuristic analogue (§3.5.3(b)). The lateral thinker's Step 1 is the explicit enactment of this principle.
Lateral thinker generates cloud-formation / lamp post shadow scenarios via cross-domain analogy	Structured Analogising for Lateral Thinking, §3.5.3(d)	The three-step analogising procedure (broaden base perspective → generate cross-domain analogies → formulate predictions) is exactly enacted: meteorological domain → train track zone mapping produces the cloud-formation scenario.
Lateral thinker produces an unbounded set of hypotheses that does not converge	L-thinking divergence failure mode, §3.5.3(b)	The thesis explicitly identifies the *"never finish failure mode"* of pure lateral thinking. The thought experiment demonstrates this: without a convergence mechanism, lateral prediction expands indefinitely.
Neither vertical nor lateral alone produces a complete, actionable specification	Transformative Thinking (T-thinking), §3.5.3(c)	T-thinking alternates between lateral phase (uniform attention distribution across all categories) and vertical phase (concentrated depth on most consequential interactions). The thought experiment motivates this synthesis: completeness requires lateral breadth;

		actionability requires vertical depth.
ATD histogram of vertical thinker shows concentrated head, zero-bin tail	Normal ATD mode, §3.5.1(a)	The vertical thinker's prediction set maps directly to a Normal ATD: high-salience categories (scheduled trains, maintenance equipment) attract all attention; environmental and perceptual interaction categories (vegetation, shadows, atmospheric artefacts) are zero-attention bins.
ATD histogram of lateral thinker shows uniform distribution across categories	Uniform ATD mode, §3.5.1(b)	The lateral thinker's equal-probability treatment of all interaction categories produces a uniform ATD, no category is privileged over any other a priori.
AIC framework integrates A (lateral exploration) and C (vertical convergence) through I	AIC Balanced Predictive Thought Process, §3.2, §E.4.2	Appreciation operationalises lateral exploration (infinite possibility space); Control operationalises vertical convergence (finite, goal-directed actions); Influence represents the emergent value produced by their interaction, the synthesis that T-thinking aims to achieve structurally.
Lamp post shadow / cloud-formation scenarios constitute novel, non-obvious interaction classes	Black Swan scenarios, §3.7	These scenarios satisfy the Black Swan criteria: original (not predicted by conventional vertical analysis), atypical (outside the normal ATD head), and potentially impactful on ML perception reliability.
ArcMatrix and Actions Matrix operationalise bounded lateral thinking	Bounded L-thinking via ArcMatrix, §3.5.3(b), §4.8	The thesis states that bounded L-thinking is operationalised through the ArcMatrix (§4.8), which constrains the interaction space to the defined AIC ontology, preventing the unbounded divergence failure mode whilst preserving lateral coverage. Note 9 in §3.5.3(d) explicitly identifies this as the basis for the ArcMatrix tool.

E.4.6 Inference: Why the AIC Approach is a T-Thinking Approach

The thought experiment demonstrates that:

1. In an open, complicated problem space, it takes prohibitively longer to construct a reliable, complete axiomatic system about all possible behaviours. Since vertical thinking operates with the intent of optimising time and analytical effort to the reliability of axioms and theorems, a purely vertical Architect may systematically fail to reach intellectually conclusions about interaction classes that lie outside the founding axiom space, including precisely the low-salience, high-impact scenarios that constitute Black Swan hazards.
2. A purely lateral Architect generates these classes but cannot converge on actionable, prioritised design requirements without a bounding mechanism.
3. The AIC framework, through the structured interplay of Appreciation (lateral), Control (vertical), and Influence (emergent synthesis), provides the bounding and convergence mechanism that prevents lateral divergence whilst preserving lateral coverage breadth.

4. The ArcMatrix operationalises this balance structurally: it bounds the interaction space to the defined AIC ontology (preventing unbounded lateral divergence) whilst requiring pairwise exhaustive consideration of all interaction categories within that space (preventing vertical zero-bin neglect).
5. The resulting ATD distribution, long-tailed rather than concentrated or uniform, is the empirical signature of T-thinking in operation: the Architect maintains depth on high-salience interactions (vertical head) whilst systematically populating the low-salience tail (lateral coverage), rather than sacrificing one for the other.

This inference directly motivates the T-thinking characterisation of the AIC systems approach in §3.5.3(c) and §7.2.4 of the thesis.

E.4.7 Where is lateral predictive thinking more effective?

Where is lateral predictive thinking more efficient, valuable, or critical than vertical Predictive thinking?

Generating Comprehensive AI Training Datasets

Lateral predictive thinking is invaluable in creating robust AI training datasets because it allows architects to grasp the complexities of diverse scenarios intuitively. While vertical thinking focuses on refining specific errors within the dataset, it often misses rare or unconventional scenarios (**Black Swan events)**.

- **Example**: Consider training a self-driving car's AI system. Vertical thought processes might address obvious edge cases within the dataset, such as poor lighting or extreme weather. However, lateral thinking introduces counterintuitive scenarios, like a situation where the moon, reflected in a car’s windshield, is misinterpreted as a traffic signal. Vertical thinking may overlook such a possibility because it falls outside of linear assumptions, whereas lateral thinking anticipates unconventional analogies and anomalies.

Discovery of Black Swan Scenarios

Black Swan scenarios—rare, high-impact events—are often counterintuitive and defy the expectations set by traditional models. Vertical Predictive thinking struggles here because it operates within defined boundaries and assumptions, while lateral thinking thrives in ambiguity.

- **Example**: Imagine a drone tasked with monitoring wildlife in a dense forest. A vertical Predictive approach would focus on refining detection algorithms based on common animal behaviours. In contrast, lateral thinking might uncover an unlikely event, such as

a group of animals mimicking a predator's movement to deter threats. This discovery requires a creative leap that vertical methods may not naturally pursue.

Solving Compounded Epistemic Uncertainty Problems

In systems where uncertainties are interdependent and do not follow normal distributions, lateral thinking becomes critical. Vertical thinking is often constrained by its reliance on predefined probabilities and logical consistency, limiting its ability to address complexities that emerge from nonlinear interactions.

- **Example**: A Predictive Problem Solver is tasked with modelling a climate prediction model attempting to forecast weather patterns in the Arctic that might rely on vertical thinking to model known factors like temperature and wind interactions. However, lateral thinking introduces the possibility of unexpected cascading effects, such as minor shifts in ice reflectivity amplifying heat absorption in previously unmodeled ways. This generative approach broadens the architect's predictive scope, capturing possibilities that vertical thinking may miss.

E.4.8 AIC Balanced Predictive Thought Process

Regarding AIC, Appreciation is a method of appreciating a pool of infinite possibilities. Control is a method to increase the probability of implementing finite actions based on an appreciated possibility. A bias towards control tends to have a less influential thought process in terms of discovering Black Swan scenarios. A bias towards appreciation renders less influential thought processes in analysing the vertical turn of events given one possible assumption occurs. We need a holistic thought process that helps balance between appreciation and control to be more effective. The architect's predictive thought process is challenged to think in appreciation and control in order to ensure influence occurs.

Balancing lateral and vertical thinking is critical to designing robust predictive systems approaches that address various scenarios, including Black Swan events for dynamically evolving complexities. The Appreciation-Influence-Control (AIC) framework is uniquely suited to achieving this balance by integrating the exploratory and generative aspects of lateral thinking with the focused, error-resolution-based precision of vertical thinking. Below, we present an argument demonstrating how AIC achieves this balance.

Integration of Divergent and Convergent Thinking:

Lateral thinking emphasises the exploration of possibilities, embracing contradictions, and considering counterintuitive scenarios. Vertical thinking, by contrast, narrows focus, builds on

established assumptions, and resolves contradictions to maintain logical consistency. AIC combines these two modes by:

- **Influence (I):** Functioning as the desired emergent value or property contextualising appreciation and control. Influence captures the interplay between lateral exploration and vertical refinement, ensuring that desired output values are both adaptive (due to appreciation) and practical (due to control action). For example, alerting a human security guard for a potential threat in a train track zone.
- **Control (C):** Imposing structure and refining the desired influence into direct actions, akin to vertical thinking. For instance, controlling the drone orientation concerning train track zone.
- **Appreciation (A):** Encouraging the architect to explore infinite possibilities broadly and generate new categories of thought, akin to lateral thinking. For example, in an autonomous system like a drone, appreciating unexpected environmental factors (e.g., unusual weather conditions or Novel obstacles) expands the system and the architect's understanding of the operational complexity.

Handling Contradictions:

Vertical thinking treats contradictions as errors to be resolved, so naturally, when the architect thinks vertically, their mind may subconsciously be blind to what might be counterintuitive or contradictory. Meanwhile, lateral thinking views contradictions as clues for expanding understanding. AIC addresses contradictions in the following ways:

- **Influence** emerges from balancing the A and C, it is appointed in the middle between soft appreciation and hard control. It is an indirect control, which entails a tangible action that does not directly cause a change because it is the control action that does this. Influence allows the architect to understand the extracted value of control and appreciation and how they feed into the whole through desired influence. With this, the architect can test the validity of A and C. For example, drone perception influences drone track and trace functionality. A potential contradiction at the influence level can be discovered.
- Through **Control**, the architect may harden the vertical rigour and only consider what control actions are required to support influence action. Thus, contradictions are resolved by refining influence and by adding rules to align observed scenarios with system goals. This might involve updating the drone's object detection algorithm to handle edge cases more differently.

- Through **Appreciation**, the architect may soften the vertical rigour. Contradictions are welcomed as opportunities to explore alternative scenarios. In this thought process, the architect drops the vertical thinking hat and opens their mind to welcome the unexpected. For example, if a security drone misidentifies an object on a train track as a threat, appreciation would prompt the architect to consider various analogies or new classifications for the type of objects that may lead the drone to fail, including unlikely ones like a “picture of a clown”.

Mitigating Bias:

Vertical thinking’s reliance on predefined rules and assumptions can accumulate bias, narrow the system’s perspective, and reduce its ability to address Black Swan scenarios. Lateral thinking minimises bias by treating all possibilities equally valid, but risks decision paralysis due to a lack of convergence. AIC mitigates these issues by:

- Allowing **Influence** to dynamically adjust the balance between appreciation and control, enabling the architect to understand why appreciation and control are needed. Thus, it adds a layer of questioning regarding the relevance of A and C.
- Applying **Control** to focus the architect's attention on the most relevant actions, ensuring that the architect remains goal-oriented and objective.
- Using **Appreciation** to explore diverse perspectives and uncover hidden biases in the architect’s assumptions about the control action and desired influence.

Resilience to undesirable surprises:

Real-world complexity fields are inherently dynamic, requiring systems and their architects to be resilient to Novel changes. Vertical thinking struggles with such changes due to its reliance on fixed assumptions, while lateral thinking risks losing focus in the face of overwhelming possibilities. AIC enhances resilience by:

- Encouraging **Appreciation** of the full range of potential future scenarios, including counterintuitive and low-probability events.
- Leveraging **Control** to maintain direct relevance to the present reality, stability, and coherence in the face of current uncertainty.
- Using **Influence** to ensure that the system remains extracting value from A and C, balancing exploration and exploitation as conditions evolve.

Example: Autonomous Drone in a Train Track Zone

An autonomous drone tasked with securing a train track zone provides a practical illustration of how AIC balances lateral and vertical thinking:

- **Appreciation (lateral thinking):** Appreciating what is inside and outside the problem domain. The architect considers a wide range of possible objects on the track, including non-obvious threats (e.g., maintenance equipment, debris, or holographic illusions). This lateral exploration ensures that no potential scenario is dismissed prematurely.
- **Control (sequential/vertical thinking):** Control what is inside the problem domain. The architect applies predefined requirements and rules to categorise objects and take appropriate actions, such as stopping the train or sending alerts. Vertical reasoning ensures that the system remains operationally effective.
- **Influence (Lateral and Vertical thinking):** Consider the indirect impact on other complexes within the problem domain. The architect can meaningfully adapt drone behaviour to a desired influence by integrating appreciation and control. For example, if a drone encounters an object that defies existing classifications, it uses appreciation to generate new hypotheses while using control to prioritise immediate safety actions.

E.5 General Lateral Predictive Thinking Process

The primary motivation for this process stems from the increasing number of bizarre cases where ML systems (in this case, perception) fail to operate reliably in relatively everyday situations. One instance is Tesla cars mistaking the moon's appearance for a red traffic light, halting unexpectedly in the middle of the road[17]. One of the most critical factors associated with ML trustworthiness in safety-critical applications is how the architect can trust its performance during Black Swan scenarios. To predict such scenarios, we need something more than computation thinking; we also need lateral thinking. This section presents a structured framework for applying lateral thinking to predictive systems engineering.

Unlike traditional vertical thinking, lateral thinking emphasises exploring broader possibilities, embraces irrelevance and unrelated interactions, accepts some form of ambiguous reasoning, and uncovers counterintuitive or Novel scenarios. The process outlined here aims to equip system engineers with a systematic approach to generating novel insights, operational scenarios, and expanded knowledge bases. It can be articulated using the STCoT (Systems Engineering Chain of Thought) format or a more generalised process framework, offering flexibility while retaining analytical rigour. In this section, we presented the thought process in a generic format for variation.

E.5.1 Step 1: Broaden base perspective.

Assume a broad perspective without anchoring to existing logical rules of interactions.

The first step involves broadening any initial perspective by deliberately avoiding reliance on fixed common-sense assumptions. This step challenges traditional views by considering even seemingly irrelevant or stationary elements as potential contributors to emergent interactions. You can use general systems hypothesis or philosophical intuitions to guide the broadening of the base perspective to make a prediction. For example, suppose you want to consider the operational environment for autonomous airliners using machine learning (ML) perception at cruise altitude. It is clear that there is not much going on at 40,000 feet in terms of visual complexity (in comparison to ground level); nonetheless, you wish to discover Black Swan events. Then, start by adopting a broader perspective, considering broader possibilities, and intuition (assume the operational domain as a complex of uniformly distributed events where any possible event is a highly probable concern). For example, while all objects may differ, some may appear visually similar to a predictive observer.

Based on such intuition, the architect might ask themselves: What objects could deceive the perception of the airliner and cause it to resemble other objects? The architect may ponder, what if a cloud, under certain lighting conditions, resembles the tip of a mountain? Bogdanov's Ingression Principle inspires the earlier rule: *There are always intermediate Complexes linking multiple Complexes, allowing for understanding through a continuum of similarities[18]*.

- **Thought Step 1.1**: Begin by defining a context without preconceptions. For example, avoid assuming that "a stationary bird standing on a stationary lamp post cannot directly influence a car". Instead, acknowledge that such an object might exert indirect or Novel impacts through environmental, contextual, or systemic relationships.
- **Thought Step 1.2**: Analyse both direct (control) interactions, such as physical impact, and indirect (appreciation or influence) interactions involving environmental elements or other actors in the system. For instance, a lamp post might indirectly influence a car by altering the behaviour of pedestrians or casting shadows that affect the car's sensors.

This step establishes a foundation for exploring the latent complexity of interactions within a system.

E.5.2 Step 2: Generate analogies.

Generate analogies across different contexts.

Building on the broader perspective established in Step 1, this step focuses on exploring alternative patterns of influence by drawing parallels across different domains and considering counterintuitive scenarios.

- **Thought Step 2.1**: Develop analogies from both similar and unrelated domains. For instance, compare the influence of a stationary lamp post on an autonomous car to how static objects in ecosystems (e.g., a tree in a forest) influences the behaviour of dynamic entities (e.g., animals).
- **Thought Step 2.2**: Explore surprising or humorous scenarios to provoke unconventional thinking. For example, use thought-provoking memes or counterfactuals like, "It would be funny or hilarious if [some unexpected interaction] occurred". This approach encourages the exploration of interactions that defy common expectations and reveal hidden dynamics within the system.

This step broadens the scope of analysis, fostering creative and innovative approaches to understanding system behaviour.

E.5.3 Step 3: Formulate predictions.

Formulate hypotheses and generate new classes of operational scenarios (predictions)

This step formalises the observations and analogies from previous steps into actionable hypotheses and operational scenarios, enabling the system to adapt to new patterns of complexity.

- **Thought Step 3.1**: Treat each observation or analogy as an opportunity to hypothesise new operational scenarios. For example, hypothesise how a lamp post might indirectly influence a car's behaviour under varying conditions, such as poor lighting or sensor malfunction.
- By focusing on these hypotheses, the architect can identify emerging classes of interactions that might not have been previously considered, paving the way for more coverage of the infinite possible input acceptable problem through a comprehensive understanding of its operational domain.

This step transforms abstract insights into concrete scenarios that guide system design and testing.

E.5.4 Iterate your thought process!

Iteratively Build a Broader Knowledge Base

Throughout the process, the focus is on expanding the system's understanding by incorporating new observations, interactions, and insights into its knowledge base. This dynamic process ensures that the system remains adaptive and can address evolving complexities.

- Instead of refining or narrowing assumptions, this step encourages continuous reinterpretation and reassociation of observations. For example, link newly observed patterns to prior analogies or hypotheses, even if they initially appear contradictory.
- Integrating diverse perspectives will allow the system to accommodate a growing variety of possibilities, thus enhancing its ability to generalise and adapt.

This iterative process fosters the development of a robust and expansive knowledge base, enabling the system to address increasingly complex and Novel scenarios. We can describe the process using GSN to describe how the predictive thought can be modelled:

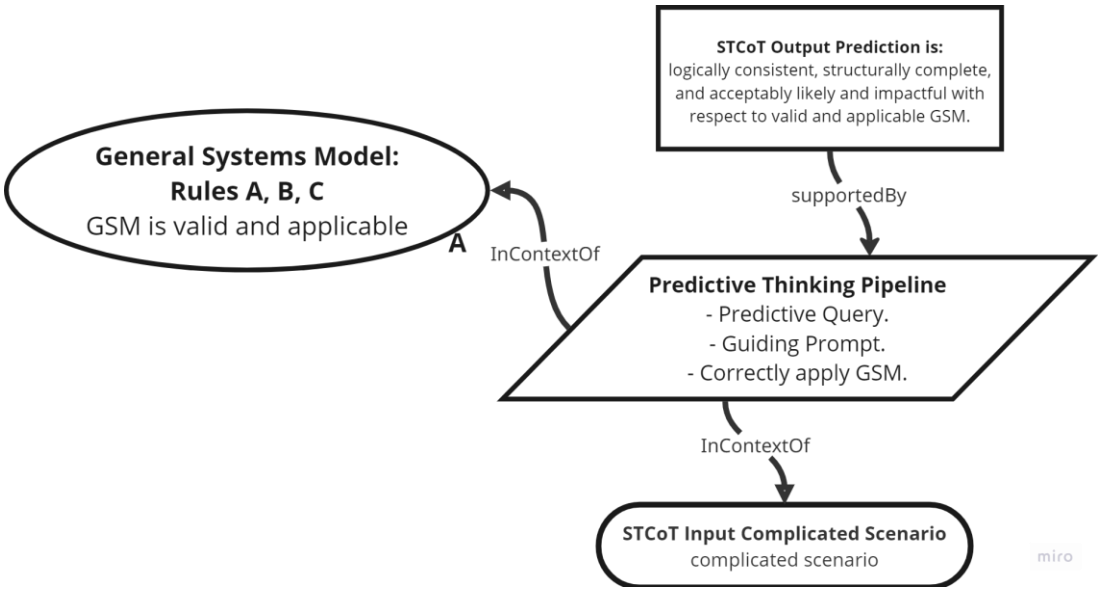

Figure E.4 Abstract Lateral Predictive Thinking model using GSN language!

Figure E.4 describes the lateral thinking process using the GSN format. The thinking steps are modelled as an inference strategy that asserts the predicted scenario to be true. At the same time, the input problem is considered as a context where the thinking attempts to resolve into a set of predictions. The predictive thinking bases its assumptive (axiomatic) perspective on the nature of the problem and potential solution complicatedness based on the context of General Systems Mental Model:

General System Rule A: The nature of problem domain distribution of events is heavy-tailed, which means Black Swan events cannot be ignored. (as observed now).

General System Rule B: The nature of the solution domain is a uniformly distributed complexity field whereby all events are plausible and cannot be ignored (as I will assume it to be).

General System Rule C: All events are probable and reasonably acceptable, and there is no such thing as a contradiction.

The **General Lateral Predictive Thinking Process** provides a structured framework for exploring the latent potential behaviours of complicated complexity fields. By shifting perspectives, generating analogies, formulating hypotheses, and iteratively expanding the knowledge base, system engineers can uncover novel patterns, identify new operational scenarios, and predict efficiently to ever-changing complexity. This approach complements traditional vertical thinking by embracing ambiguity and exploring the unpredictable, ensuring robust and resilient complexity field designs in dynamic and uncertain environments.

E.6 Architect's Predictive Thinking Process (ArcPT)

> ***Predictive Thinking*** *is the process that reduces the complicatedness of the complexity of a complex to update some initial priors, by anticipating Novel scenarios before they actually happen, and demonstrates reduction of uncertainty, ALARP justifiably.*

The output of ArcPT is a cognitive component of the Architect's Predictive Mental Model (ArcPM):

> *A* ***Predictive Mental Model*** *a set of general rules and types about how a general complex behaves, which justifies the prediction of Novel behaviours before they actually occur used to update initial priors.*

It also included a structured representation, governed by the abstract rule set and categories of aspects, which enables an architect to forecast types of information in a complexity field. For example, the AIC approach to complicatedness resolution, STAMP, SHARCS, and FRAM guidewords and categories of actions, failures, and events.

Predictive Mental Models are responsible for the architect's engineering judgment and decisions in the face of aleatoric uncertainty. While aleatoric uncertainty cannot be reduced (as more things are involved), epistemic uncertainty can be reduced through an enhanced Predictive Thinking Process. In the context of designing a mental model, epistemic uncertainty would mean:

The residual doubt about the choice of a mental model design:

- *Will be able to meaningfully classify all possible types of information in any complexity, expected or unexpected.*
- *Will help to make predictions about the complexity accurately.*
- *Will be consistent and verifiable when instantiated.*
- *Will or will never require adjustments over time and experience in dealing with any complexity.*
- *Residual doubt should be reducible over time and variety of experiences with complexity.*

Predictive Systems Thinking is the topic that provides the tools and techniques to enhance an architect's predictive thinking by minimising sources of epistemic ignorance. Epistemic ignorance is an inherent deficiency associated with the architect's limited ability to account for all possible sources of randomness in an open operational environment's complexity field, as perceived by the architect. Therefore.

The architect's attention is inherently biased, leading to epistemic blind spots.

Based on the above principle, we understand that the purpose of the systems approach and systematic design processes is to reduce the architect's perception deficiency. Since epistemic uncertainty is reducible through the acquisition of more knowledge, the architect needs a process for acquiring knowledge and drawing epistemic inferences.

Epistemic inference is an inference from a set of beliefs to new or similar beliefs about an observation that either updates or validates the original set[19]. Shannon defines information as a message from some source to a receiver, and that information can be characterised as the removal of uncertainty for the receiver[20]. However, the receiver who lacks (thus experiencing an epistemic need) the knowledge carried by the information needs to make sense of the message first. Suppose the source of information is an observation of some complexity, and the message processor is the observing architect, who has some epistemic uncertainty about the observed complexity. Based on their existing belief system, the architect's epistemic inference minimises their uncertainty[3]. Through epistemic uncertainty reduction processes, the architect

[3] The core concepts are rooted in epistemology and the formal belief revision theory. We contextualise them in predictive architecting context.

makes a series of inferences, which can be thought of as predictions to be added to the architect's knowledge store (or, dare we say, memory).

If we revise the above description, we realise that making a prediction truly starts from some level of ignorance (lack of knowledge) arising from a shift in perspective that may be induced by a new experience or evoked by a relative change. This means a sense of "epistemic need" may be necessary to initiate the architect's predictive thinking process. The more varied the information that must be processed by the architect, the greater the lack of knowledge, the stronger the urge to minimise it, which leads to the emergence of "epistemic need" in the architect's predictive thinking process.

We define "architect prediction" as the architect's epistemic inference driven by epistemic processing of information received about some complexity, underpinned by some set of beliefs or axioms. We see this epistemic inference processing of information as the Architect's Predictive Thinking Processes (ArcPT), which underpins the architect's predictive capability, a general problem-solving function of the architect.

Perhaps paradoxically, to store knowledge, which serves as a remedy for insufficient knowledge, the information conveyed about the situation's complexity must evoke a heightened sense of ignorance in the architect. The output of such a predicament process is either a re-confirmation (re-validation) of what is already known or an eureka (new knowledge realised). Among the nine categories of systems engineering problems in Table E.1, Stochastic Confusion provides the least information to the architect, resulting in the highest degree of epistemic need. In other words.

> *The more the architect realises their ignorance, the more knowledge can be sought after and thus potentially gained. Consequently, the stochasticity of the problem will eventually become less confusing for the architect, and they will become more certain (and therefore confident) in their design decisions, despite an exponential increase in uncertainty.*

Given the evident influence relationship between the stochastic nature of a problem and the level of epistemic uncertainty that the architect possesses concerning that problem, it follows that to address a Stochastic Confusion issue effectively[4] It would be more efficient to approach the problem of uncertainty, which compounds very fast, not as a standalone issue - an

[4] Such as the development of a safe and autonomous system comprised of non-deterministic components within an open, non-deterministic environment.

"observer-reasoner independent"- but rather as an "observer-reasoner dependent problem," where the architect takes a predictive observing problem solver role.

Therefore, a solution approach for the Stochastic Confusion problem would focus on carefully and rigorously challenging the architect's situation of knowledge from multiple perspectives and making them question what they learned at different stages of the solution approach. This way, the systems approach would evoke the architect's predictive capability to look at the problem domain from varying perspectives of world views. In doing so, the architect starts to appreciate yet-to-be-thought-of unknowns (Black Swans) that may lurk under the hood of the heavy tail of uncertainty. Consequently, the systems approach only needs to focus on reducing the architect's epistemic confusion (ignorance or need) through more comprehensive predictive thinking processes. Hence, we operated in this PhD on the perspective that the CUPE is "an architect's predictive capability problem".

To make a prediction[5], considering the Bayesian perspective, the architect needs to have the following activities present:

1. **Prior Belief System and Method of Inference:** The architect must have a prior set of beliefs in two tiers of abstractions:
    - **Abstract general beliefs:** They apply to all possible complexities. For example, all things are systems and thus have parts.
    - **Concrete beliefs:** these instantiations of applying the general core beliefs in the context of a specific complexity field. For example, a drone is a thing, thus a system. Therefore, it has parts. Which provokes a follow-up question: What are those parts?

Evidence
When we applied HAZOP, STAMP, and FRAM, our predictive thinking process was constrained by the fundamental definition of what a clear system is, which they indirectly or directly instil. FRAM assumes that systems are sub-organisations of control functions within larger organisations of systems, and some control interactions resonate with others, leading to an emergent amplification of impact. HAZOP assumes failures occur in systems, such as a

[5] Our elaboration is fundamentally driven by modern interpretations of predictive processing in cognitive science, which posit that the brain continuously generates and updates a model of its environment to minimise prediction error[21] A. Bubic, D. Yves von Cramon, and R. I. Schubotz, "Prediction, cognition and the brain," *Front. Hum. Neurosci.*, vol. 4, no. https://doi.org/10.3389/fnhum.2010.00025, 2010.. In these models, encountering new information or surprising events induces a sense of uncertainty that then drives mental-model updating.

traceable analogue control system, in which only a subset of deviations, subject to the observer's expertise, can occur. STAMP assumes complexity as a first-order cybernetics problem. A problem of a hierarchical control feedback loop. Those prior beliefs about the general rules of complexity directly impact our overall predictive performance.
Another piece of evidence we learned from our experiments is that attempting to generate a reliable dataset strategy can enable ML models to perform as expected in cases of relatively surprising data shifts (away from the distribution of training datasets) in the operational domain (Black Swan). The nature of the distribution of the training and validation datasets (whether typical operations or a mix of random images and Black Swan scenarios) really dictated the model's reliability. The nature of the dataset distribution can be understood as our engineered priors; an ML model ought to reflect on the real-world operational complexity. If we supplied the model with pictures of a drone purely outside the operational domain, it performed poorly on Black Swan datasets (in-context Black Swan). See section H.10. This demonstrates that the nature of the dataset context dictates the reliability of the ML model's predictive capability in the face of problem complexity.

2. **Realisation of Ignorance:** A sense of lack of knowledge (epistemic need) induced due to the uncertainty of disbelief in the validity of some prior belief system in experienced complexity[6].
    - Such a situation may be triggered due to a Novel change in the circumstances around the problem. For example, the drone incorrectly detected a Target Object of Interest (TOI). If the incorrect detection is a thing, (Which provokes a follow-up question), then what are its parts? However, detection of TOIs is intangible; (provoking another question) how can it have tangible parts?
    - or, evoked by another architect or system approach that makes the architect realise some potential inconsistency in their existing belief systems about the problem. For example, a systems approach step prompts the architect to consider some empty space, encouraging the architect to question the original set of belief systems; a space is also a thing. How does it have parts?

Evidence
When we applied HAZOP, STAMP, and FRAM, our predictive thinking process was prompted by constantly asking ourselves questions about how we would interpret the deviation guideword

[6] This is why we understood Bogdanov perspective about systems being an "organisational experience" between the observer and the observation.

in an interaction context. This delta in epistemic lack helped us predict different failure scenarios between one guideline and another. The realisation of lack of knowledge (ignorance) in applying abstract general rules (guidewords) was a triggering factor in predicting a failure scenario.

3. **Making a prediction (epistemic inference):** The first two activities constituted a chain of conditions we experienced while implementing the processes. The realisation of ignorance made us wonder how a general rule, like a guide word, may manifest in the real world. This predictive thought process led us to recognise a lack of knowledge, or ignorance. Subsequently, a series of rationally interrelated questions emerged that provoked some inferences. At the end of this thought process, we arrived at a conclusion, which is the prediction.

Evidence
When we applied HAZOP, STAMP, and FRAM we were unaware of the failure events we discovered later after applying the processes. We predicted more ways the problem complexity may scale up and increase in complicatedness. Our epistemic ignorance was reduced every time we predicted new interaction.

If we combine the generic predictive thought process with concepts of aleatoric uncertainty, we realise the following assumptions:

- The solution of aleatoric uncertainty (randomness in the observed complexity) is determinism of the processes among complexes, such that emergence is predictable and verifiable (independent of the observer). For a CUPE, this is a very hard ask.
- The solution of epistemic uncertainty (unpredictability of the observed complexity) is the ability to be certain in predictions made about involved components and the observed processes' output based on an epistemic mental model of the observed complexity, which is underpinned by general belief systems about how complexity in general behaves.

Therefore, Prediction or inference, in general, minimises architect epistemic uncertainty and prompts the reason to continue the cycle of predictive thinking.

Hence, we believe that Architect's Predictive Thinking Processes (ArcPT) provide a structured approach to refining a Predictive Problem Solver's capability, enabling them to navigate and minimise Epistemic Uncertainty effectively. By combining systematic and systemic thinking approaches, ArcPT aims to enhance the architect's capacity to predict complicated behaviours and uncover Novel interactions. We realise that for a predictive thought to iterate or be powered

to carry on generating predictions, there needs to be the following key activities to challenge the architect's situation of knowledge continuously:

- Set the general belief system: General, Concrete or both.
    - We opted to use general systems theories as the basic belief system for the architect to base their prediction-making process on.
    - We also used AIC (Appreciation, Influence and Control) to be part of the belief system behind the reasoning for prediction.
- Evoked a sense of ignorance (lack of knowledge) through questions.
    - We opted for a predictive question method based on the set belief. we named such methods as Predictive Questions.
- Remedy the sense of lack of knowledge using a guiding prompt.
    - We opted to use a prompt like general guidance to guide the architect's predictive thinking process towards making an informed and informing prediction.
- Strategically shift the basis of a belief system that was initially set, to stretch their imagination and consider more Black Swans.
    - We opted for a perspective-shift process to deliver the core value of lateral thinking.

This section introduces the Predictive Systems Theoretic Chain-of-Thought (STCoT), a process inspired by chain-of-thought prompting techniques used in large language models. STCoT represents a comprehensive, step-by-step framework that guides architects in systematically applying predictive thinking grounded in General Systems Theory (GST) and systems thinking frameworks. Its ultimate purpose is to improve the architect's ability to balance lateral and vertical thinking, ensuring a rigorous, transparent, and creative problem-solving process.

E.6.1 Predictive Systems Theoretic Chain-of-Thought (STCoT)

Earlier, we examined a systematic process for predictive thinking. We realised that each step builds on the step before it in a chain reaction triggered by a sense of ignorance. This chain reaction of continuous inquiry into the nature of observation (executed by the architect's predictive mind) is analogous to the idea of "Chain-of-thought" (CoT) associated with large language models. This time, we refer to it as predictive architect chain-of-thought. The nature of a predictive CoT really can be determined by two main aspects:

1. Primary indicator: The nature of the belief system (and how correct they are).
2. Secondary indicator: The nature of the reasoning method used to correctly validate the set of beliefs in a context of observed complexity to infer concrete beliefs.

and?

Suppose the set of beliefs is general systems theories, and the reasoning philosophy is based on a systems approach (like AIC, for example). In that case, the chain of thought is classified as a predictive system thinking process. When those Predictive CoTs (or let us call them PCoTs) are used in the context of Systems engineering, they become Systems Theoretic Chain-of-Thought (STCoT). If PCoTs are based on any other belief system and use any reasoning, the PCoTs become predictive thinking of the general topic that belief and reasoning systems are part of. For example, suppose the belief system is related to the mechanical aspect of physical systems. In that case, the PCoT can be classed as a predictive mechanical engineering chain-of-thought. Suppose the set of beliefs on some ethical general rules, and the output is an engineering judgment that leads to a design consideration for an autonomous system. The PCoT can be classed as a predictive ethical engineering chain of thought. For example, the latter may help ensure ethical consideration in designing autonomous systems.

So What?

Therefore, STCoT is a process that involves a systematic and rationally related chain of predictive systems thinking steps (systems thoughts) based on systems thinking frameworks or GSTs. Implementing a STCoT aims to transition ArcPT objectively from a state of confusion to one of clarity. Effectively shifting the view of complexity from being a harder-to-predict-and-ascertain Stochastic Confusion to the situation of an easier-to-predict-and-ascertain system to manifest the PrimeP. In doing so, STCoT aims to indirectly enhance ArcPT's problem-solving capability, enabling it to predict or make assertions that can be explained and qualified.

By doing so, we can probe the design process deeply in case mistakes occur during the deployment of intelligent behaviours by understanding exactly how the architect predicts certain behaviours or overlooks considerations of some behaviours. STCoT provides evidence to support claims of justification for design decisions for safety cases. For example, part of the SACE safety case framework is an artefact called Sn5.1[AS design justification], which means an argument that justifies key design decisions taken at any stage in the design process (see Appendix L, section L.4.5.2). This approach further provides evidence of the claim of alignment to human values and allows for the implementation of human values into developing system requirements.

The cognitive process that STCoT performs on Architect Predictive Thinking is what we refer to as:

T-Thinking-based Prediction-Provocative Prompting

In this process, STCoT encourages the architect to think about general aspects of what they are trying to evaluate or create. The "Lateral" part of T-thinking is experienced by considering a general rule learned from various universal systems when making an inference. This is

accomplished through a thought-provoking question and a guiding prompt on how to answer the question. The expected output of STCoT is an architect prediction or assertion in the following format: “The architect asserts that …”. For example, based on implementing AIC Complicated systems principles, we can derive the following STCoT:

Table E.9 Example lateral rule and predictive question

General Systems Axiom (Lateral Rule)	STCoT thinking step
AIC systems principle: Primary Purpose No observed set of coexisting complexes can be described as a “system” without the servitude of at least one shared Primary Purpose (PrimeP) to manifest. A set of coexisting elements that do not satisfy this principle is considered a “Stochastic Confusion”	**Predictive Question:** Given the observed phenomenon, given principle 1, What is the complex of coexisting complexes of concern and their PrimeP? **Guiding Prompt:** Identify a list of complexes of concern that share a common PrimeP and define each complex's primary purpose. If you find complexes that can be grouped to serve a clear PrimeP, identify the encompassing whole, not each element. For example, a tracking zone, a train, a train driver, and train passengers can all be grouped under one supra-system, which can be termed a “train network”.

Table E.9 presents an example of how a General Systems Model (Lateral rule) is systematically translated into a thinking step to guide the architect in making structured predictions about a given observation. It applies a key AIC systems principle, the Requisite Primary Purpose. In situations where a set of coexisting elements can only be considered a system if they serve a shared Primary Purpose (PrimeP). If no such shared PrimeP exists, the observed elements are merely a "Stochastic Confusion", not a system.

The Predictive Question is derived from the central concept of the general principle and re-wording it to evoke the architect's perception to notice some behaviours and define their primary purposes. The predictive question intends to ensure that the architect does not describe elements in isolation but rather identifies whether these elements function together toward a shared objective (PrimeP) or are a disjointed, Stochastic Confusion. The purpose of this question is to prompt a structured analysis where the architect must evaluate:

- What elements are interacting in the observed problem domain?

- Are they serving a unifying purpose that justifies treating them as a system?
- If no common PrimeP is found, then what missing factors might be causing the confusion?

The Guiding Prompt further refines the prediction process by directing the architect toward a practical lateral rule application process.

E.6.2 Predictive STCoT Structure

We define a particular structure of fully elaborated STCoT, which comprises the following elements captured in a table:

- **STCoT:** Define a particular descriptive title that represents the inference goal.
- **STCoT Input:** Define the input information needed to implement the CoT. A description of some inquired situation (an interaction or a situation).
- **GSM:** A set of general systems rules about any complexity dynamics. In other words, it is a general rule that makes any complexity behave like a desirable system. The main assumption here is that the GSM is valid and applicable. The output of the thinking approach must preserve those rules. They help define what acceptable output looks like and does not. In this Section, various types of axioms can be used to guide the thinking approach of the designer:
    - Hard systems axioms, such as those related to physical systems dynamics.
    - Soft systems axioms, such as those related to soft concepts like ethical or human value consideration.
- **ArcPT Approach:**
    - **Predictive Query:** Prediction-provoking question derived from GSM.
    - **Guiding Prompt:** Prediction-Guiding Prompt that directly instantiates GSM in a given scenario.
    - **Step completion criteria:** describe how the step is judged to be completed.
- **STCoT Output Prediction:** The series of predictions made across the steps from all thinking pipelines that define new anticipated situations or situations. The assertion statement claims: “the architect asserts that the output predictions are logically consistent, structurally complete, and they are acceptably likely and impactful, with respect to the validity and applicability of GSM”.

A Thinking Step: is a thought-provocative prompt that influences the Architect’s predictive thinking approaches (specifically the architect’s predictive attention distribution approach through altering or refining the meaning of the internal ontology used) to make an informed prediction based on some general system perspective. It intends to focus (influence) the Architect’s attention to synthesising a potentially complicated interaction from some inquired

context by applying general rules about systems. It contains the following thinking activities:

A Predictive Thinking Pipeline: A conceptually interrelated series of STCoTs and other thinking steps. A Predictive Thinking Pipeline title represents the primary prediction goal, expected to be asserted by the interrelated constituent thinking steps.

A Predictive Thinking Pipeline prediction: Set of all inferences.

Impactful

The above construct is presented in table format. For example:

Table E.10 General predictive thinking process example

Process title	AIC perspective shift STCoT.
Input	Security drones inhibit human intrusion into train track zone.
Predictive Thinking Pipelines	**Predictive Thinking Pipeline 1:** Perspective Shift by Altering Interaction's AIC Type. **STCoT_7:** **GSM:** AIC perspective shift results into atypical novel black swan scenarios. **ArcPT:** **Thinking step 1:** identify AIC interaction of concern. **Thinking step 2:** Alternate the goal's AIC type. **Thinking step 3:** describe a scenario that captures the change in complicatedness. **Predictive Thinking Pipeline 2:** Perspective Shift by reversing effect direction. **Predictive Thinking Pipeline 3:** Perspective Shift by alternating effect type. **Predictive Thinking Pipeline 4:** Perspective Shift by alternating multiple perspective shift types.
Predictive Thinking Pipelines Output Prediction	Four different Novel scenarios for every given system phenomenon.

Table E.11 presents an example of a STCoT (Systems Engineering Chain of Thought) titled "AIC Perspective Shift STCoT", which illustrates its purpose, input, process, and output. The primary purpose of this STCoT is to assist in predicting unexpected and Novel emergent scenarios resulting from anticipated systems interactions. The input focuses on security drones inhibiting human intrusion into a train track zone. The process is structured into a predictive thinking pipeline with multiple steps designed to shift perspectives by altering AIC interaction types,

reversing effect directions, or alternating effect types. Each step prompts the architect to think about interactions in new ways, ultimately leading to STCoT's output prediction: four different Novel scenarios for every system phenomenon.

An example of a thinking step structure is that we will use Thinking Step 2: Alternate the goal's AIC type.

Table E.11 Predictive Thinking step example (a variation of the general STCoT)

STCoT Title	AIC perspective shift STCoT
Input	Some AIC interactions, for example, Eagle Drone denies local people's intrusion into the train track zone. Eagle Drone has a control goal over local people's intrusion.
General Systems Mental Model	**Axiom A:** In any AIC relationship, a new complexity may emerge from the interactions of coexisting elements; when an element changes its AIC goal, the direction of impact is reversed.
Thinking Process	**Predictive Question:** What would happen if the interaction flow and effect type remained the same, but the goal's AIC type and action type were altered in the future? **Guiding Prompt:** Review the AIC dynamics of the observed systems and alter the nature of the goal and action. Then, define an appropriate action to bear the new AIC types and describe a scenario in the shifted context. **Step completion criteria:** the step is completed when a scenario is graphically modelled and the scenario is written.
Output prediction	**Architect assertion:** This is the output prediction of the thinking step. The assertion is to be written in the following format: "The architect asserts that". For example, the architect asserts that there may be a potential Novel emergent situation where the drone appears to attract unwanted intrusion by local people into the train track zone. In this case, the original perspective of the Eagle Drone being in control of local people's access has shifted to be appreciated.

Table E.8 presents an example of a thinking step structure focused on "Thinking Step 2: Alternate the goal's AIC type". The input involves a specific AIC interaction, such as an Eagle Drone designed to deny local people's intrusion into a train track zone, with a control goal over this access. The thinking process employs a general systems rule (Axiom A) that predicts new complexities emerging when an element's AIC goal changes, reversing the direction of impact.

The predictive question asks what would occur if the interaction flow and effect type remain constant, but the goal's AIC type and action are altered in the future.

A guiding prompt encourages the architect to review the AIC dynamics, alter the goal's nature, and define a new scenario under the shifted context. The step is completed when the scenario is graphically modelled and elaborated in detail. The step's output takes the form of an architect's assertion, such as predicting a potential Novel situation where the drone unintentionally attracts local people to the train track zone, shifting the drone's original role from control to appreciation. This structured thinking approach highlights how altering goals can reveal emergent scenarios.

E.6.3 Engineered datasets' epistemic risks

An interesting observation made by Joseph Nelson, Roboflow cofounder and CEO, about the approach of most ML perception developers, saying[22]:

> "We seek to build computer vision models that generalise to as many real-world situations as we can, even when we cannot anticipate them. It is a bit of a catch-22: build deep learning models that predict a world that you may not have anticipated. But that is what is at the crux of preventing overfitting. Unintentionally, we build models that best predict our training data, but not the real world (or the test set)".

Nelson describes the general attitude of ML software engineering in practice when it comes to training safe and reliable models. He emphasises key factors or aspects that impact trust in any ML model training and dataset gathering processes:

- The real challenge is that ML models are expected to generalise well in scenarios that the developer cannot anticipate (not just hard for them to predict).
- Predicting a real-world complexity that the developer may not have previously predicted requires them to consider real-world scenarios, not just clever techniques to circumvent the burden of thinking harder and deeper to predict more possible scenarios.
- The prevention of overfitting due to hidden biases in the dataset, due to a skewed or biased dataset gathering and development process (which is the task of the developer).
- ML developers can be prone to unknowingly or unintentionally having a mindset to focus their model training and validation around predicting the training data, but not the real-world.

These challenges all stem from a common root:

ML Developers have not fully engaged in a balanced, objective, and comprehensive thinking process that meaningfully reduces their epistemic uncertainty about real-world conditions.

Which in turn leads to gaps in the dataset's epistemic coverage, what we can call:

Engineered dataset epistemic risk.

Because developers often assume that their training data implicitly captures the diversity of real-world scenarios, they fail to ask, "What don't we know about the environments our model will face?" As a result, they collect and curate data that reflects their expectations and biases rather than the true breadth of possible situations, which hides risks of overfitting to familiar patterns. When hidden biases remain unmitigated during dataset curation, models appear to perform well on held-out splits but falter in unanticipated conditions.

In essence, skipping the deeper cognitive work of identifying and filling knowledge gaps about the world means datasets lack critical examples (epistemic coverage is incomplete), and the model cannot be trusted sufficiently to generalise in Novel scenarios. This engineered dataset epistemic risk manifests whenever developers over-trust or over-rely on clever tricks to patch errors, instead of systematically expanding their understanding of what the model might encounter outside the lab, through augmentation alone. In a safety-critical application, a safety quality regulator should care more about "has the developer deeply thought about the real-world operational domain" rather than "has the developer gathered sufficiently-sized datasets and applied clever automated augmentation techniques".

In other words, a regulator should care more about "has the developer's predictive thinking process enabled them to predict those hard-to-predict scenarios?"

The ML architect engages in epistemic inference through the Architect's Predictive Thinking Process (ArcPT), a reasoning mechanism that updates prior beliefs about any complexity in response to new information observed or discovered. However, when the information about an observed complexity (such as a dynamic environment for a vision-based AI system) is sparse, ambiguous, or inherently stochastic, the architect's epistemic need (a drive to reduce uncertainty) becomes more noticeable. As described in ArcPT, this need initiates a predictive reasoning chain, prompting the architect to ask: "What do I not know about this complexity?" The deeper this uncertainty, the greater the potential for unrecognised unknowns (Black Swan events) to emerge.

This cognitive gap reflects directly on how datasets are constructed; when architects and dataset curators fail to recognise or fully appreciate certain operational complexities (whether due to unfamiliarity with Black Swans, novel conditions, or failure scenarios), the resulting data will be **epistemically ignorant**[7] of those scenarios, which risks truing the completeness of datasets.

E.6.3.1 Epistemic Ignorance in Image-Based Datasets

This epistemic ignorance manifests tangibly in image-based datasets; in our case, we are concerned about training AI perception systems (e.g., for object detection, navigation, or collision avoidance). A dataset that does not give the architect sufficient confidence that it captures a variety of operational states, environments, lighting conditions, object variations, or anomalous behaviours introduces a silent gap: the AI system becomes blind to what it has never seen, and unexpected behaviours may blindside the architect.

> If software has “bugs” that can lead to harm, a formal methods approach reduces it. If a physical system has “flaws” or “faults” that can lead to harm, formal systems engineering approaches reduce them. Similarly, a dataset of images has “Epistemic Ignorance” which means the absence of valuable information that prevents harm[8], can be reduced by reducing the architect's epistemic ignorance using the AIC systems approach, traceability of images to requirements and the systematic prediction and mitigation of Black Swan pictorial scenarios.

Pictorial-driven hazards emerge from the *absence or biased presence* of critical visual information in the training data. Pictorial hazards are "soft hazards"[9] that pose indirect risks by denying the AI model the opportunity to learn from, detect, or anticipate certain operational states. For example, if the training dataset for a collision avoidance drone omits images of hot-air balloons at low altitudes, the resulting model will not be prepared to react appropriately in that scenario (despite its critical safety implications).

---

[7] Epistemic Ignorance is a well-established concept in information theory (see section 2.1.15)
[8] As in harm-preventing information is absent.
[9] See section E.3 for definition of soft-hazards.

E.6.3.2 Linking Architect Confusion to Dataset Design Failures

The origin of these pictorial hazards lies in the architect's peripheral perception blind spots; a failure to foresee the whole landscape of possible environmental complexities (seeing the wood for the trees). This is compounded when:

- The dataset is designed from a static or biased worldview of problem domain complexity about typical operation scenarios,
- The architect relies on historic familiarity and incomplete concrete belief systems, and
- The epistemic need[10] to explore "unknown unknowns" is not actively evoked.

This misalignment leads to an unassured system: one that may perform well under common conditions but fails unpredictably in rare or complex situations (those most critical to safe autonomous operation).

E.6.3.3 Towards Mitigation: CuneiForm-based Epistemic Intelligence Development

In general, Intelligence is the reduction of ignorance or confusion.

To mitigate epistemic pictorial ignorance, the CuneiForm development process enhances the architect's knowledge by systematically addressing their epistemic needs, challenging their assumptions, and broadening their worldview by seeing the operational environment from an intelligent machine's perspective. This process involves iterative predictive questioning, perspective shifts, and revision of the belief system to uncover invisible complexities in the operational domain. The aim is to ensure that the dataset design is not only comprehensive but also epistemically sound, reflecting not only what is known but also what may be unknown.

By systematically bridging the gap between architect epistemic uncertainty and dataset epistemic coverage, we can better anticipate pictorial hazards, because we transform the problem into an architect's predictive thinking problem. In doing so, we shift from reactive data collection to proactive dataset engineering, ensuring datasets serve as instruments for knowledge discovery.

[10] A need to acquire new knowledge

E.7 The CuneiForm characteristics

To close the semantic gap between descriptive and pictorial representations of TOIs and behaviours, we define two main activities:

> Meaningful mapping of real-world TOIs and their environment's behaviours to pictorial models.

Meaningful mapping ensures that relevant elements of the textual description correlate with specific visual features or patterns in a CuneiForm, establishing a shared understanding between the CuneiForm creator and the observer/quality checker/validator. The mapping is based on mitigating potential perception failures due to its TOIs' pictorial behaviours. We define this mapping by identifying and mitigating the failure of a training dataset to enhance the perception model's robustness in recognising required TOIs in all possible scenarios.

E.7.1 Valuable Minimum Variety condition

Table E.9 lists potential absence-of-harm-prevention-information (equivalent to failure) modes in developing training datasets, along with the corresponding mitigations in the CuneiForm design process. Note that we specified the "valuable minimum variety" condition as a mitigation for "insufficient variety," as the term "sufficient" cannot be measured. In contrast, the term "valuable" can be qualified by demonstrating that it meets expectations, in the context of "the added images contribute towards minimising epistemic uncertainty through meeting requirements," which makes them "valuable". In other words, a valuable image is:

- An image whose pictorial behaviour can be quickly and unambiguously traced to a descriptive parent requirement.
- An image that contributes towards minimising the epistemic uncertainty of a given class or feature related to the recognisability of TOI.

A valuable image adds value by:

- Assisting with the trustability of a claim made in the safety case.
- Contributing towards enhancing the computer vision model knowledge system.

A low-value image is an image that may add value to enhancing perception model capability, but:

- It is hard to understand how valuable it is in terms of meeting expectations.

- Also, it is hard to understand how it contributes towards minimising epistemic uncertainty since there is no specific class it belongs to.

For instance, consider an image of a dog swimming in a river, which is part of a dataset used for classifying dogs in a task. Suppose this image was not included in an explicit requirement. While such an image might be intended to enhance robustness, it ultimately lacks value in explaining how such an image is needed. What if adding such an image may reduce the robustness of dog recognition in a non-river-based scenario?

One way to think about the design assurance inefficiency introduced by low-value images is that they may cause more cognitive overload for human validators. Imagine a dataset containing thousands of low-value images used for safety-critical functionality; how can a regulator trust that safety requirements have been faithfully captured? Or even how can the regulator or design assurance authority trust that Black Swan scenarios have been extensively predicted and designed?

We also used "minimum" since variety alone is an immeasurable quality. However, with descriptors like "valuable minimum," we are tacitly saying this is a minimum variety, and there can be more, but this is as far as we can get for now. It can demonstrate how our process satisfies this quality and allows us to dictate a limited set of generic characteristic types that can be applied to any TOI and can be qualified against meeting expectations:

Perception training dataset development may fail to minimise AI epistemic uncertainty due to the following sources of pictorial aleatoric uncertainties:

Table E.12 Pictorial Hazards Impact on Perception Reliability. The mitigation plans provide the characteristics that must be depicted in a CuneiForm.

Pictorial Hazard	**Mitigation:** CuneiForm characterisation must help **the** perception training architect to consider ...
Insufficient variety of TOI's visual aesthetic complexity, leading to the absence of information that could prevent harm.	A valuable minimum variety of TOIs' aesthetic features across perceived images.
Insufficient variety of TOI's visual complexity during motion and in dynamic states, leading to the absence of information that could prevent harm.	A valuable minimum variety of TOI's motion situations and optical effects across perceived images.

Insufficient variety of environmental objects, leading to the absence of information that could prevent harm.	A valuable minimum variety of pictorial environment objects across.
Insufficient variety of environmental objects' motion, leading to the absence of information that could prevent harm.	A valuable minimum variety of Pictorial Environment Objects, Motion, and Dynamic optical situations across perceived images.
Insufficient variety of TOIs' change of positions in the perception view, leading to the absence of information that could prevent harm.	A valuable minimum variety of TOIs' positioning across perceived images.
Insufficient variety of TOIs' change of 3D orientations in the perception view, leading to the absence of information that could prevent harm.	A valuable minimum variety of spatial orientations of TOIs in 3D space.
Insufficient variety of changes in TOIs' distance from the perception view, leading to the absence of information that could prevent harm.	A valuable minimum variety of Changes in the distance cause relative changes in TOI's image size concerning total frame size.

One important aspect we realised while developing the process was that the greater the variety of classes introduced for defining a characteristic for a single CuneiForm, the less valuable it becomes in terms of cognitive ease when validating or designing CuneiForms. Suppose a CuneiForm image contains too many variations of characteristic classes in a single characterisation. In that case, matching pictures to the parent abstract Cuneiform becomes inefficient during a quality check. This can result in mistakes and defeats the purpose of using Cuneiform. Cognitive overload during the design and validation process, which ML engineers typically perform, is a risk as it may lead to human errors. Therefore, in the process, we introduce some heuristic numbers to constrain the complexity of the designed variety in every CuneiForm.

E.7.2 Machine Training topics

Based on the analysis in Table E.9, we define generic dimensions (analogous to topics about the world that we aim to train the machine to recognise) that can be captured pictorially in the CuneiForm image. The primary conceptual rationale for representing each dimension is the assumption that aleatoric uncertainty is a source of confusion for architects and machines,

meaning that randomness in the operational domain is visually captured in the dataset, thereby introducing randomness into the pictorial dataset. This randomness, known as aleatoric uncertainty, can confound the architect, resulting in the loss of potentially useful knowledge that should be incorporated into the dataset used to train the machine for a specific task. Thus, the randomness in the pictorial dataset may influence the epistemic uncertainty of the architect and the machine. Each dimension aims to reduce the epistemic uncertainty of both the architect and the machine.

E.7.3 Topic 1: Pictorial Horizon topics

One source of pictorial hazards comes from the infinite acceptable input space of Digitally Perceived Pictorial Horizon Attitude.

In the context of a camera system, the Pictorial Horizon refers to the apparent boundary where the sky meets the Earth or sea, as seen through the camera lens. The inclination and attitude of the horizon are influenced by the camera's position, 3D attitude, and environmental factors, including atmospheric conditions. Figure E.5 illustrates the various horizons relevant to a camera system. There are two types of horizons:

- The astronomical horizon is the theoretical line where the Earth and sky would meet if there were no obstructions, considering the Earth's curvature.
- The true horizon is the theoretical at sea level, unaffected by terrain or other local obstructions.
- The Pictorial Horizon is the pictorial line that separates the earth and sky impacted by objects.

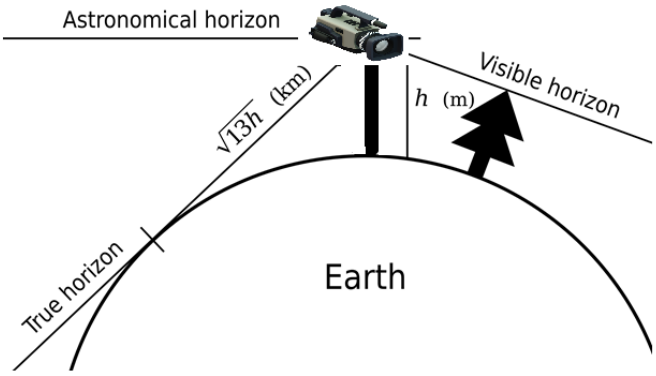

Figure E.5 Types of horizons for cameras taken from

Understanding the orientation of the horizon within an image is critical for accurate perception and decision-making. The horizon attitude in an image can be manipulated through the camera's Frame Roll and Frame pitch. To standardise this for any set of pictures in a computer

vision training set, we could create a specification that dictates the Frame Roll and Frame pitch settings for different horizon attitudes. The justification for defining the horizon in a pictorial sense stem from the fact that the sky and earth have two distinct visual complexities.

E.7.3.1 Specifying Perceived Pictorial Horizon Attitude

To objectively define a process to specify the horizon from an image or a video frame, we need to establish the following:

- A universal pictorial horizon for any digital image. Let us call it an “absolute pictorial horizon reference”.
- We need a measurement tool to measure the relative horizon in the image or video frame itself relative to the absolute pictorial and digital horizon reference.

We define a general horizon reference as the horizontal line that splits any picture into exact halves. The actual horizon in the image or video frame itself can then be compared against our standard horizon to determine the Perceived visible frame horizon metric using the aircraft attitude indicator instrument metric to specify the variety of frame horizon attitudes. The adapted instrument is a Pictorial Horizon Attitude Indicator (PHI).

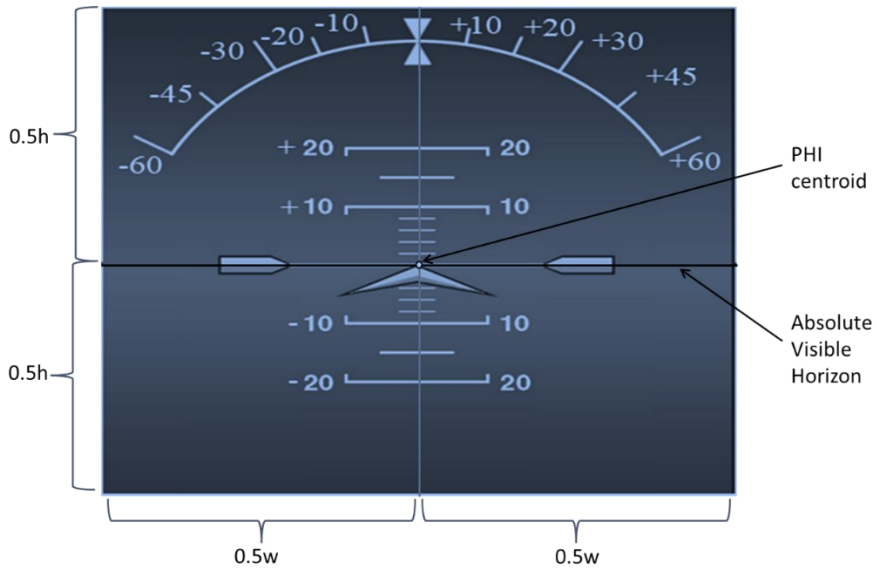

Figure E.6 Pictorial Horizon Attitude Indicator (PHI)

Figure E.6 illustrates the Pictorial Horizon Attitude Indicator (PHI), inspired by flight attitude indicator metrics, designed to manually and objectively define and measure horizon attitudes in images or video frames. The absolute Pictorial Horizon is a horizontal line splitting the image into two halves, ensuring a universal reference point for horizon measurements. The PHI centroid acts as the focal point for measuring relative horizon attitudes, enabling precise alignment and quantification of frame roll and pitch. This tool provides a structured framework

for specifying and analysing the perceived Pictorial Horizon, aiding in consistently characterising horizon-related attitudes across digital images and training datasets.

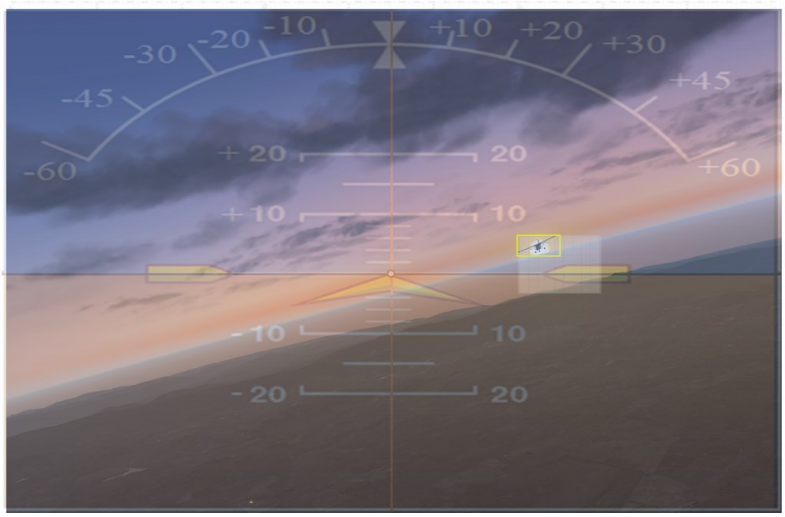

Figure E.7 Perceived visible frame horizon attitude metric example. PHI is superimposed over the image to validate the horizon attitude.

In Figure E.7, We placed the matrices to match the frame size and made the metric transparent.

E.7.3.2 Measuring the Roll and Pitch of the frame horizon using PHI

We employed a manual measurement process using visual tools like a ruler to validate or define the roll and pitch angles of the horizon in images. This process involves aligning the PHI with the horizon in the image and measuring the inclination. Below are the steps for measuring both the roll and pitch of the horizon:

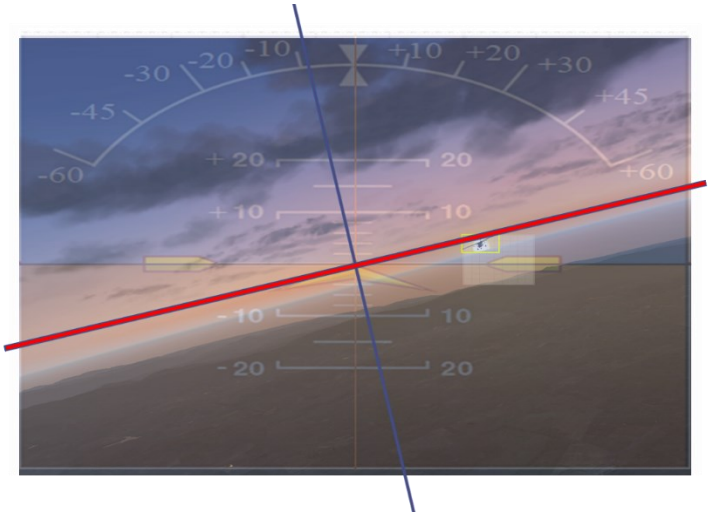

Figure E.8 We placed the cross ruler at the centroid to measure the roll angle of the horizon attitude

Measuring the Horizon Roll Angle Process:

1. **Visual Ruler Setup**: Use a red horizontal line as a ruler to capture the horizon's inclination. A perpendicular blue line helps align the PHI centroid for accurate measurement.
2. **Placing the Ruler**: Position the centre of the ruler at the PHI centroid to capture the roll angle relative to the horizontal axis.
3. **Measuring the Roll**: Observe the inclination angle of the horizon roll as indicated by the blue line. For example:
    - In Figure E.8, the horizon roll is at approximately -10 degrees.

Measuring the Horizon Pitch Angle

1. **PHI Rotation**: Rotate the PHI metric image to align with the ruler's roll angle for ease of pitch measurement.
2. **Ruler Placement**: Place the horizontal line of the ruler on the Pictorial Horizon.
3. **Measuring the Pitch**: Measure the vertical distance or inclination of the horizon relative to the ruler's position. For instance:
    - In Figure E.9, the PHI is aligned, and the horizon pitch is measured as approximately -20 degrees.

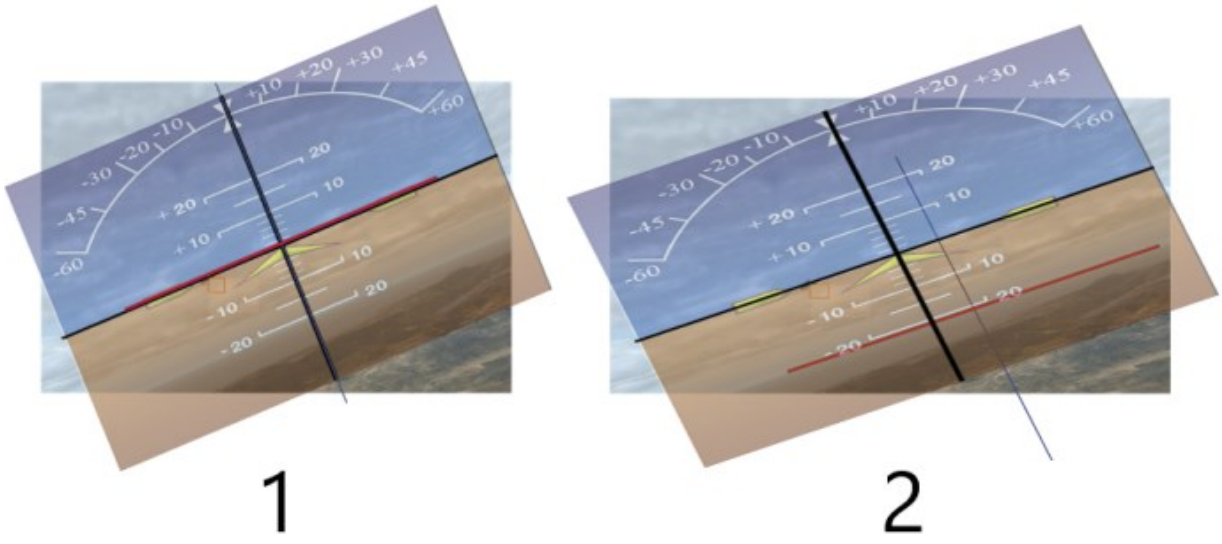

Figure E.9 1) We rotate the PHI to align it with the ruler, then 2) We place the ruler's horizontal line on the horizon.

This manual process provides an intuitive way to validate the frame horizon's roll and pitch angles in training datasets or for image analysis. It ensures that horizon attitudes are accurately recorded for calibration of the perception model. The process can be adapted for use with automated systems or tools to achieve greater precision on large-scale image datasets.

Captured image rotational movements definitions:

- Frame Roll (Camera Rotation around the front-to-back axis):

Frame Pitch (Camera Rotation around the left-to-right axis): The up/down tilt of the camera frame relative to the horizontal plane, resulting in the horizon moving higher or lower in the image.

Frame Yaw (Camera Rotation around the vertical axis): The left/right pointing direction of the camera. Yaw changes what appears in the background and can change apparent relative motion of objects across frames, even when the TOI itself remains constant.

E.7.4 How to specify horizon attitude classes in a CuneiForm

For the purpose of CuneiForm traceability, the horizon attitude should be specified as an explicit, checkable label rather than an implicit property of an image. Practically, this means assigning each image (or each CuneiForm exemplar) to a discrete *horizon attitude class* defined by roll and pitch ranges, using the Pictorial Horizon Attitude Indicator (PHI) as the reference instrument. The goal is not photogrammetric precision; the goal is *repeatable categorisation* that (i) can be applied consistently by different annotators and (ii) is informative for dataset coverage arguments.

    - **Positive Frame Roll:** The camera tilts to the observer's right, raising the image's left side and lowering the right**.**
    - **Negative Frame Roll:** The camera tilts to the observer's left, raising the image's right side and lowering the left.
- Frame pitch (Camera Rotation around the side-to-side axis):
    - **Positive Frame pitch:** The camera tilts upward, lowering the horizon line in the image.
    - **Negative Frame pitch:** The camera tilts downward, raising the horizon line in the image.

Horizon Minimum Variety of Attitude Categories:

Table E.11 categorises horizon attitudes based on defined ranges of roll and pitch values, capturing a variety of camera orientations in images or video frames. The categories include neutral views like "Level Horizon" (minimal roll and pitch) and more dynamic perspectives such as "Positively Tilted Elevated Horizon" and "Bird's Eye Ground View," which represent extreme tilts and elevations. These classifications provide a structured framework for standardising and analysing horizon orientations, which is essential for training and validating perception systems

in diverse operational scenarios. Including categories like "Rocket Sky View" highlights the adaptability of this framework for applications requiring extreme camera angles.

Table E.13 Horizon attitude categories are defined by roll and pitch ranges.

Horizon attitude training class (category)	Roll Range	Pitch Range	Visual Description
Level Horizon 	−1≤ROLL≤1	−1≤PITCH≤1	The horizon appears flat and straight across the center of the image.
Positively Tilted Level Horizon 	1<ROLL≤90	−1≤PITCH≤1	The horizon tilts downward from left to right, remaining near the center vertically.
Negatively Tilted Level Horizon 	−90≤ROLL<−1	−1≤PITCH≤1	Horizon tilts downward from right to left, centred vertically.
Elevated Level Horizon 	−1≤ROLL≤1	1<PITCH≤45	Horizon appears level but shifted upward in the frame, sky dominates the view.
Positively Tilted Elevated Horizon 	1<ROLL≤90	1<PITCH≤45	Horizon tilts left-to-right and is elevated in the frame.
Negatively Tilted Elevated Horizon 	−90≤ROLL<−1	1<PITCH≤45	The horizon tilts from right to left and sits high in the image.
Acute Angled Bird's Eye Ground View 	−90≤ROLL≤90	45<PITCH≤80	Ground dominates the frame, seen at a steep angle, with no horizon visible. Only looking at the ground from some angle.
Bird's Eye Ground View 	−90≤ROLL≤90	80<PITCH≤90	The image appears to be almost straight down; the horizon is not visible, only the terrain below.

Lowered Level Horizon 	−1≤ROLL≤1	−45≤PITCH<−1	Horizon appears flat but is low in the frame; more ground or cockpit visible.
Positively Tilted Lowered Horizon 	1<ROLL≤90	−45≤PITCH<−1	The horizon tilts from left to right and is lowered toward the bottom of the image.
Negatively Tilted Lowered Horizon 	−90≤ROLL<−1	−45≤PITCH<−1	The horizon tilts from right to left and is positioned low in the frame.
Acute Angled Rocket Sky View 	−90≤ROLL≤90	−80≤PITCH<−45	Sky dominates, no horizon visible and an upward pitch gives a “rocket” view climbing at an angle.
Ascending Rocket Sky View 	−90≤ROLL≤90	−90≤PITCH<−80	Looking nearly straight up into the sky, no horizon is visible. Like climbing up in a rocket.

To put Table E.13 in context, the following is a histogram of how images captured in AVOID dataset (our case study in Chapter 7, Appendix I) performs in terms of coverage of all possible categories of horizons:

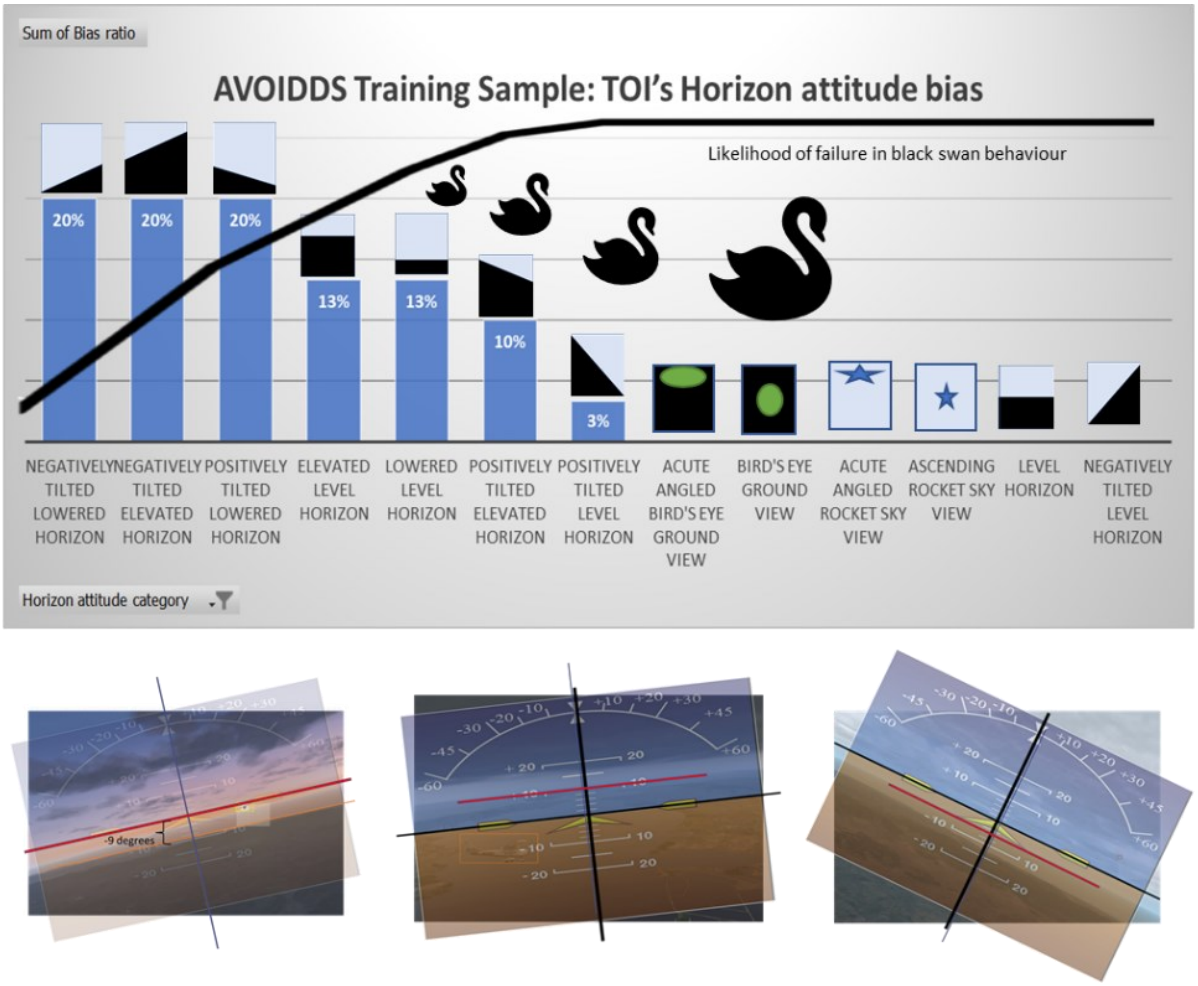

Figure E.10 Pictorial horizon coverage for AVOIDDS sample training.

Figure E.10 illustrates that several horizon categories are not represented in the AVOID sample training dataset, which comprises 30 images selected randomly from the training dataset. Furthermore, we utilise the Black Swan symbols to highlight the pictorial hazards associated with potential gaps in coverage, indicating that the dataset's reliability is compromised, particularly in its ability to manage black swan scenarios within those absent categories. Consequently, from this perspective, the AVOIDDS training dataset may be deemed untrustworthy. The black curve shown in the graph represents an indicative curve that indicates the level of distrust; we refer to it as the "architect distrust curve".

E.7.5 Topic 2: TOI's aesthetical features complexity topic

One source of pictorial hazards comes from the infinite acceptable input space of TOI's aesthetic variety of features.

The delineation of TOI pictorial features, particularly their aesthetic complexity, is a nuanced process that directly affects the reliability and safety of machine learning models. For example, a drone's shape, colours, and coating design. Since datasets impact designer comprehension, a

challenging aspect is defining how many aesthetic design features should be included in a single CuneiForm. As the dataset increases in size, the number of required CuneiForms also increases in size. This increase affects the cognitive overload of dataset handlers (during the dataset design and validation processes). We recommend describing no more than 3 aesthetic features in a single CuneiForm. We recommend, based on our experience designing cuneiforms in our case studies, and inspired by the magic no.7 rule of thumb[23].

E.7.6 Topic 3: TOI's perceived motion situations and optical effects topic

One source of pictorial hazards comes from the infinite acceptable input space of TOI's motion situations and optical effects.

The TOIs' visual motion situations and optical effects are critical for accurately depicting how an object's movement can impact image capture quality and clarity. To describe this characteristic, the following minimum variety of aspects can be defined:

- **In-Motion states:** TOIs' motion patterns constitute significant variables impacting their detectability by perception systems. For simplicity, we define the following minimum variety of situations to be considered:
    - **Still State:** The TOI is stationary relative to the environment and the camera frame. This is the simplest scenario for object detection. For example, if the adversarial drone is hovering in place, it may appear static relative to the environment.
    - **Linear Motion:** The TOI moves in a straight line across the set of captured images. For example, the adversarial drone flies straight at a speed that may cause motion blur.
    - **Rotational Motion:** The TOI is spinning or rotating. This is captured in a series of images where the TOI exhibit a rotational motion. For example, the adversarial drone performs barrel rolls or spins.
- **Optical effects:** The dynamic situation of TOIs can affect the recognisability of the pictorial situation experienced by a perception system, such as motion blur. Motion blur is an Optical effect caused by the relative motion between the camera and the object if the object's speed is high compared to the camera's exposure time. For example, high-speed adversarial drones cause blur.

E.7.7 Topic 4: TOI's Background environment complexity topic

Based on the analysis in Figure 4.6, we recognised that the complexity of the image background influences the recognisability of the TOIs. This section addresses the integration of environmental characteristics within the CuneiForm images. The focus is on how background objects and their dynamic situations affect a perception system's recognisability of the TOI.

E.7.7.1 Impact of background objects and environmental scenery on TOI recognisability

One source of pictorial hazards comes from background objects' infinite acceptable input space.

Background objects influence the perception system's ability to accurately identify and classify TOIs. Background objects can vary in complexity, ranging from simple, uniform textures to highly cluttered scenes, as perceived by an observing human architect. Such variability may obscure or confuse the perception system, leading to misidentification or non-recognition of the TOI. The challenge is to ensure that the TOI remains distinguishable despite varying background elements. This necessitates carefully selecting background features in CuneiForm images that represent real-world scenarios yet do not overwhelm the architect's ability to comprehend that a designed CuneiForm captures a requirement, and the instantiating images capture the valuable minimum variety needed.

E.7.8 Topic 5: Dynamic situations of background objects topic

One source of pictorial hazards comes from the infinite acceptable input space of Dynamic situations of background objects.

In addition to their static aspects, the dynamic situations of background objects can challenge a perception system. Dynamic backgrounds, such as moving foliage or other objects, can introduce additional uncertainty for ML components to handle. Therefore, when constructing CuneiForm images, it is critical to incorporate a range of dynamic situations for background objects. This will train the perception system to focus on the TOI and accurately interpret its features despite its presence**.**

E.7.9 Topic 6: TOI's pictorial positioning zones definition topic

One source of pictorial hazards comes from the infinite acceptable input space of TOI's pictorial positioning.

The positions of objects within images are an essential aspect of characterising the images as they can influence perception reliability (note that we said influence, not control). This approach, inspired by the zonalisation seen in ancient cuneiform tablets such as in Figure 2.9, applies a structured grid to categorise and standardise the spatial positioning of TOIs within an image. In our adaptation, we utilise nine distinct zones within each image to define the specific location of the TOIs. The rationale behind the choice of 9 We based on our experience designing cuneiforms in our case studies and inspired by the magic no.7+/-2 rule of thumb [12]. By dividing the image into nine main zones, we can systematically describe and specify the location of TOIs. This zonal approach enables a more organised analysis of complicated visual scenes with multiple objects. It also helps reduce the ambiguity that might arise from overlapping or closely situated objects.

We created a CuneiForm development canvas to help with designing and characterising CuneiForm. The positioning is categorised into 21 types (9 main zone labels in yellow and 12 intermediate optional zones). Table E.14 describes the possible categories:

Table E.14 Generic TOI positioning zones labels

down right	**center right**	**up right**
down center	**center**	**up center**
down left	**center left**	**up left**
up left/center left	center left/center	center/down center
center left/down left	down left/down center	up center/up right
up left/up center	up center/center	Center/center right
down center/down right	up right/center right	center right/down right

Figure E.11 illustrates the TOI positioning zones, which essentially act as a guide for perception architects. Each of the nine zones represents a potential location for a TOI, thus allowing for a detailed description of its position relative to the camera's perspective and other objects in the scene. We also included further intermediate zone labels to assist with describing Toi's position if they are found between zones. They are optional and depend on the need and application.

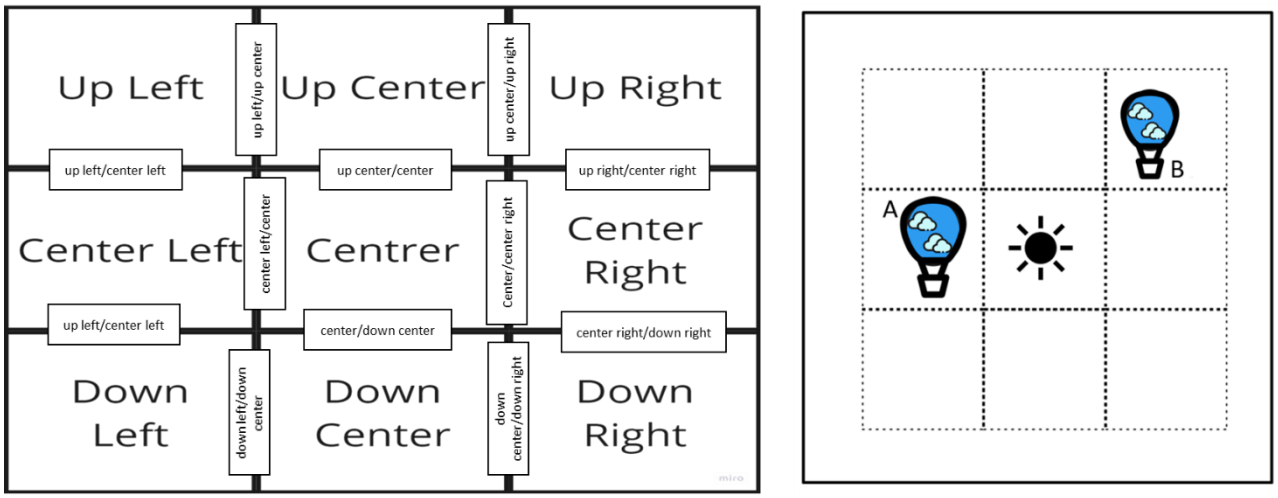

Figure E.11 TOI generic positioning zones.

The right-hand depiction gives an example of cuneiform where TOI A occupies the centre-left, and TOI B occupies the up-right zones.

E.7.10 Topic 7: TOI's 3D spatial orientation topic

One source of pictorial hazards comes from the infinite acceptable input space of TOI's spatial 3D orientation.

The spatial orientation of TOIs in three-dimensional space is an essential component of training for computer vision perception systems. The 3D orientation of TOIs can be represented by a discrete set of view angles. As a guideline, our process recommends a generic, relevant variety of no more than nine distinct view angles to describe these orientations comprehensively. However, this number depends on the application.

Additionally, our framework remains adaptable; the architect may determine a more appropriate set of 3D views tailored to the TOI's geometry. For example, in the case of a perfectly spherical object, symmetry eliminates the need for multiple orientations, reducing it to a single view. Figure E.12 below depicts our recommended set of generic 3D orientation views for any TOI. However, depending on the TOIs' geometry, the architect may adapt our recommended variety accordingly. We define the following generic views for TOIs: front, side, top, front up, front up right and left, front down, and front down right and left.

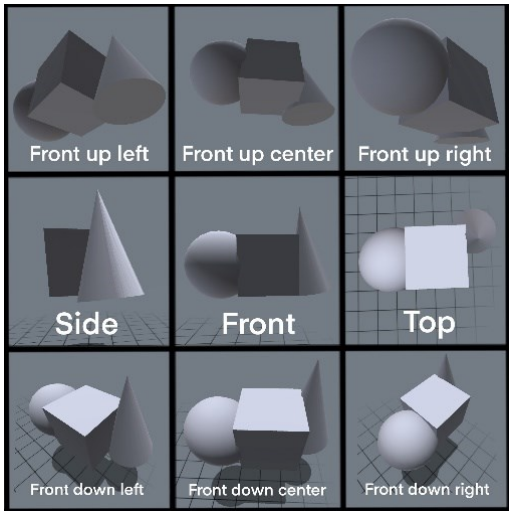

Figure E.12 shows our recommended minimum variety of possible generic 3D orientations.

In the Eagle Robot case study, developing a CuneiForm representation for an adversarial drone encompasses defining its three-dimensional orientations. This approach entails generating a set of CuneiForm representations that encapsulate all possible perspectives or views of the drone, thereby facilitating its recognition under various orientational conditions. We choose an abstract depiction of a drone and then model its orientation roughly based on the generic definition in Figure E.13.

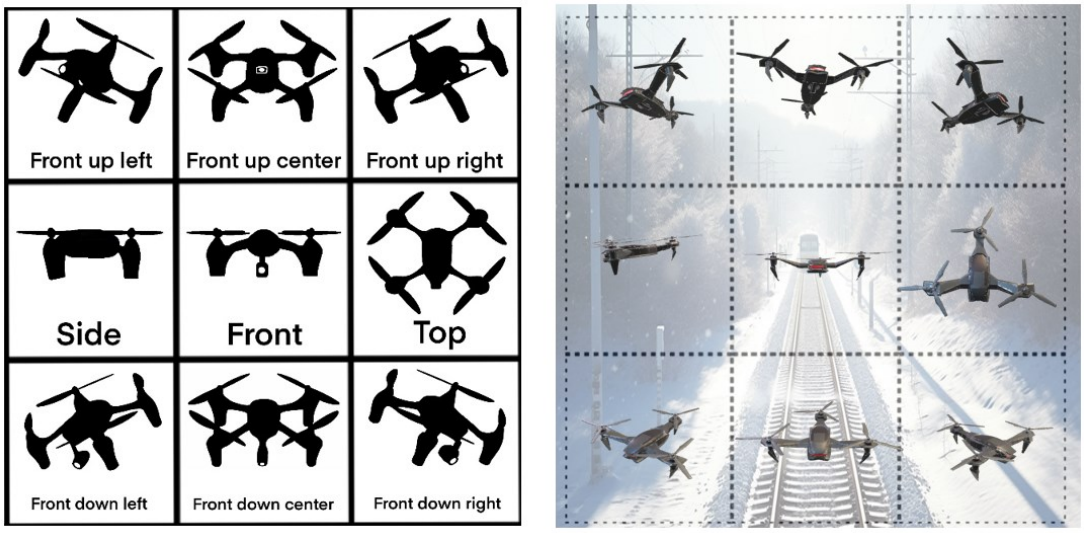

Figure E.13 Minimum variety of 3D orientation CuneiForm representations with an example image of how an instantiation of them would look like in a dataset.

For the AVOIDDS (Vision-Based Dataset for Aircraft Detection) case study, we defined 18 generic orientations since the application involved detecting intruder aircraft to avoid mid-air collisions.

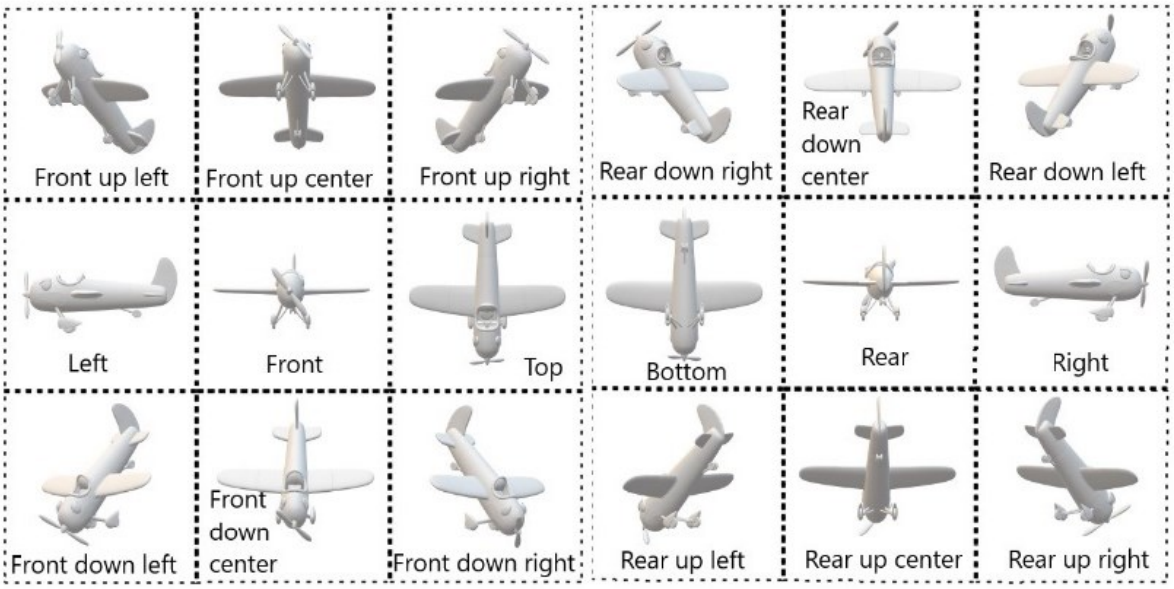

Figure E.14 Generic 18 minimum variety orientations for TOIs in the AVOIDS case study.

Figure E.14 depicts 18 distinct spatial orientations used in the AVOIDDS case study to represent the three-dimensional views of target objects (TOIs), specifically intruder aircraft. These orientations were chosen to comprehensively capture various perspectives necessary for training the perception model to detect and avoid mid-air collisions. The views include standard angles such as front, top, and side and more nuanced perspectives like front-up-left and rear-down-right, ensuring robust coverage of potential visual scenarios. This extended variety of orientations allows the system to handle diverse real-world conditions and ensures that no critical viewpoints are omitted during training and validation.

To give another example, Figure E.15 presents 18 distinct spatial orientations for a human Target of Interest (TOI), designed to comprehensively capture various 3D views required for training computer vision systems. These views include standard perspectives such as front, top, rear, and side, alongside more nuanced angles like up-front-left, bottom-front-center, and up-rear-right. This variety ensures the perception system can recognise and accurately interpret the TOI in diverse scenarios and orientations.

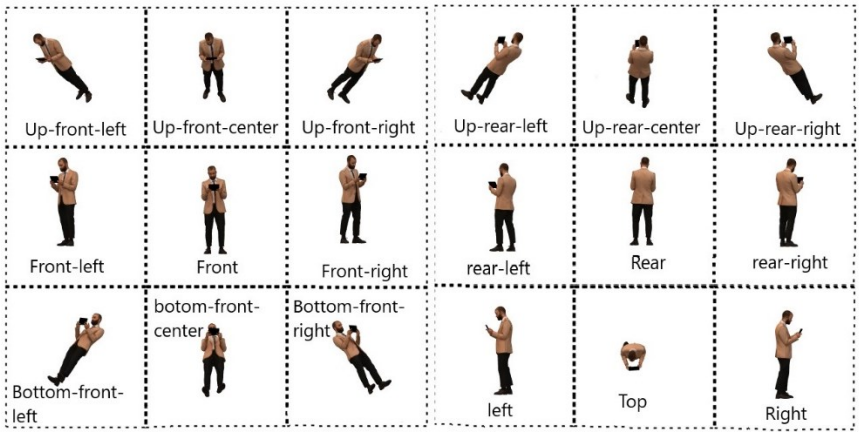

Figure E.15 Generic 18 view orientations for a human TOI example.

E.7.11 Topic 8: TOI's pictorial distance topic

One source of pictorial hazards comes from the infinite acceptable input space of TOI's pictorial distance.

Defining the distance of a TOI from a camera's focal point influences its recognizability in an image for a trained perception system. It is unclear what shape or size a TOI may be in the real world, nor are we clear about how far it will most likely be. In an open, complicated environment, all sizes and all distances are equally possible unless the operational environment is controlled. So, we had to consider a generic process specifying any TOI distance from any perception model. We define a process that quantifies this distance based on the intuition that the closer the TOI is to the camera, the larger its relative area compared to the total frame area. Thus, the pictorial area of the TOI within the camera's frame is compared to the image's total area. Depending on the requirement of how far the TOI should be recognised, the architect may specify a particular ratio.

For ease of communication and to add some precision to pictorial characterisation, we name a generic pictorial measurement of distance, represented in relative size, as *"Pictorial Nindan"* or just *"Nindan"* to describe a single unit ratio of distance as a function of a ratio of TOI's area to the frame total area. The term "Nindan" is an adaptation of a Sumerian unit that measures distance, and we reused the term in our approach for convenience and ease of specification. To measure the pictorial distance of a TOI, we do not care about how truly far it is; we only care about how the distance distorts the visual aesthetics of the object concerning the predictive observer's detection reliability. So, a Nindan captures the impact of physical distance on TOI's appearance.

So what? Let us have an example.

Suppose that we have a requirement to detect an adversarial drone at a long distance. If we are given an image with a TOI, and we need to calculate the pictorial distance of the TOI, we need a process to help us with estimating how many Nindans a TOI is far. Let us take the robot in the orange box (Figure E.16):

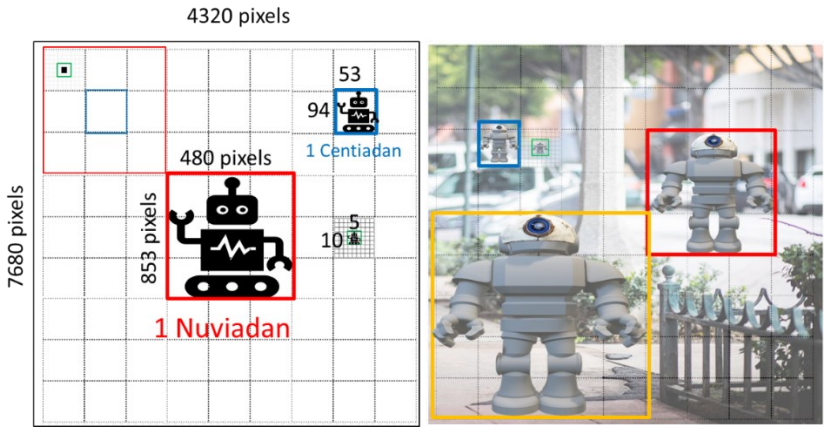

Figure E.16 Defining pictorial distances in units of Nindans. 1 nuviadan = 9 Nindans. 1 Centiadan = 81 Nindans.

The simplest way to define standard pictorial distance measurement is to divide the image into a grid of squares. We use a 9x9 (regardless of image size), which results in 81 squares. To capture smaller TOIs we keep subdividing the squares into 9x9 grids. In the case of the orange box designated as TOI (Target of Interest), we apply this 81-square grid. One *Nindan* distance is equivalent to 81 squares.

Next, we determine how many squares the TOI Occupiers; in this instance, it occupies 25 squares. This indicates that the TOI is farther than 1 Nindan.

To calculate the distance of the orange-boxed robot in Nindans, we divide the total number of squares (81) by the number occupied by the TOI (25):

$$81 / 25 = 3.24$$

Therefore, the orange-boxed robot is measured to be approximately 3.24 Nindans away. We must subdivide the image into an 81x81 grid for green-boxed TOI, which counts to 6561. Then, count the number of squares the green-boxed TOI occupies; in this case, we have 9. The pictorial distance of the green-boxed TOI is 6561/9 = 729 Nindans.

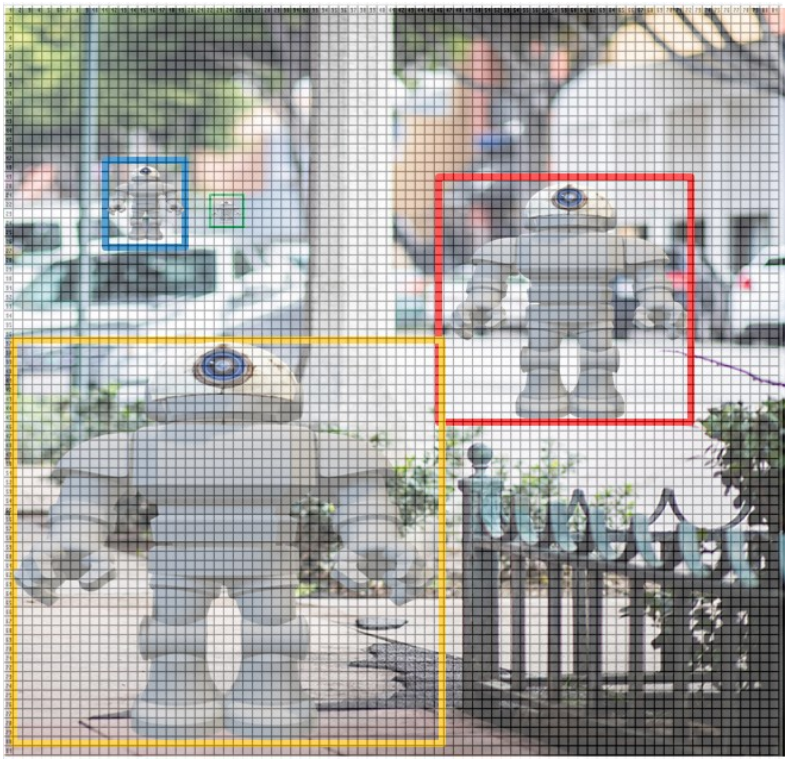

Figure E.17 green boxed TOI appearing at a Miliadan pictorial distance = 729 Nindans pictorial distance

Let us take another example: the car that appears behind, which is partially occluded.

Figure E.18 Partially occluded TOI (car)

Suppose the TOI is a car. In this case, if we are to measure how many nindans, we measure how many squares the bounding box occupies. In this case, we can manually determine the number of squares occupied: 16 (height) x 34 (width) = 544 scaled squares. To determine the pictorial distance in nindans: 6561/544 = 12 Nindans. It is worth noting that we assume the following concept:

A partially visible object is conceptually as unclear as if it were fully visible but that far.

So, for example, let us take the same image in the Figure and determine how far the robot is conceptually. The whole robot was 3.24 nindans, not only part of its head is visible. In this case, we accept the convention of “the distance of the visible part, " meaning TOIs are conceptually

as far as their visible part. We can manually count the number of squares, and in this instant: 5 (height) x 16 (width) = 80. Given 6561/80 = 82.0125 Nindans, which is nearly 25 times farther away conceptually than if it was visible entirely. See Figure E.19.

Figure E.19 Partially visible robot TOI pictorial distance of 82.0125 Nindans.

E.7.11.1 Standard pictorial distance categories topic

To simplify distance specification in a cuneiform design, we need to categorise the various class ranges. Although the pictorial distance values for classification may vary from one TOI to another, depending on the application, we will use the range specification for the aircraft mid-air collision avoidance dataset case study as an example.

Table E.12 categorises pictorial distances (Dx) into five distinct ranges based on the recognizability of a Target of Interest (TOI) for the aircraft mid-air collision avoidance dataset. These categories span from "Extremely Unrecognisable TOI Distance" (Dx > 1600 Nindans), where the TOI is nearly imperceptible, to "Dangerously Close TOI Distance" (Dx ≤ 40 Nindans), indicating critical proximity requiring immediate action. The intermediate ranges, "Moderately Recognisable," "Recognisable," and "Clear Close" distances, highlight varying degrees of TOI clarity as perceived by a human verifier or system. This classification provides a structured framework for evaluating and designing datasets, ensuring appropriate recognition capabilities across different operational scenarios.

Table E.15 Classification of pictorial distance ranges for TOI recognition in the aircraft mid-air collision avoidance dataset.

Training class (category) (wrt human verifier)	**Pictorial Distance ratio (Dx) Range**	**Example TOIs at different distances**
Extremely Unrecognisable TOI Distance	Dx > 1600 Nindans	
Moderately Recognisable TOI Distance	729 > Dx ≤ 1600 Nindans	
Recognisable TOI Distance	300 > Dx ≤ 729 Nindans	
Clear Close TOI Distance	40> Dx ≤ 300 Nindans	

Dangerously Close TOI Distance	1≥Dx<= 40 Nindans	

E.7.11.2 Pictorial distance calculation and validation

We believe that validating AI-related artefacts requires direct human intervention rather than relying on another AI system to validate a safety-critical AI system. This is because AI Processes used for verifying AI development artefacts necessitate verification of correctness. Therefore, we developed a visual grid stencil-like process to define and manually validate the distance. Although this process can be automated for the most part, fundamentally, the dataset validator must manually check how conceptually far a TOI is in an image.

Furthermore, this process can be advanced further by relying on pixel comparison. Since it is a more deterministic process, it can validate an entire dataset and present the result as a histogram of how far TOIs are considered, thus revealing another potential layer of bias. This analysis involves processing labelled images to compute a measure of pictorial distance based on the number of pixels occupied by objects in the pictures. The goal is to determine how recognisable an object of interest (TOI) appears at different distances. The general steps for defining pictorial distance are as follows (using YOLOv8 labelling format as an example):

1. Extracting Object Annotations
    - Each image has a corresponding text file containing bounding box annotations.
    - Each bounding box specifies an object's center coordinates (x, y), width, and height in a normalised format (relative to the image size).
2. Converting Annotations to Pixel Values
    - The relative bounding box values are converted to absolute pixel dimensions using the image width and height.
    - The bounding box coordinates are transformed to represent the object's actual position in the image.
3. Computing Bounding Box Area: assuming a typical square box.
    - The total number of pixels inside each bounding box is computed (width × height in pixels)
    - This represents the portion of the image occupied by the object.
4. Calculating Pictorial Distance (Dx)
    - Pictorial distance is defined as the ratio of total image pixels to the bounding box pixels:

$$Dx = \frac{Total\ Image\ Pixels}{Bounding\ Box\ Pixels}$$

- A higher Dx value indicates that the object occupies a smaller area in the image, making it less recognisable.
- Conversely, a lower Dx value means the object is larger and closer, making it more identifiable.
- If we flip the ratio, the result would be counterintuitive to the sense of far and close distance from the trainee ML model. For example, a TOI that occupies 10 pixels in a 1000-pixel image would count to 0.001. However, using the convention we propose would count to 100, which makes more sense regarding how far away from the observer.

5. Classifying Recognition Levels
    - The computed Dx value is used to categorise images into five distinct recognition classes:
        - Extremely Unrecognisable TOI Distance (Dx > 1600)
        - Moderately Recognisable TOI Distance (729 < Dx ≤ 1600)
        - Recognisable TOI Distance (300 < Dx ≤ 729)
        - Clear Close TOI Distance (40 < Dx ≤ 300)
        - Dangerously Close TOI Distance (Dx ≤ 40)

This process allows an automated, quantitative evaluation of object recognition based on the area occupied by the target in an image. The classification provides an objective measure of visibility, which can be used for further AI model training, dataset curation, and real-world object detection analysis. Table E.13, Below is an example output of analysing the pictorial distance for the AVOID training dataset sample we used for our second case study (see Appendix I). Bear in mind that the categorisation and the range are customisable based on the application.

Table E.16 Computing pictorial distance based on pixels.

Image Name	Bounding Box Pixels	Total Pixels	Pictorial Distance Dx (Nindans)	TOI pictorial distance Training Class
0.jpg	1627	5621280	5621280/1627=3455	Extremely Unrecognisable TOI Distance
1.jpg	8801	5621280	5621280/8801=639	Recognisable TOI Distance
10.jpg	4969	5621280	5621280/4969=1131	Moderately Recognisable TOI Distance

E.7.12 Traceability of images to their original descriptive text.

The concept of traceability in the CuneiForm-based system is foundational to linking pictorial elements in images to their textual counterparts. These annotations serve as a conceptual

bridge, connecting each pictorial aspect of a CuneiForm image to its corresponding descriptive representation. The process involves assigning a unique numeric annotation to every feature visualised in the CuneiForm image. This annotation directly correlates with the specific aspect of the written description it represents. Conceptual annotations are captured in between braces {x}. Sometimes, additional refinement aspects are depicted with respect to a parent aspect. The annotation may follow the format of {x.y} to capture refinements. A potential validation process is to agree or disagree with the references cited in the CuneiForm, as accurately captured in the description.

E.8 Comprehensive Multi-Environment Operational Environment Definition (Multi-ODD)

The comprehensive definition of a Multi-Environment Operational Design Domain (Multi-ODD) is critical in reducing architects' epistemic uncertainty and enhancing lateral and vertical predictive thinking in the context of autonomous systems. By establishing a structured taxonomy of natural and man-made features, the multi-ODD framework provides a systematic approach for identifying and contextualising environmental variables that interact with Autonomous Systems (AS). This standardisation aims to support the architect in objectively exploring and specifying a broad spectrum of scenarios.

To address the limitations mentioned in the literature review section 2.1.7, a general ODD definition process may be helpful to standardise defining operational domain taxonomies. We needed to describe a general process to help us standardise multi-environment ODD for our train tracks case study. We did not find a standard that considers both air and train tracks operational domain. In general, we envisage expanding the multi-ODD for various types of AS; those operating in sea, air, space, or land domains should encapsulate a range of abstract features to ensure comprehensive coverage. These features are categorised into two main taxonomies: Natural and Man-made related features.

E.8.1 General ODD Definition Process

Here is a general ODD characterisation process that defines the minimum information needed for a comprehensive ODD:

Step 1: Define natural surrounding ODD taxonomy:

1. **Dynamic or cyclic changing natural classes**: Factors that include elements that move or change position. Also, it includes cyclic state-based phenomena.

Examples: birds, fish, wind currents, seasonal changes (spring, summer, autumn, winter), tidal changes.

2. **Static natural classes:** Classes of factors that include relatively stationary natural elements complex.

Trees, rocks, and coral reefs are examples. In the space domain, this may include the sun, moon, or any relatively stationary celestial objects (including far stars in the background).

3. **Natural surrounding noise and interference classes:** Natural ambient noise and interference refer to background sounds and signals originating from natural sources that can impact an AS's functionality. These noises and interferences are typically omnipresent and can vary significantly depending on the operational domain.

Example: an underwater drone encountering communication difficulties due to dolphin echolocation clicks or the noise from underwater volcanic activity.

Step 2: Define manmade surroundings taxonomy:

4. **Informational environment classes:** Classes of factors that include the set of informational signals and data streams generated by human-made systems that assist or influence the operations of an is.

Example: a GPS navigation system providing turn-by-turn directions to a vehicle.

5. **Dynamic or changing engineered classes:** Classes of factors that are human-made elements that move or change position.

Example: traffic signals (changing from red to green), cars, trains, aircraft, cultural events.

6. **Static or static engineered classes:** Classes of factors are human-made elements that remain stationary.

Examples: bridges, buildings, roads, dams.

7. **Classes of man-made surrounding noise and interference:** Classes of factors that include man-made engineered noise and interference encompass unwanted signals or disruptions generated by human-made systems that can impact the functioning of an AS.

Example: These can include electromagnetic interference (EMI), radio frequency interference (RFI), acoustic noise, and other disturbances.

These categories ensure that relevant aspects of an AS's environment are considered, allowing for safer and more effective operation within its designated domain.

E.8.2 Comprehensive Multi-ODD that combines Land and Air-based AS applications

Traditional ODD frameworks for autonomous road vehicles often overlook the complexity and variability of other environments. By systematically defining dynamic, static, and noise-related factors in both natural and manmade contexts, we ensure that a wider range of potential challenges and operational constraints are addressed. This comprehensive consideration is crucial for designing safety into Autonomous Systems, as it allows for anticipating and mitigating a broader array of risks.

For complete taxonomy, refer to Appendix A. The general ODD process outlined above was systematically applied to create a comprehensive ODD framework encompassing land and air-based AS. This comprehensive ODD framework ensures that all relevant aspects of the operational environment are considered, allowing for the discovery of failure modes and Black Swan Scenarios and, thus, safer, and more effective AS operations within their designated domains. The following elaboration details how each step of the general ODD process was applied to combine land and air-based AS applications.

E.8.2.1 Step 1: Define Natural Environment ODD taxonomy:

Dynamic or cyclic changing natural classes: The process involved identifying natural factors that are constantly moving or periodically evolving to account for dynamic or cyclic changing natural classes. These factors are crucial for understanding the environmental variability that AS must adapt to.

- **Weather Conditions Variety:** This includes a comprehensive range of weather conditions such as temperature, precipitation, wind, humidity, pressure, and visibility. For instance, temperature variations can significantly impact the performance and efficiency of AS, particularly those with sensitive electronic components or battery dependencies.
- **Time of the Year Seasons-specific Changes**: Seasonal changes, including spring, summer, autumn, and winter, bring distinct environmental conditions that affect AS operations. For example, winter may introduce snow and ice, obstruct pathways and reducing sensor accuracy, while summer may present heat waves affecting cooling systems.
- **Illumination Variety Definition**: This includes variations in natural light conditions such as sunrise, sunset, twilight, and night. Additionally, it considers the positioning of

celestial bodies like the moon, which affects visibility and navigation for both land and air-based AS, particularly during night-time operations.

Static, unexpected, or non-cyclic changing natural classes: These classes include natural elements that remain relatively stationary or do not follow a predictable cycle but still influence AS operations.

- **Landscape Types Variety**: The variety of landscapes, such as urban, suburban, rural, mountainous, forest, desert, coastal, water, industrial, and arctic landscapes, is considered. Each landscape type presents unique challenges and considerations for AS, such as the need for urban navigation algorithms or rugged terrain handling capabilities in mountainous regions.
- **Geographical Region-specific Natural Phenomena**: This includes phenomena unique to specific regions, such as atmospheric phenomena (e.g., northern lights, monsoons), hydrological phenomena (e.g., floods, tsunamis), geological phenomena (e.g., earthquakes, volcanic eruptions), and biological phenomena (e.g., animal migrations).
- **Perceived Pictorial Horizon Attitude**: This involves the perception of the horizon as captured by AS sensors, which includes image rotational movements like frame roll and pitch. Understanding how the horizon is perceived helps calibrate sensors and ensure accurate navigation and stability.

Natural surrounding noise and interference classes: Natural ambient noise and interference can significantly impact the functionality of AS. These factors are typically omnipresent and vary depending on the operational domain.

- **Natural Auditory Noise**: This includes noise from wind, animal sounds, vegetation movement, and thunderstorms. Such auditory noise can interfere with acoustic sensors and communication systems, necessitating robust filtering and noise-cancellation techniques.
- **Natural Non-audio Interference** includes electromagnetic interference (EMI) from natural sources such as solar flares and terrain-induced interference. Such interference can disrupt electronic systems and communication links, requiring AS to be equipped with shielding and error-correction capabilities.

E.8.2.2 Step 2: Define Manmade Environment Taxonomy

Dynamic or Changing Engineered Classes: This step focuses on human-made elements that move or change position, affecting AS operations.

- **Road, Train, and Air Traffic Conditions**: Dynamic traffic conditions require AS to adapt to varying speeds, densities, and flow patterns in real-time.
- **Dynamic and Impermanent Road, Train, and Aerospace Objects of Interest**: This includes temporary obstacles, construction zones, and other transient objects that AS must detect and navigate around.
- **Socio-Political Factors**: Events like cultural festivals, political gatherings, and public demonstrations can create dynamic changes in the operational environment, requiring AS to adjust routes and operational plans.

Informational Environment Classes: This step involves identifying the set of informational signals and data streams generated by human-made systems that assist or influence AS operations.

- **Road Transport Connectivity**: This includes Vehicle-to-Vehicle (V2V) and Vehicle-to-Infrastructure (V2I) communication systems that provide real-time traffic information, navigation guidance, and safety alerts.
- **Railways Transport Connectivity**: This includes communication systems between trains (Train-to-Train) and between trains and wayside infrastructure (Train-to-Wayside) for coordinated operations and safety management.
- **Air Travel Connectivity**: This includes communication systems for airborne vehicles, such as Airborne Vehicle-to-Infrastructure (V2I) and Airborne Vehicle-to-Airborne Vehicle (AV2AV) communications, essential for air traffic management and collision avoidance.

Static or Static Engineered Classes: This step includes human-made elements that remain stationary and form the consistent landscape within which AS operates.

- **Road, Train, and Aerospace Physical Infrastructure**: Permanent structures like bridges, buildings, roads, dams, rail tracks, and airports provide a stable framework for AS navigation and operations.
- **Static and Permanent Road, Train, and Aerospace Objects of Interest**: This includes road signs, signal lights, and fixed landmarks essential for navigation and operational reference.

Manmade Surrounding Noise and Interference Classes: This step addresses unwanted signals or disruptions generated by human-made systems that impact AS functionality.

- **Land-based Noise and Interference Factors**: This includes electromagnetic interference (EMI), radio frequency interference (RFI), and acoustic noise from urban environments, industrial zones, and transport systems.
- **Air-based Noise and Interference Factors**: This includes interference from radar systems, communication signals, and other airborne sources that can disrupt the operations of UAVs and other airborne AS.

We have created a comprehensive framework that covers relevant aspects of land—and air-based AS operations by applying the general ODD process. This framework ensures that the unique requirements and challenges of these domains are addressed, enhancing the effectiveness of ODD for Intelligent Systems design in such domains.

E.8.3 ODD Aleatoric Uncertainty grading

In section 4.1 we explained the direct relationship between randomness in the physical environment (aleatoric uncertainty) and impact on epistemic uncertainty (ignorance) for the architect and the machine.

To effectively integrate safety into the training and testing of autonomous systems, we require a straightforward process for identifying what constitutes an "unsafe" operational design domain (ODD). This understanding will enable us to evaluate the criticality of various environments and inform the design process accordingly.

The rationale behind the grading system for ODD is based on the recognition that environments are complicated complexes where confusion and uncertainty can arise for autonomous systems and their architect. As the ODD conditions become hard to predict (confusing complicatedness), the likelihood of encountering unexpected environmental behaviours also grows. In other words, the more energetic the ODD, the greater the aleatoric uncertainty, the greater the architect's epistemic uncertainty about the resilience of their assumptions, heightening the potential for Novel events and safety challenges (Black Swan events). See Appendix B for a full definition of the uncertainty Grading System.

E.8.4 Breakdown of the Grading System for ODD Uncertainty

Appendix B outlines an Operational Environment Uncertainty Grading System designed to support ML component safety training for autonomous systems like the Eagle Robot. The framework categorises operational environments into five grades (A to E), ranging from Grade A ODD Uncertainty (Ideal, assuming normal-complex conditions) to Grade E ODD Uncertainty (Not Favourable, assuming Stochastic Confusion conditions). These grades are based on

various environmental characteristics, providing a systematic approach to evaluating and modelling an ODD's uncertainty levels for AS design and validation purposes.

E.8.4.1 Key Components of the Grading System:

The following is a structure of how grading criteria can be integrated as part of ODD definition:

1. **Solution System and Operational Space**:
    - Example: The Eagle Robot's typical area of operation (e.g., a train track zone) serves as the reference operational space.
2. **Environment Characteristics**:
    - Environments are characterised by natural and contextual factors such as lighting, weather, and geographical features.
3. **Grading Schema**: The following are the assumptions made about the degree of aleatoric uncertainty involved. The more energetic and extreme the ODD, the more uncertain and chaotic it becomes. The more potentially chaotic the ODD is, the more potential for Black Swan scenarios, thus increasing the likelihood of facing an utterly Stochastic Confusion field for the architect and their trainee machine.
    - **Grade A ODD Uncertainty**: Ideal conditions assuming relatively minimal uncertainty, such as sunny weather, clear visibility, and low environmental variability. The least ODD uncertainty of the grading system.
    - **Grade B ODD Uncertainty**: Favourable conditions with slight variations, such as partly cloudy skies or moderate wind speeds.
    - **Grade C ODD Uncertainty**: Partly favourable conditions with noticeable variability, such as mixed sun and clouds or moderate precipitation.
    - **Grade D ODD Uncertainty**: Less favourable conditions with significant uncertainty, such as overcast skies, strong winds, or low visibility.
    - **Grade E ODD Uncertainty**: Not favourable conditions where operational challenges and risks are maximised, such as severe storms or foggy conditions. The highest ODD uncertainty in the grading system.
4. **Environmental Factors and Grading Criteria**:

Grading can be used to describe a change in ODD conditions.

- Natural Lighting Conditions: Transition from sunny (Grade A) to overcast or hazy (Grade E).
- Weather Conditions: Graded by precipitation, wind speed, humidity, and visibility.

- Time of the Year: Seasons-specific environmental characteristics.
- Landscapes type variety definition.
- Geographical region-specific natural phenomena.
- Time of the Day.
- Perceived Horizon Attitude.
- Sun sphere positioning.
- Moon sphere positioning.
- Specialised zone features.

E.8.4.2 Purpose and Application

The Uncertainty Grading System serves as a foundational tool for:

- **Designing and Training ML components specialised in a particular grade:** Ensures datasets capture a wide range of environmental uncertainties.
- **Risk Assessment and Safety Assurance**: Enables systematic evaluation of environmental factors to anticipate potential operational challenges.
- **Operational Context Definition**: Establishes a clear framework for characterising the complexity and uncertainty within the ODD.

E.9 Assuring the reduction of risk during Novel scenarios

Developing an ML model through a training pipeline is one thing; developing a safety-defensible model with empirical evidence of the safety of its emergent properties, such as intelligence, in Novel scenarios is another.

The primary reason for using ML models (e.g., for perception) is to leverage their powerful capabilities to operate correctly in Novel operational scenarios that may arise during development. Here is the catch:

Part of being "Novel" is demonstrating that a scenario is correctly Novel! Subsequently, the model has been trained correctly for it.

The second catch related to this challenge is the assurance of safety:

ML model's emergent generalisation shall reliably and correctly[11] emerge safely during those Novel scenarios.

[11] By correctly, we mean to correctly manifest the desired properties (the architect's intent).

Traditionally, in deterministic safety-critical systems, we manage the risk of missing requirements through rigorous systematic approaches (such as SHARCS) to achieve completeness. However, when developing ML models, our intuition changes markedly; we do not worry about the majority of missing operational requirements because we expect the model to perform as intended for unaccounted-for operational requirements, at least those that are similar to the training and validation data. Nonetheless, we still need to demonstrate that safety properties correctly emerge during those Novel scenarios.

Similarly, it is one thing to develop datasets and ensure they enable the safe emergence of expected behaviours in Novel scenarios (within the problem domain but not mentioned in training, validation, or testing) from the trained model; it is another. In consideration of Novel scenarios, we account for both adversarial and natural data shifts resulting from changes in operational domain complexity.

Assurance is the systematic process of ensuring that the desired properties are reflected in the system's actual, concrete, and tangible behaviour. However, the assurance of emergent intelligence considerably relies (for the most part) on the intangible information stored in its dataset in pictorial format. Here, the topic of certifiability of required safe emergent intelligence becomes directly relevant and highly challenging when it comes to convincing a regulator to trust a safety-critical AI system.

Hence, the topic in this section directly relates to the challenge of ML-model robustness certifiability in the face of Novel perturbations relative to training and testing datasets, for example PROVEN framework[23]. PROVEN technically aims to show that, if we consider only random perturbations from a known distribution, we can often obtain much larger certificates: the model is correct with probability ≥ some value over a measurable distance from a specific reference or threshold.

White-box Certified Robustness Approaches include two main types[24]:

- **Robustness Verification:** Certifies that a model's prediction remains unchanged within a perturbation bound.
- **Robust Training:** Trains models to maximise this certified robustness.

In our black-box approach to certifiability of robustness, we rethink the problem in a different light, and view it as:

> **Defensibility of a design and validation evidence about ML model behaviour in Novel scenarios to reduce a competent human regulator's uncertainty and increase their trust.**

If we view it this way, it means that any systematic approach in demonstrating emergent properties, regardless of how mathematically rigorous, will help reduce a regulator's uncertainty and increase their trust. We would welcome a multi-dimensional approach, combining white and black-box processes to achieve this purpose. In our research, we focus on providing a systematic and holistic approach to achieve this goal. In the context of pictorial datasets, we propose some holistic terms encompassing the types of Novel scenarios, which help us define different strategies to validate performance. We summarised the types of Novel pictorial datasets in our conclusion chapter.

Our primary concern is to provide a demonstrable and defensible systematic approach for compliance with regulatory standards (for example, CAA SORA [19]) when making claims about the extension of safety properties beyond the performance over test datasets (i.e., Novel data). We approach the problem from a black-box perspective. We do not claim that this is the ideal approach; rather, our black-box approach complements any white-box approach to provide the necessary defensibility of claims regarding the provision of empirical guarantees of performance in a limited number of pictorial situations outside the test datasets.

Concepts in this section are based on our experiments conducted in sections 7.2 and H.10. We learned from these experiments that to ensure the safety of ML components, we need to address the challenge of guaranteeing the emergence of safe intelligence in Novel scenarios outside the scope of training and testing datasets. So, if we examine the case of ensuring emergent properties emerge in Novel scenarios (we call them Black Swans), we need a development process that clearly demonstrates a systematic discovery of Novel scenarios and that they are mitigated in training and validation.

We learned the latter principle from our experiments in translating requirements into a pictorial training syllabus. This section provides a theoretical foundation that we used to develop the process in Stage 6 of the approach (see Section 7.2).

The most challenging aspect is constructing an objective, convincing process that can demonstrate model performance during Novel scenarios. Our process in stage 6, and especially the approach in step 7 (section 7.2.8), aims to provide a proposed solution for ensuring the preservation of safety requirements during Novel scenarios. Our experiments in Section 6.8 taught us a great deal about what “Novel” means at a pictorial level. The concrete doing of the training syllabus happens at the level of images and the uncertainty of information. This is why we cannot simply rely on the claim that Stage 4 provides satisfactory evidence for deliberate risk reduction during Black Swan conditions.

Our approach towards objective evidence of reducing the risk of unsafe operations during Novel scenarios (Black Swan operations) is based on the following principles:

E.9.1 Motivation: Mandatory CAA Assurance Robustness

The Civil Aviation Authority (CAA), through the JARUS Specific Operations Risk Assessment (SORA) framework, establishes a structured approach to evaluating the trustworthiness and safety assurance of unmanned aircraft systems (UAS). A central concept in this framework is "robustness," defined as the combination of integrity (how effectively a risk mitigation or operational safety objective reduces risk) and assurance (the ***confidence*** that this effectiveness is reliably achieved). The SORA outlines three levels of robustness, low, medium, and high, based on a matrix that correlates integrity and assurance levels. Notably, robustness is constrained by the lower of the two dimensions; for instance, medium integrity combined with low assurance results in low overall robustness.

The expected assurance levels range from self-declared compliance with limited verification (low assurance), to evidence-backed claims using testing or operational data (medium assurance), to third-party validated compliance reports (high assurance). This structured expectation for development process rigour and evidence-based compliance reinforces the need for systematically demonstrating UAS safety and trustworthiness in complex airspace operations.

The key notion in SORA robustness requirements lies in the definition of "confidence," which refers to the Regulator's Confidence that the supplied evidence demonstrating compliance indeed achieves the expected quality (for example, high assurance). To be more precise, SORA explicitly associate certainty to the level of assurance,

> "The degree of certainty with which the level of integrity is ensured".

Achieving certainty in the effectiveness of a risk mitigation process, in this context, is rather ambiguous and hard to quantify since it depends on the assessor's point of view. Another challenging quality SORA introduces is the quantification or qualification of "how good".

> "Level of integrity (i.e., how good the mitigation/objective is at reducing risk)"

Ensuring that risks are minimised during Black Swan operations, along with other scenarios within the operational domain of the machine learning (ML) model that were not considered during the design phase, presents the architect with two main challenges when it comes to providing evidence of compliance with a high level of robustness.

1. The challenge is to convince the regulator's sense of confidence (certainty) that the training process indeed developed and assessed the model during the Black Swan scenario. This can extend to some residual scenarios that are not part of the training and testing.

2. Convince the regulator that the CuneiForm syllabus is good at reducing the risk during Black Swan scenarios.

For that, we understand the above challenges as follows:

1. **Satisfying Regulator's sense of Certainty in the effectiveness of training process to enable safe ML behaviour during Black Swan operations:** We understand the term "certainty" to be related to reducing the assessor's epistemic uncertainty so that there is minimal doubt about the developer's approach to reducing a particular risk or achieving a specific safety goal. To achieve a convincing argument, the architect can follow the following principles:

    d. Black Swan scenarios have been separately considered and predicted**.**

        - **So what?** We dedicated Stage 1 to distil the problem domain (prior to the introduction of the solution system into the problem complexity field). Then, in Stage 4, we further enhance the approach to discover more Black Swan scenarios, with the solution being part of the problem domain.

    e. The process of predicting those Black Swan scenarios is scientifically rigorous.

        - **So what?** We introduced STCoT processes to justify the architect's engineering judgment of why a Black Swan scenario is such and how to tackle it. STCoT demonstrates a clear application of lateral and vertical thinking to predict Black Swans.

    f. They are traceable to the datasets.

        - **So what?** We introduced CuneiForm syllabus development and validation to demonstrate that datasets can be traced to safety requirements.

2. **Demonstrating to the regulator how good the mitigation is at reducing risk during ML-training process (at dataset level):** to elevate this challenge, the architect would need to demonstrate that the risk of failure during Black Swan

operations has been reduced through a systematic, evidence-based trial-and-error approach**:**

d. The architect needs to demonstrate to the regulator that the model training process has been systematically training and testing, thus improving an initial configured model on a series of Black Swans and that the model clearly has improved in performance during Black Swan operations.

- **So What?** We adapted the **SHARCS** process principle of a systematic and incremental approach to introducing capability into systems design. We treated Black Swan scenarios as additional capabilities for the system and incrementally introduced an initially trained and tested model to Black Swan scenarios.

- We assessed the model's ability to handle such scenarios, and in the case of success or failure, we demonstrated deliberate training and assessment. With this approach, we demonstrate that discovered Black Swan scenarios are indeed challenging situations for a model configuration, and that our training log shows consistent improvement in various Black Swan scenarios.

To support the above approach, we need to clarify a number of key concepts:

E.9.2 Novelty and Data Distribution Shift

In a pictorial sense, we interpret **a data distribution shift** as meaning the training and validation strategy has never encountered a similar (exact copy or somewhat the same) scenario. This can be demonstrated by showing that there is no pictorial depiction in the training and validation sets that statistically resembles the pictorial scenario in question (OOD). A novel pictorial scenario means:

Pictorial Novelty: following the definition of novelty in section 3.13, a pictorial dataset is said to be novel if it is demonstrated to be relatively Original and Atypical:

Original: The pictorial dataset is claimed to be original when demonstrated to be out of distribution OOD with respect to a dataset that has been demonstrated to be a typical operational scenario (White Swans). To measure OOD, the dissimilarity measure of datasets can be used.

Atypical: the pictorial datasets are traced to requirements which are traced to an atypical situation or interaction discovered during the design. It can also mean an atypical operational

scenario, for example, if the project team defines regular quad-copters as the typical operational scenario for a perception system to detect, then any drone in any other shape would be classed as not a typical (atypical) operational scenario. In the case study, a camouflaged adversarial drone is considered an atypical operational scenario.

So, how do we produce a defensible argument that helps a regulator trust that we have intentionally addressed in our design?

In Section H.10, our experiment shows that it is possible to provide a systematic, defensible argument that demonstrates performance in white ghost scenarios. We need to demonstrate to the regulator that:

- We have followed an objective, systematic approach in predicting normal operational scenarios, and we used those to initialise training of our model (we use stage 3 to do so).
- We then followed a separate systematic process to predict Black Swan, non-typical operations (stage 4 of our process).
- We need to demonstrate that we tested the model over OOD data with respect to some typical operations, training, and validation (white swans) to demonstrate "data-shift of pictorial scenarios". These are the Black Swans if they can be traced to Black Swan scenarios.
- We then need to show that we have systematically included those Black Swans into the initial white swan training process.
- We then need to test the new, trained model (White + Black swans) on another dataset that is in-distribution with CuneiForms.
- We also need to demonstrate that the model is trained and tested over Novel scenarios. These are out-of-context images.

Thus, in-and OOD similarity testing provides an approach for demonstrating training and performance during data shifts in Black Swan scenarios. Systematic audit train of constructing a dataset is a useful demonstration that at a given time of stepwise building of the dataset, it can be said that the training and validation have or have not seen an additional dataset distribution using.

E.9.3 Reduction of epistemic ignorance ALARP

Now we understand the nature of the Novel scenario at a pictorial level. With that in mind, we need a systematic audit trail that clearly exposes those sources of ignorance and shows that they have been accounted for in the training process. With that in mind, we developed our approach, outlined in Table 7.12. The approach shows clearly that there have been a audit trail of discovering OOD related to real world data shift (Black Swan CuneiForms) and that a

concurrent configuration trained model failed at it, but we included it systematically and now we have a model that clearly trained to perform and preserve safety properties in Novel scenarios because we have a long history of training in those scenarios.

E.9.4 The risk of unmitigated OOD data

Unmitigated OOD data, which we understand to be associated with a model's epistemic ignorance, arises when a model is confronted with data or scenarios that were not represented during training. In such cases, the model's performance may degrade substantially, manifesting in underperformance or outright failure on OOD or out-of-context (OOC) datasets. Demonstrating that a dataset is genuinely OOD is a necessary but not sufficient condition for establishing confidence in generalisation; it is equally important to incorporate out-of-context (OOC) data, as this provides additional empirical evidence regarding the model's behaviour under unfamiliar conditions. However, one must be cautious not to rely solely on OOC datasets as a proxy for all Novel scenarios, since the space of possible data shifts is effectively unbounded.

To provide empirical evidence of unmitigated data shifts, a systematic, incremental process is useful. Rather than aggregating all available images or samples at once, the training regimen should be structured so that data are introduced progressively. Maintaining a detailed training logbook that clearly shows a regulator how unmitigated OOD and out-of-context (OOC) examples were taken into consideration becomes indispensable in this context, as it provides a chronological record of model iterations, hyperparameter configurations, and evidence of performance under Novel conditions. More importantly, it needs to document instances in which a candidate model fails to generalise, thereby revealing potential sources of data shift. In practice, this requires demonstrating empirically that a particular input distribution or environmental condition makes the model's predictions unreliable. A configured model that experiences such failures during testing can then be retrained on the newly identified distribution, retested, and the results recorded. Through this cycle of testing, failure analysis, and retraining, one can accumulate empirical evidence of epistemic ignorance and systematically expand the model's coverage of the data manifold.

From an assurance standpoint, it is crucial to retain instances of model failure on OOD or OOC data, as these failures serve as objective indicators of residual epistemic uncertainty. By cataloguing these occurrences, one can explicitly qualify the extent to which the training and validation sets were insufficient to capture the full variability encountered in deployment. In turn, this enables practitioners to articulate, in clear terms, where the model's ignorance lies and to what degree it has been mitigated. If repeated iterations demonstrate persistent failures

in specific regions of the input space, despite progressive retraining, it becomes evident that simply augmenting the dataset may not be sufficient. Instead, this insight may motivate alternative approaches, such as revising model architecture, domain adaptation techniques, or targeted data collection campaigns aimed at previously underrepresented conditions. Ultimately, maintaining a transparent record of these failures underpins a rigorous argument regarding the remaining OOD risk and guides strategic decisions to fortify model generalisation.

E.9.5 Adapting the SHARCS process for Black Swan-driven training pipeline

Safety assurance processes aim to clearly and unambiguously demonstrate that errors in a practitioner's engineering judgment have been identified and remedied appropriately, as observed by a regulatory body. In other words, the ultimate goal of any safety assurance process is to reduce the following uncertainties:

- The architect's epistemic uncertainty.
- The qualifying regulator's epistemic uncertainty.

One profound approach for the safety-defensibility of safety-critical software is SHARCS. It handles the reduction of epistemic uncertainty related to the preservation of safety properties during failures in a few ways:

- Using the STAMP-inspired deviation of the control actions process.
- Using Event-B proof obligations.
- Logical and provable traceability of subsystem-level requirements to system-level requirements.
- Provision of clarity through incremental increase of complexity (abstraction).

Using those four methods, SHARCS helps the assurance process in two dimensions:

- The architect realises any hidden ignorance about the design by discovering vulnerabilities in their design decisions due to incomplete engineering judgment.
- The regulator's ability to be in no doubt that the architect approached the design systematically, discovering flaws and mitigating them, accordingly, thus increasing trust in the system.

Essentially, SHARCS prompts the architect to model a level of abstraction in control structure. By the end of that step, the architect ceases the abstraction modelling when they sense that they know enough about the system. SHARCS does not stop here; it then prompts them again to challenge their own epistemic certainty through the STAMP process of predicting deviations from expected control actions. The architect then realises their ignorance and that they may have overlooked further actions and iteratively updates their knowledge about the system

design and the problem. It seems as if SHARCS tacitly makes the designer predict what he thinks about what a complete solution should be, then it injures the architect's sense of confidence in their mental model using STAMP deviations guide words and offers to remedy this broken sense of confidence with more requirements.

Therefore, in general, the assurance part of this process boils down to SHARCS' ability to provide a systematic audit trail, in which the practitioner realises where they were wrong or missed something, mitigates it, and incorporates it back into the design requirements. Eventually, those mitigation plans (in AMLAS terms called Safe Operating Concept) will manifest themselves at the concrete code or hardware level.

However, for that to happen, we need an ontology that bridges the gap between high-level STAMP deviations, mitigation, and code. Event-B provides such an ontology. In code, the same process applies; we have code checkers that discover semantic and syntactic bugs, which technically alert the software engineer to sources of pragmatic errors or oversights in their engineering judgement that they may have missed. In hardware engineering, HAZOP uses a similar approach through guide words. The practitioner's ignorance is then exposed and remedied through mitigation plans.

From such articulation, we understand safety assurance in general at a concrete level to be a process that unambiguously demonstrates the following:

- Sources of ignorance had been exposed and validated to be legitimate errors.
- Remedies had been accounted for and unambiguously included in the design.

Therefore, if we wanted to concretely assure a model behaviour in Black Swan or Novel operations domain, we need to be able to demonstrate the following:

- Sources of pictorial ignorance had been isolated and validated as legitimate missing information.
- Remedies had been accounted for in the dataset by their unambiguous inclusion in the training process.

The question then forces itself:

- How do we demonstrate exposure of pictorial ignorance at the dataset level?

Similar to what SHARCS does for the practitioner, we demonstrate the reduction of risk during Black Swan scenarios in the ML training process at the dataset development stage. While the architect (in SHARCS) seeks to discover ***deviations*** from ***expected*** control actions, the architect in our training approach seeks to identify those pictorial subsets ***whose distribution deviates*** from ***expected operations*** datasets, assess model performance to check how truly challenging those OOD subsets are, and then systematically and incrementally mitigate them

by increasing the complexity of the training and testing dataset. The following are the main concepts we learned through our experiments in section H.10 and H.11:

E.9.6 Systematic, incremental pipeline to Black Swan training and testing of ML models

The following are the main steps of our approach to demonstrate that a model has been trained to handle Novel scenarios. We tailored this process in section 7.2:

1. **Initial Training and In-Distribution Validation ($A_1 \rightarrow A_2$):**

   1.1. Assemble **A_1**, the in-distribution training set, by sampling data that typifies normal operations (the "White Swans"). This data originates from Stage 3 of our overall process, in which we enumerate standard operational scenarios.

   1.2. Train the model on A_1 and evaluate its performance on A_2, a disjoint holdout of the same distribution. Successful performance on A_2 establishes a baseline of competence under known conditions.

2. **Generation of the First Black Swan Test Set (B_1):**

   2.1. Identify a distinct operational scenario that the model has never encountered, one that lies outside the A_1/A_2 distribution. For example, consider a set of "CuneiForm" images that exhibit attributes radically different from those in A_1 (e.g., lighting, texture, or structural anomalies).

   2.2. Curate **B_1**, a collection of such Black Swan images, explicitly designed to probe the model's capacity to handle rare or anomalous events. B_1 is traced to "Black Swan scenarios" as described in Stage 4 of our process.

3. **Isolated Black Swan Evaluation:**

   3.1. Without mixing B_1 with the original in-distribution test set (A_2), evaluate the A_1-trained model on B_1. By preserving B_1 as a separate test set, we ensure that any performance degradation directly evidences an OOD failure rather than statistical noise.

   3.2. Record all relevant metrics, accuracy, precision, recall, within the training logbook. This logbook entry documents the introduction of a Black Swan scenario and the scrutiny of the model's response (success or failure).

4. **Outcome Analysis and Iterative Refinement.**
    - **Case 4a: A_1-Trained Model Succeeds on B_1.**

      4a.1. If the model performs at or above an acceptable threshold on B_1, then we have empirical evidence that it can generalise beyond A_1/A_2 to at least this particular Black Swan scenario. One may record this result as partial compliance with robustness requirements for data-shifted situations.

      4a.2. To further reinforce robustness, we incorporate B_1 into the training set (forming $A_1 \cup B_1$) and retrain the model. This "inoculation" step ensures that the model internalises features of B_1 rather than merely demonstrating a fortuitous pass.

      4a.3. Generate a new test set, $\mathbf{B_2}$, composed of additional images reflecting the same Black Swan characteristics (e.g., another CuneiForm variant). Evaluate the B_1-trained model on B_2.

        - If performance on B_2 remains satisfactory, we consider the Black Swan class (as represented by B_1/B_2) effectively learned. Record these findings, along with versioned data, hyperparameters, and metrics, in the training logbook. Cease further iteration for this class and compile the evidence for compliance demonstration.
        - If performance on B_2 degrades below the threshold, we have detected a generalisation gap within the Black Swan class. We then include B_2 (or those B_2 samples on which the model failed) into the training corpus (now $A_1 \cup B_1 \cup B_2$), retrain, and generate $\mathbf{B_3}$ (a fresh batch of the same class). Repeat this cycle until the model meets performance requirements on B_n, at which point we conclude that the model has effectively learned the Black Swan class in question.
    - **Case 4b: A_1-Trained Model Fails on B_1:**

      4b.1. Failure at this stage is equally informative: it indicates that the original training regime (A_1 only) did not capture this Black Swan phenomenon, thereby revealing an "unmitigated Black Swan" scenario.

      4b.2. Proceed identically to Case 4a, Step 4a.2 onward: add B_1 to the training set, retrain, and evaluate on a new test subset B_2. Continue iterative refinement

($B_2 \rightarrow B_3 \rightarrow \ldots$) until the model's performance on the Black Swan class satisfies the predetermined robustness criterion.

5. **Extension to Context Shifts (Out-of-Context Testing):**

5.1. The procedure above demonstrates resilience to distributional shifts within the same operational context (i.e., variations of CuneiForms). To address **contextual shifts**, where the entire scenario differs in a semantically meaningful way, we perform a parallel set of experiments using out-of-context (OOC) datasets. These datasets may be drawn from open-source repositories or synthetic scenarios that represent an entirely different operational domain (for example, an unrelated object class or a simulated environment with different lighting, background, or physical constraints).

5.2. As with Black Swan evaluation, maintain an OOC test set (C_1, C_2, ...) distinct from A_1/A_2. Evaluate the model on C_1, record successes or failures. If the model fails, incorporate C_1 into training, retrain, and generate C_2, iterating until the model reaches acceptable performance on C_n.

This loop, OOD Black Swan and OOC testing, retraining on similar (in-distribution) data, and re-testing, demonstrates a deliberate and methodical design. It shows that the model has been repeatedly exposed to Novel inputs and then fine-tuned using similar examples. This process builds a defensible argument for extending the model's safety claims beyond the original test set. It aims to provide regulators with evidence that the model is not only trained for known scenarios but also has built-in resilience for unfamiliar yet statistically similar ones.

If the model later fails in a live operational scenario, we can investigate whether the failed image is similar to any past training or test data. If it is not, this could reveal hidden biases or gaps in our dataset, which we acknowledge as necessary for future research, although it falls outside the current scope of this PhD.

In summary, although some of these "friendly ghost" scenarios have not been directly tested, our systematic Black Swan approach provides a defensible, evidence-based reason to trust that the model can handle them safely and as intended.

What gives us confidence in the good performance of the friendly Ghost Pattern?

The training syllabus enables the model to perform as expected on an in-distribution dataset in experiments 1, 2, and 9.1. In those example experiments, a considerable portion of the test dataset was in-distribution with the training and validation. We did not find any in-distribution

dataset, in all our experiments, where the trained model did not perform as expected. The challenge primarily involved OOD datasets. It needed a journey for us to build trust over such data shifts. This means, in theory, if we have a test set made up of OOD images, then we can also expect that any missing test data (Ghost Patterns) which are in-distribution with the testing images that are OOD with our training and validation, we can be reasonably assured that the model performance extends to them too, but only them!.

However, where there are friendly Ghost Patterns, there are also many unfriendly Ghost Patterns. These are images and scenarios that are OOD with our testing, training, and validation. We could start bagging or collecting those unfriendly Ghost Patterns by producing more data, conducting similarity tests with the entire existing dataset, and then testing the model over them. However, such research can be done in the future, not part of this PhD for now. Busting Ghost patterns would be an interesting investigation in the safety assurance of AI systems.

E.9.7 Safety Regulation Compliance Justification

This section demonstrates why the Systematic, Incremental Approach to Black Swan Training and Testing (Section E.9.6) constitutes suitable objective evidence for claims about model performance and safety preservation under limited Novel (OOD Black Swan and OOC) scenarios and thus meets the assurance requirements of safety-critical regulations.

E.9.7.1 Premises: What the Regulator Requires

Typically, regulators in safety-critical domains insist on the following foundational assurance:

Traceable and Systematic Evidence: Every development and test activity must be fully auditable. Datasets, model versions, experiment configurations, logs, and performance metrics must be versioned, timestamped, and cross-referenced to specific design requirements or risk scenarios. For example: **UL 4600 "Standard for Safety for the Evaluation of Autonomous Products"** requires that "Traceability of training and testing data to ODD coverage". Also, in section 8.5.4 "Machine learning-based functionality shall be acceptably robust to data variation," UL4600 mandates the following:

> " a) Description of the method for and results from evaluating the functionality robustness of machine learning-based aspects of item:
>
> 1) Mitigation of risks due to any lack of robustness

b) Description of method for and results from evaluating response to distributional shifts:

1) Mitigation of risks due to distributional shifts of data

c) Field engineering feedback regarding performance when “surprises” have been encountered”

So, how does our approach in E.9.6 clearly demonstrate compliance with the UL4600 requirement?

By beginning with **Step 1**, training on in-distribution data (A_1) and establishing a performance baseline on A_2, we create a **reference** against which any Black Swan dataset can be determined to be OOD using similarity measures. Subsequent steps (E.9.6.2–E.9.6.5) then ensure that any model failure on a Black Swan test is:

1. **Identified and Quantified:** Failures are logged with metrics, demonstrating where the model’s epistemic blindness lies.
2. **Remediated Through Iteration:** Each failure triggers a retraining cycle with the newly discovered OOD samples until performance meets the robustness criterion.
3. **Documented in a Traceable Manner:** Every data collection, model build, and evaluation result is recorded in the logbook, satisfying the regulator’s demand for auditable evidence.
4. **Extended to Contextual Variations:** By explicitly testing on C_j (out-of-context sets), we ensure that the model’s resilience encompasses not only distributional but also contextual shifts. We can evidence that a set of images represents examples of context shift, by showing that out-of-context images do not comply with any CuneiForm.

Consequently, our “Systematic, Incremental Approach to Black Swan Training and Testing” provides a **comprehensive, evidence-based demonstration** that:

- The model’s behaviour under Novel scenarios is not assumed but empirically validated.
- Data collection, model versioning, and performance metrics are fully traceable.
- Risk mitigation measures for distributional and contextual shifts are systematically invoked and documented.
- The process aligns with UL 4600 requirements regarding robustness evaluation, risk mitigation, and field feedback for “surprises”.

Thus, this approach furnishes regulators with the **objective evidence** needed to substantiate claims of safety preservation and robustness under both known and Novel scenarios.

E.10 Wholeness and Oneness Theory

"I believe that our understanding of wholeness and oneness is the most important topic in the systems field. Everything else is dependent on how we interpret, understand, and enact it

My take is very simple. The reason I came to systems was that it helps me be more sure that I take into account all the factors that affect what I am trying to do. So, this alone gives me my view of what wholeness is - *everything that affects or is affected by what I am trying to do.*

Complex behaviour is also very simple. How do I characterise general systems - what I am trying to do? The answer for me is " *purpose*". So now I have a very simple definition of complex behaviour that includes external phenomena and the component aspect.

A system is a set of typical relationships between a purpose
and everything that affects or is affected by that purpose.

Now we come to ask what we mean by - *affects or is affected by*. We have to accept that any purpose is conducted in a context, which means (a component and external phenomenon), so a component behaviour based on purpose has to understand both purpose and context. I have defined context as consisting of three relationships to the components' purpose:

1. That component can control.
2. That component can't be controlled but can be influenced.
3. One component can neither control nor influence but can only appreciate it.

But once I found these three relationships, I found that not only do they apply to the context they apply also to purpose So we have three types of purpose:

1. Goal - that you can control
2. Values - that you can't control but can influence
3. Ideals – that you can neither control nor influence but can only appreciate.

Now we come to the relationship between wholeness and oneness. This is given at the most general level by the principle that the whole is in every part and every part is a whole.

Every whole is a context and every whole can be divided into its three parts. There are nine possible combinations of context and purpose. There are nine possible relationships between wholeness and oneness. So every system and every system of systems can be divided into sets, levels, or dimensions of coherent A I C relationships.

In practice, we have found that any purposeful system needs at least five dimensions of AIC relationships. Five dimensions give requisite variety to wholeness and the three AIC relationships together give oneness to purpose. A table illustrating these five dimensions of AIC relationships is akin to a periodic table of system relationships".[25-27]

List of References

[1] N. Moller and S. O. Hansson. (2008, June 6) Principles of engineering safety: Risk and uncertainty reduction. *Reliability Engineering & System Safety*. 798-805.

[2] H. Shimodaira, "Improving predictive inference under covariate shift by weighting the log-likelihood function," *J. Stat. Plan. Inference,* vol. 90, no. 2, pp. 227–244, 2000.

[3] H. Daumé and Marcu, D., "Domain adaptation for statistical classifiers," *J. Artif. Intell. Res,* vol. 26, pp. 101–126, 2006.

[4] R. Senge and e. al, "Reliable classification: Learning classifiers that distinguish aleatoric and epistemic uncertainty," *Inf. Sci,* vol. 255, pp. 16–29, 2014.

[5] V. Vapnik, "Principles of risk minimization for learning theory," *Adv. Neur. Inf. Process Syst.,* vol. 4, pp. 831–838, 1992.

[6] N. N. Taleb, *The Black Swan*. Harlow: Penguin Books, 2008.

[7] G. J. Klir, *Facets of Systems Science*. New York: Springer Science+Business Media, LLC, 2001.

[8] J. Lechner, N. Schlueter, and A. Fahrner, "Empirical Study for System Development in a VUCA-World: Development of a Resilient and Sustainable Method for Risk and Technical Change Management in Automotive Industry,," in *2023 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*, Singapore, Singapore, 2023, 2023.

[9] J. Bonnie, "Towards a theory of engineered complex adaptive systems of systems," in *2018 Annual IEEE International Systems Conference (SysCon)*, Vancouver, BC, Canada, 2018.

[10] T. Aven, "Implications of black swans to the foundations and practice of risk assessment and management," *Reliability Engineering and System Safety,* vol. 134, pp. 83-91, 2014.

[11] E. Pat ´e-Cornell, "On “Black Swans” and “Perfect Storms”: Risk Analysis and Management When Statistics Are Not Enough," *Risk Analysis,* vol. 32, no. 11, pp. 1823-1833, 2012.

[12] H. Giles, "A rigorous tool-supported methodology for assuring the security and safety of cyber-physical systems," UNIVERSITY OF SOUTHAMPTON, SOUTHAMPTON, 2019.

[13] D. Kritzinger, *Aircraft System Safety*. Woodhead Publishing, 2006.

[14] *Systems Engineering Handbook*. INCOSE, ISBN: 978-1-119-81431-3, 2023.

[15] S. G. Pendse, "Ethical Hazards: A Motive, Means, and Opportunity Approach to Curbing Corporate Unethical Behavior.," *Journal of Business Ethics,* vol. 107, pp. 265–279, 2012.

[16] V. Kocevski, J. A. Valdez, B. K. Derby, Y. Q. Wang, G. Pilania, and B. P. Uberuag, "Predicting and accessing metastable phases," *Materials Advances,* vol. 4, no. 4, pp. 1101-1112, 2023.

[17] T. Levin, "Tesla's Full Self-Driving tech keeps getting fooled by the moon, billboards, and Burger King signs," ed: Insider, 2021.

[18] G. Gorelik, "Bogdanov's Tektology: Its Nature, Development and Influence.," *Studies in Soviet Thought,* vol. 26, no. 1, pp. 39–57, 1983.

[19] J. Pearl, "The Logic of Structure-Based Counterfactuals," in *CAUSALITY: Models, Reasoning, and Inference*. New York: CAMBRIDGE UNIVERSITY PRESS, 2000, pp. 209-210.

[20] G. Mobus, Kalton M., *Principles of systems science*. London: Springer, 2015.

[21] A. Bubic, D. Yves von Cramon, and R. I. Schubotz, "Prediction, cognition and the brain," *Front. Hum. Neurosci.,,* vol. 4, no. https://doi.org/10.3389/fnhum.2010.00025, 2010.

[22] J. Nelson, "Why to Add Noise to Images for Machine Learning. Roboflow Blog," ed: Roboflow, 2020.

[23] "Horizon," ed: wikipedia.org.

[24] T. L. Saaty and M. S. Ozdemir, "Why the magic number seven plus or minus two," *Mathematical and Computer Modelling,* vol. 38, no. 3-4, pp. 233-244, 2003.

[25] T.-W. Weng and e. al., "PROVEN: Certifying Robustness of Neural Networks with a Probabilistic Approach," arXiv:1812.08329, 2018.

[26] L. Li, T. Xie, and B. Li, "SoK: Certified Robustness for Deep Neural Networks," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2020.

[27] W. Smith, "LESI SIG Workshop Part 1 - Advances in Systems Sciences ", ed: ISSS, 2022.